



Teach your computer to see objects on bioimages with Ilastik

Nicolas Peredo & Benjamin Pavie

Practicalities

- For the ones that have not done it. Follow the instructions on https://github.com/vib-bic-training/Ilastik_Course to install Ilastik, FIJI and Ilastik and Morpholibj plug-ins
- WIFI: If you do not have access to campusroam or eduroam access with the info on the right.
- Let's configure Ilastik executable and memory
- Download the datasets

WI-FI FOR VISITORS

Wifi voor bezoekers

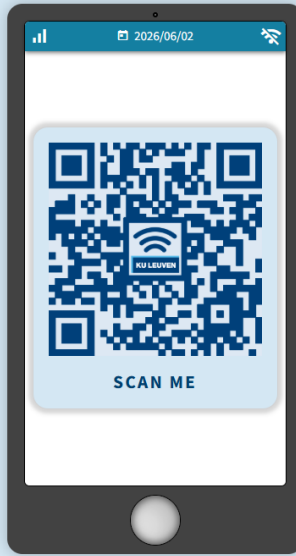
Scan de QR-code om verbinding te maken met het wifi-netwerk. Dit zal u automatisch verbinden met het netwerk zonder dat u het wachtwoord handmatig hoeft in te voeren.

Heeft u geen QR-scanner op uw toestel? Geen probleem! Hieronder vindt u de netwerknaam en het wachtwoord voor ons wifi-netwerk dat u kunt gebruiken om handmatig verbinding te maken.

NETWORK / NETWORK:	iold
WACHTWOORD / PASSWORD:	McKPG74
GELDIG TOT / VALID UNTIL:	2026/06/07 03:00

To connect, simply scan the QR code on this page. This will automatically connect you to the Wi-Fi network without the need to manually enter the password.

Don't have a QR scanner on your device? No worries! Above you'll find the network name and password for our Wi-Fi network that you can use to manually connect.



GitHub Repo



Overview

- Introduction to image analysis
- Supervised machine learning
- Introduction to Ilastik
 - Training a pixel classifier from image to batching and results
 - Training an object classifier
- Using BioImage Model Zoo NN in Ilastik (demo)
- Working with multi-scale data (demo)

Getting everything ready

- For the ones that have not done it. Follow the instructions on https://github.com/vib-bic-training/Ilastik_Course to install Ilastik, FIJI and Ilastik and Morpholibj plug-ins
- WIFI: If you do not have access to campusroam or eduroam access with
- Let's configure Ilastik executable and memory
- Download the datasets

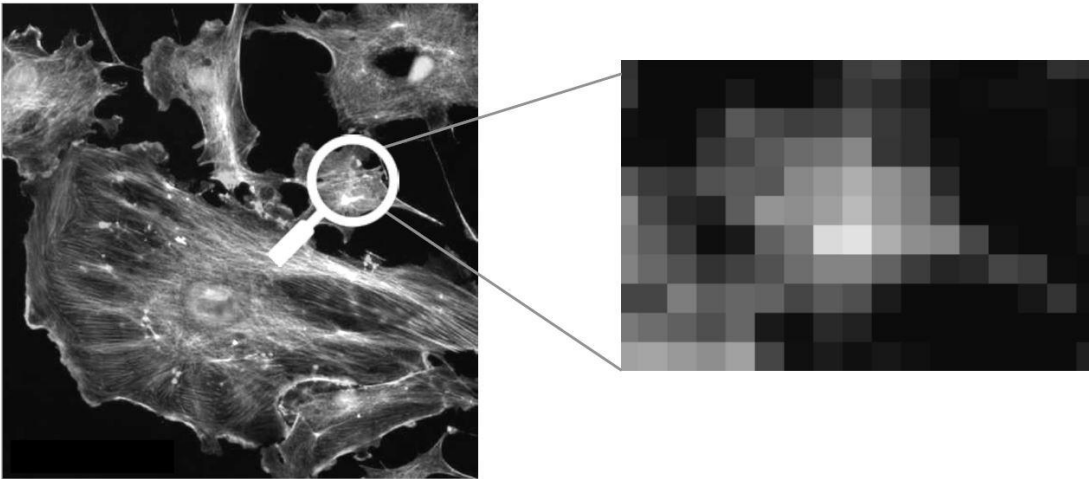
Introduce yourself

- Name & function?
- Experience with image analysis?
- Image analysis question?

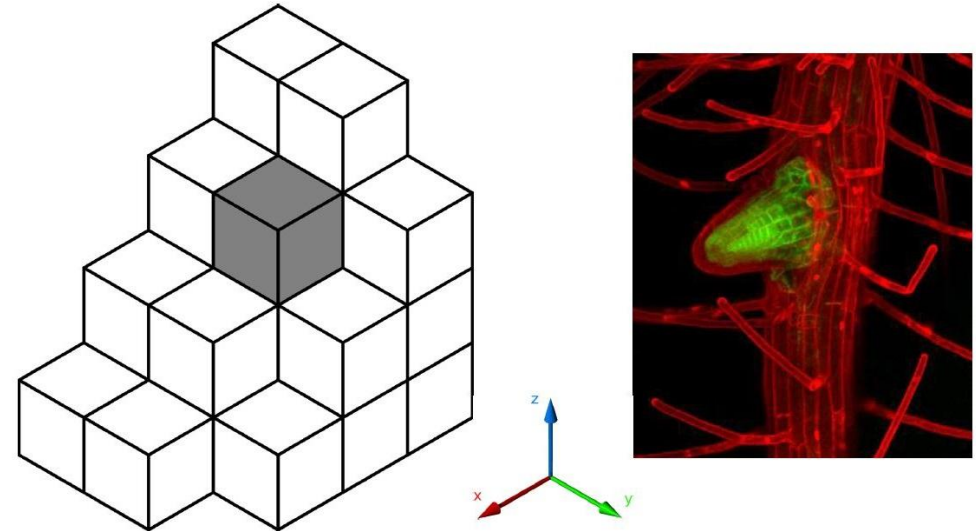
01 Introduction to Image Analysis

An image contains pixels/voxels

2D = pixels

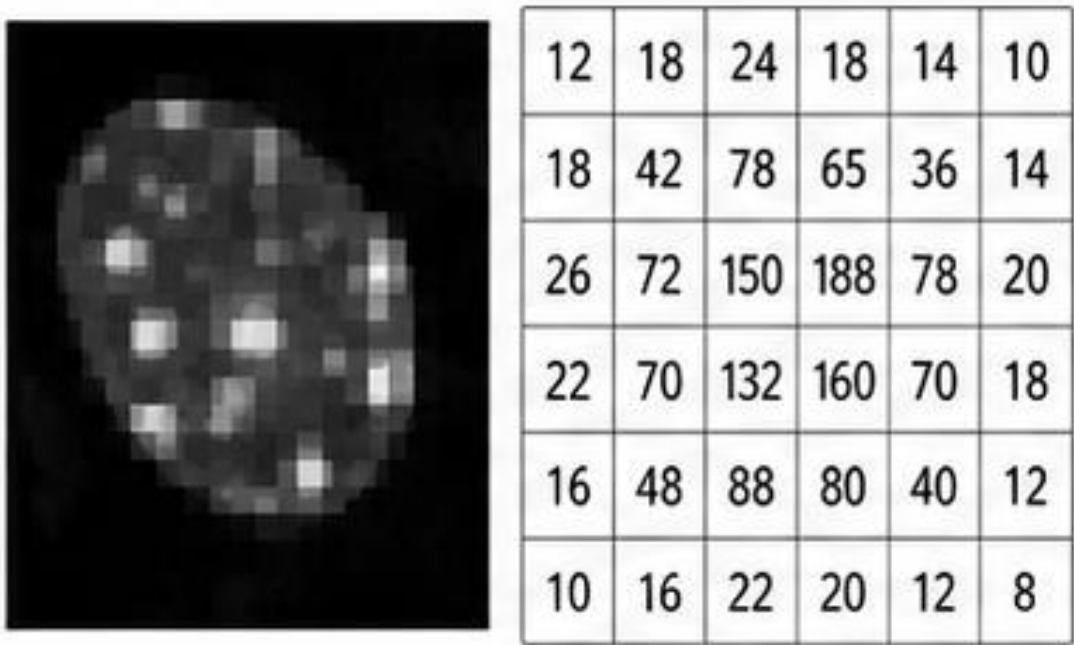


3D = voxels



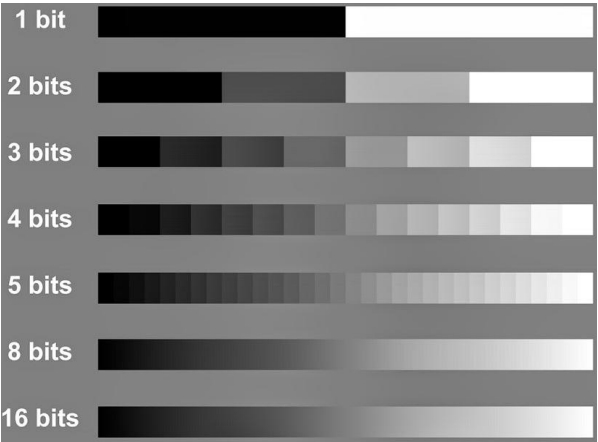
Pixel intensity and bit depth

Each pixel has an intensity value and a coordinate



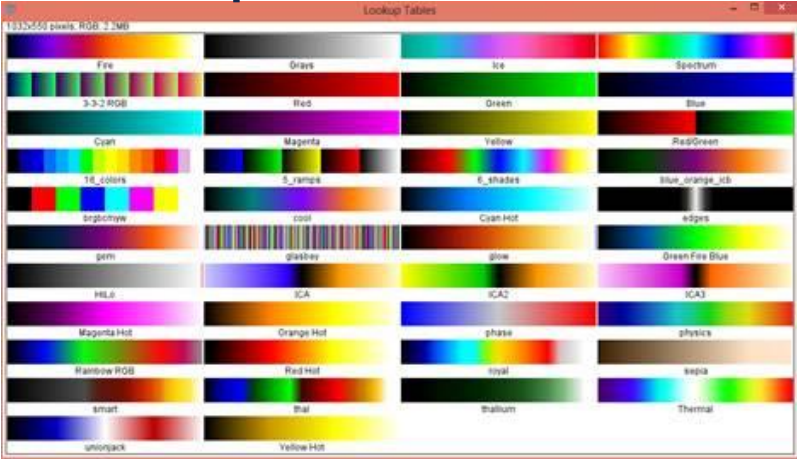
Grayscale intensity values (0–255)

Bit depth



2^n possibilities

Look-up tables



From sample to data

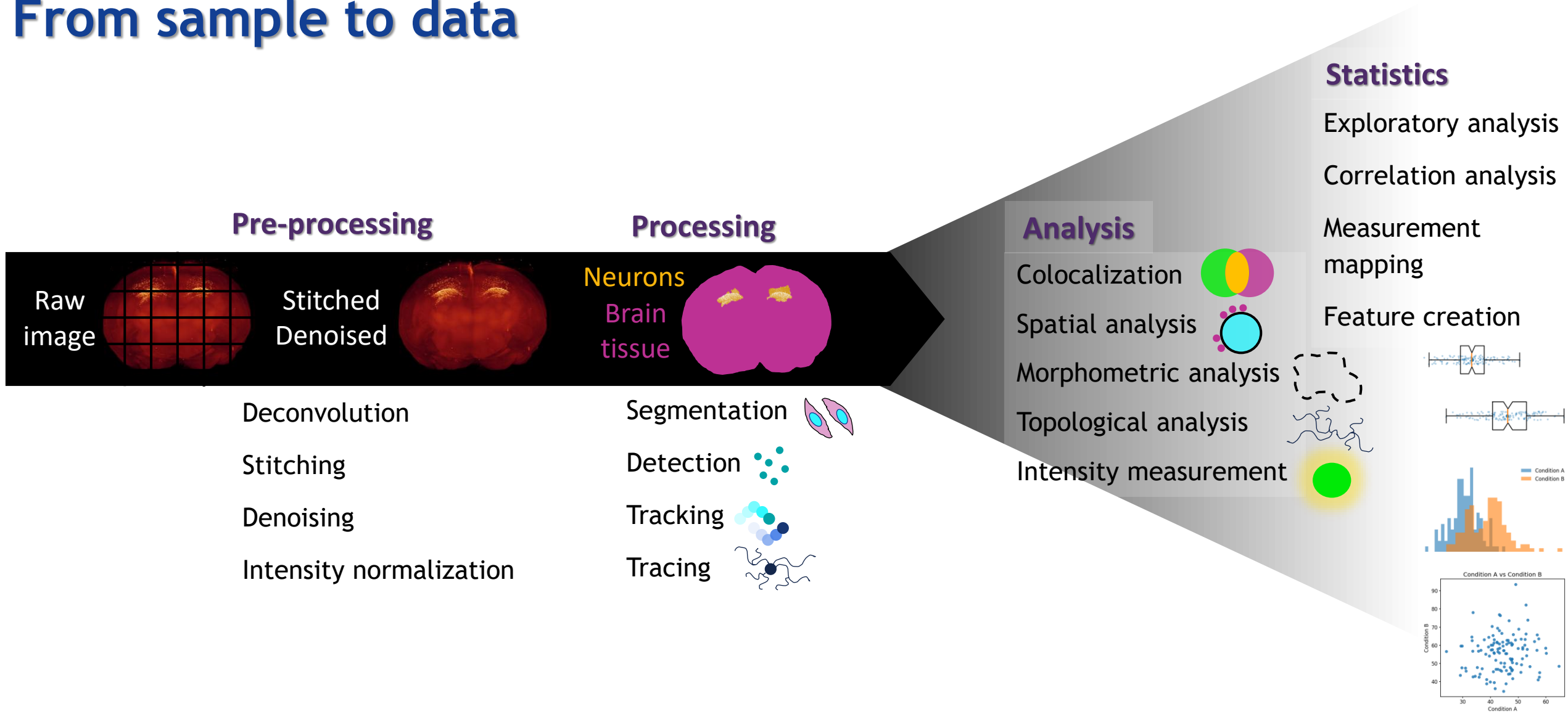
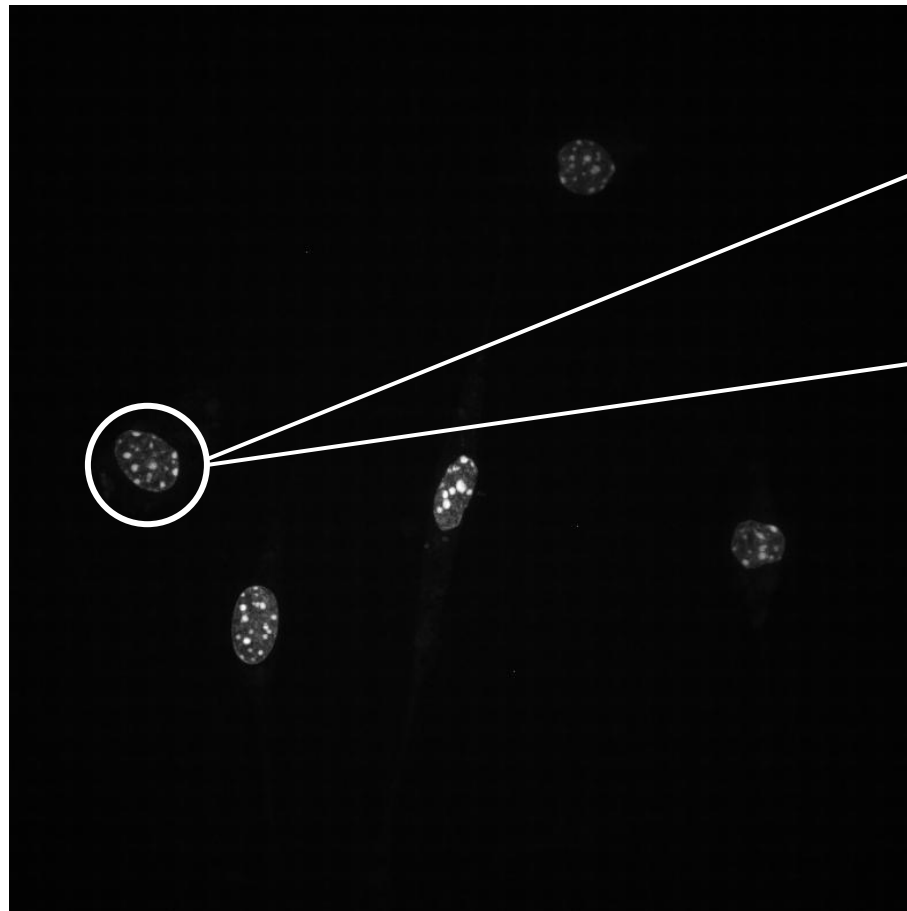
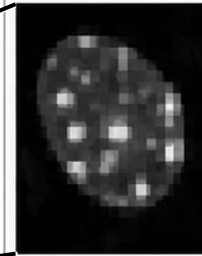


Image segmentation: from pixel intensities to labels

Source microscopy image



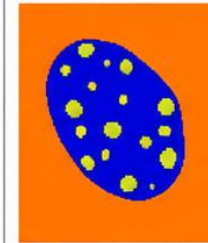
1. Raw intensities



12	18	24	18	14	10
18	42	78	65	36	14
26	72	150	188	78	20
22	70	132	160	70	18
16	48	88	80	40	12
10	16	22	20	12	8

Grayscale intensity values (0–255)

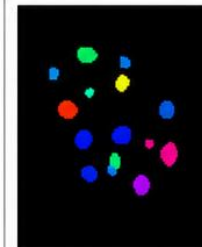
2. Semantic segmentation (classes)



0	0	0	0	0
0	1	1	1	0
0	1	2	1	0
0	1	1	1	0
0	1	2	1	0
0	0	0	0	0

0 = background
1 = nucleus
2 = nucleoli

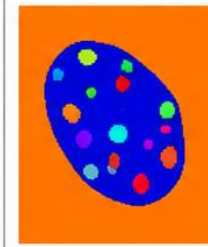
3. Instance segmentation



0	0	0	0	0	0
0	1	0	2	0	0
0	3	0	4	5	0
0	0	6	0	7	0
0	8	0	9	0	0
0	0	0	0	0	0

0 = background
1, 2, 3... = individual nucleolus instances

4. Panoptic segmentation



0	0	0	0	0
0	10	10	10	0
0	10	21	10	0
0	10	10	22	0
0	10	23	10	24
0	0	0	0	0

0 = background
10 = nucleus (semantic / stuff)
21, 22, 23... = individual nucleolus instances (thing)

Summary: Raw intensities are the original pixel values. Semantic segmentation assigns one class label per pixel. Instance segmentation assigns a unique ID to each object instance. Panoptic segmentation combines both: stuff classes (e.g., nucleus) and thing instances (e.g., nucleoli) in a single label map.

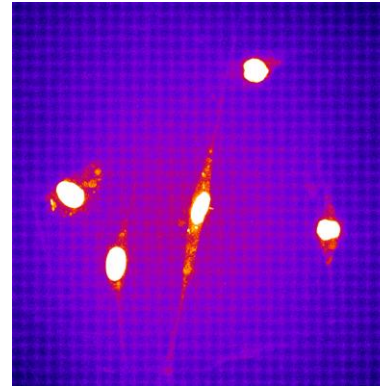
Segmentation is **THE** bottleneck of image analysis



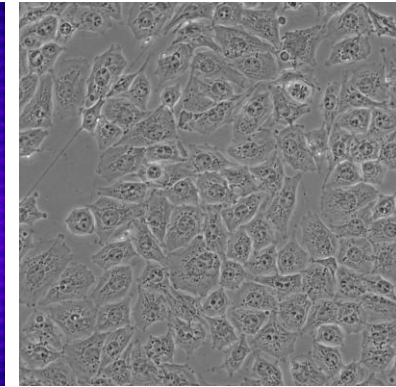
Image segmentation: from pixel intensities to labels

- A few things can make accurate segmentation harder.
- Classic segmentation needs image processing knowledge: filters, thresholds, etc.
- Deep learning needs a lot of training data

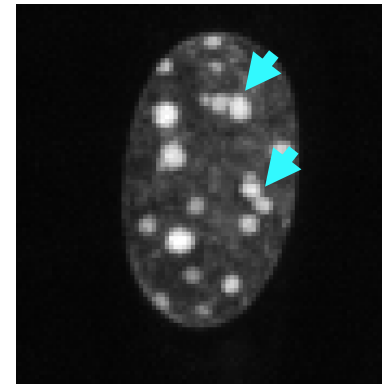
Local intensity differences



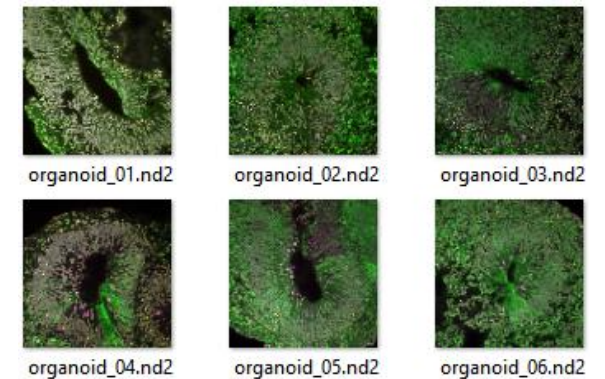
No clear intensity difference between classes



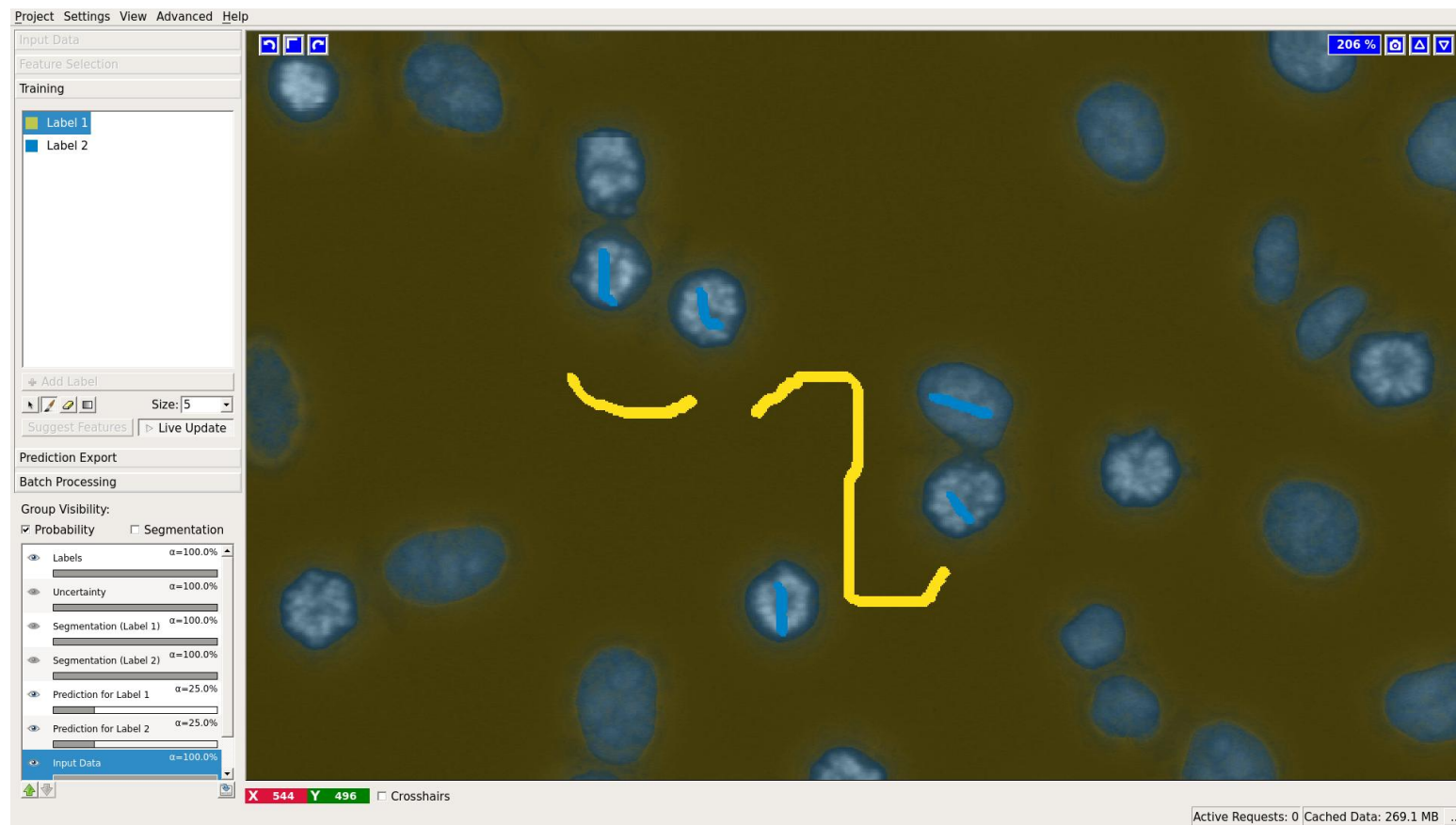
Touching objects



Batch differences



Ilastik: Segmentation for the people



Anna Kreshuk

ILASTIK: INTERACTIVE LEARNING AND SEGMENTATION TOOLKIT

Christoph Sommer, Christoph Straehle, Ullrich Köthe, Fred A. Hamprecht

Heidelberg Collaboratory for Image Processing (HCI), University of Heidelberg

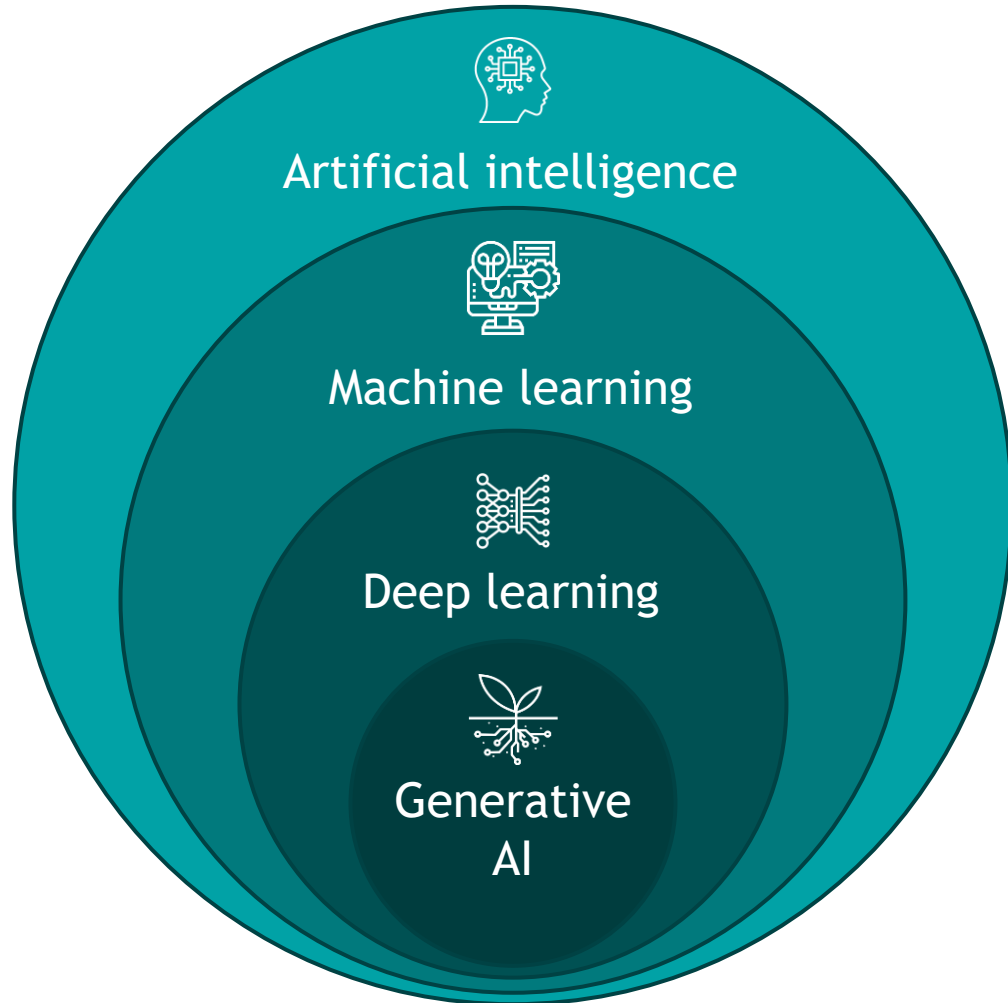
2011 doi: 10.1109/ISBI.2011.5872394

Ilastik pixel classifier demo

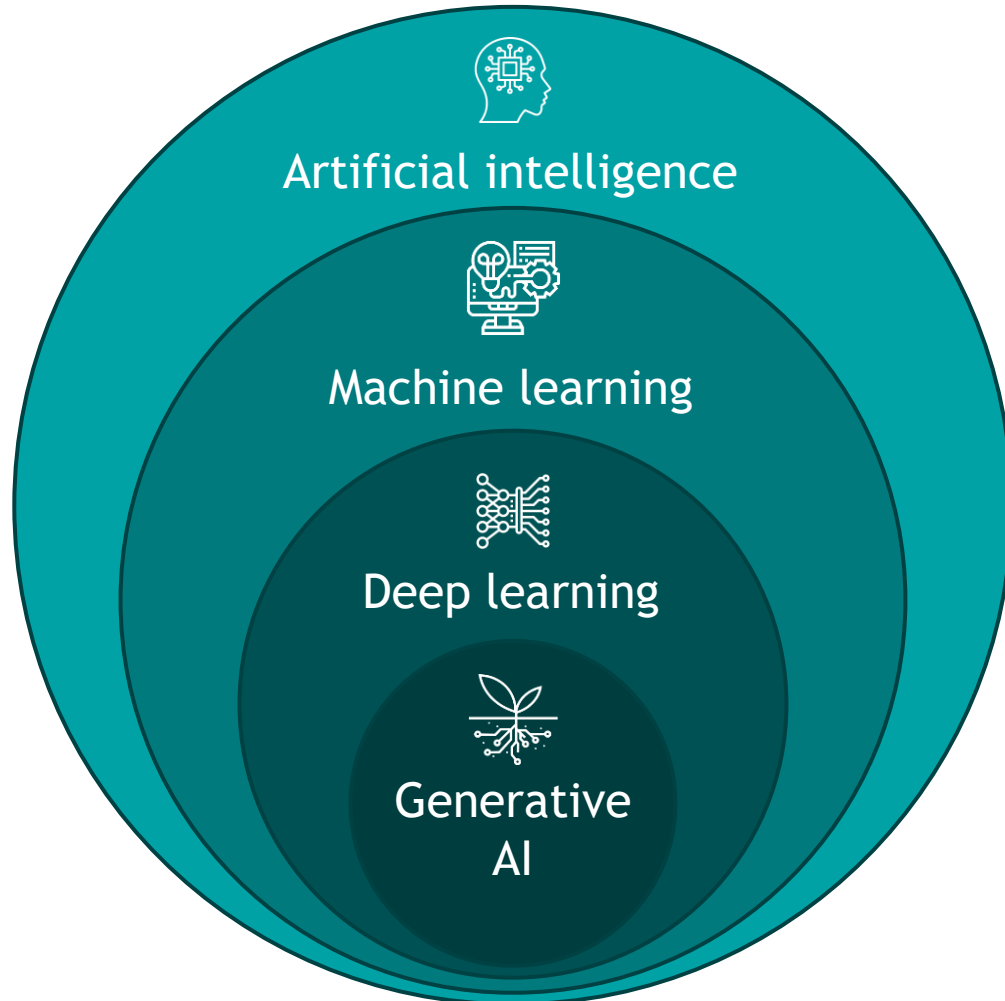


02 Supervised machine learning

Artificial intelligence



Artificial intelligence

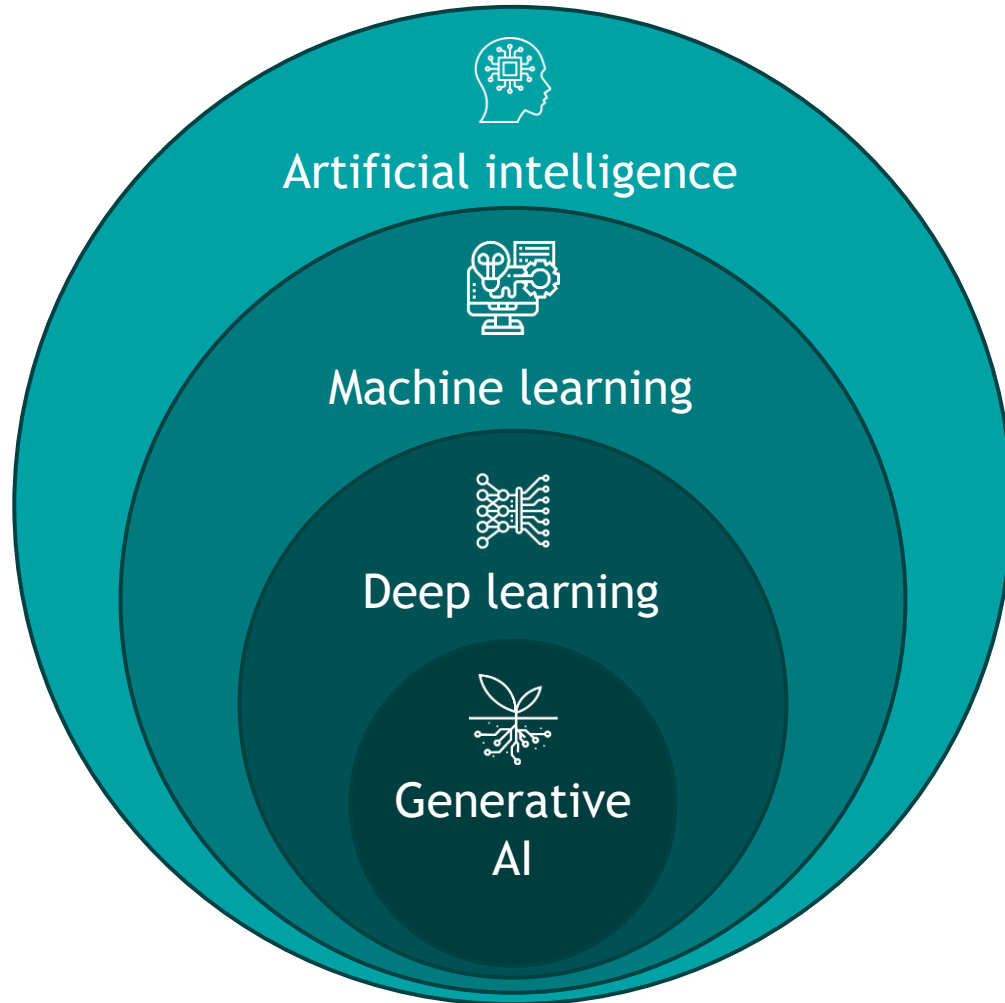


Artificial intelligence - machines show “human intelligence” (problem solving, decision-making, etc.)
e.g. DeepBlue (logic-based chess machine), Google Maps



Google Maps

Artificial intelligence

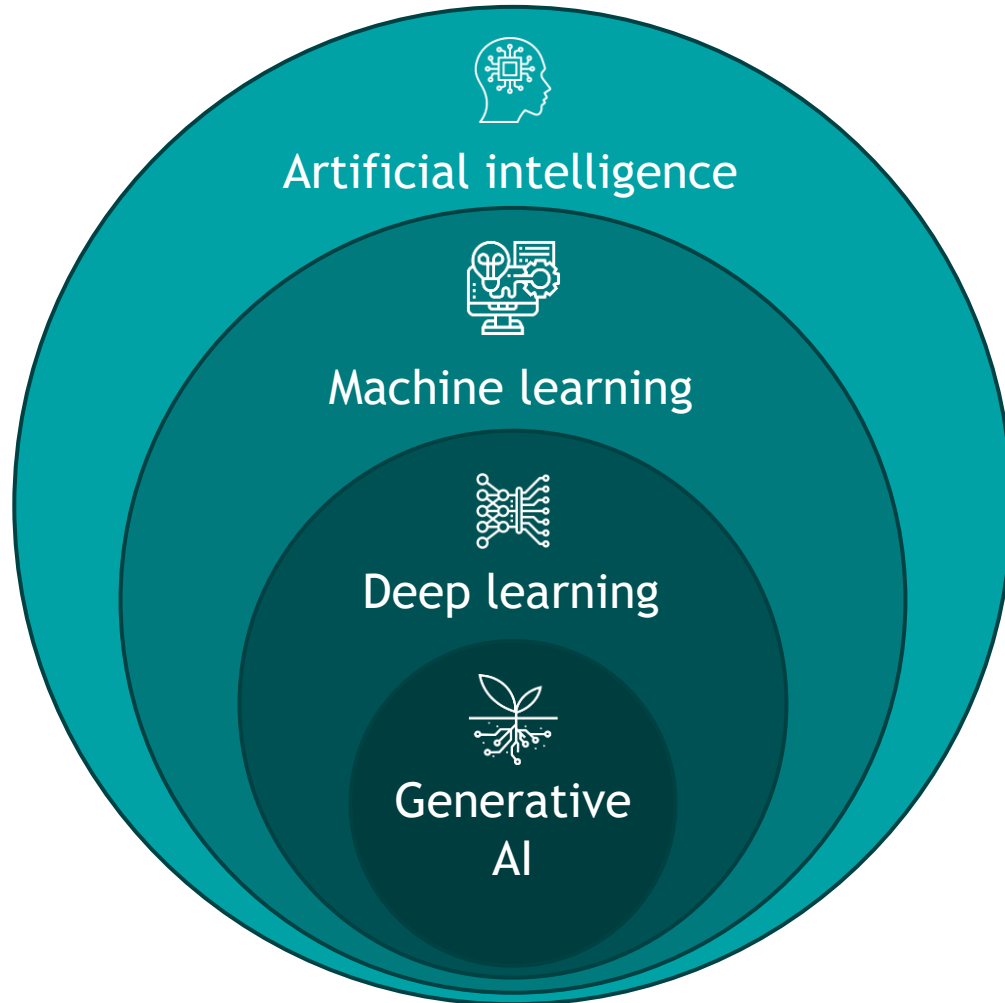


Artificial intelligence - machines show “human intelligence” (problem solving, decision-making, etc.)
e.g. DeepBlue (logic-based chess machine), Google Maps

Machine learning - system learns the rules from previous data
e.g. House price predictions, credit card fraud detection



Artificial intelligence



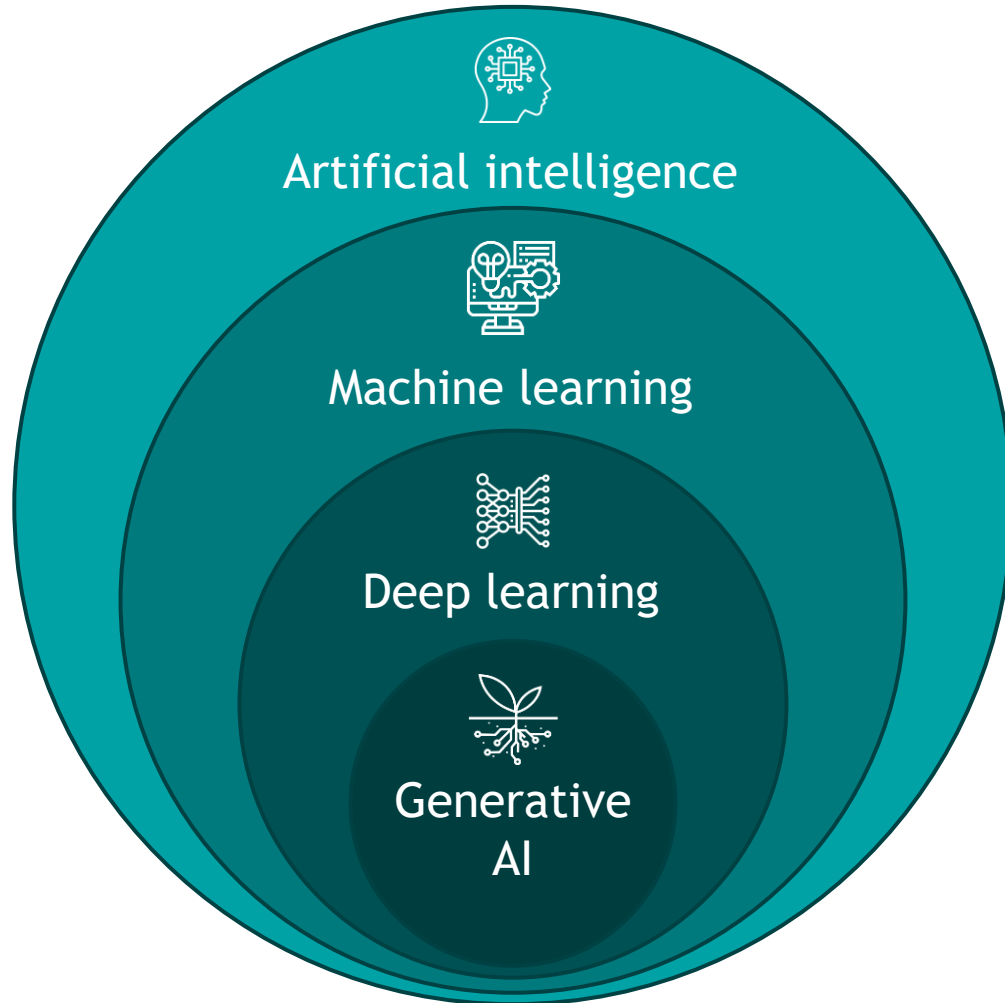
Artificial intelligence - machines show “human intelligence” (problem solving, decision-making, etc.)
e.g. DeepBlue (logic-based chess machine), Google Maps

Machine learning - system learns the rules from previous data
e.g. House price predictions, credit card fraud detection

Deep learning - machine learning with deep neural networks
e.g. Face recognition, speech-to-text



Artificial intelligence



Artificial intelligence - machines show “human intelligence” (problem solving, decision-making, etc.)
e.g. DeepBlue (logic-based chess machine), Google Maps

Machine learning - system learns the rules from previous data
e.g. House price predictions, credit card fraud detection

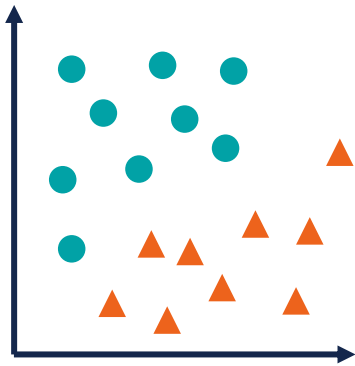
Deep learning - machine learning with deep neural networks
e.g. Face recognition, speech-to-text

Generative AI - creates new content based on learned patterns
e.g. ChatGPT



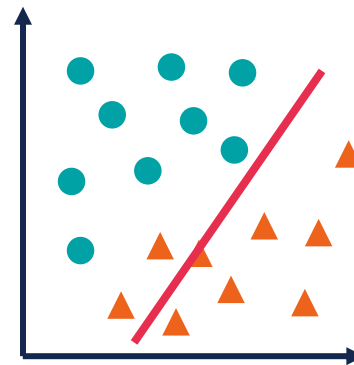
What is machine learning?

DATA

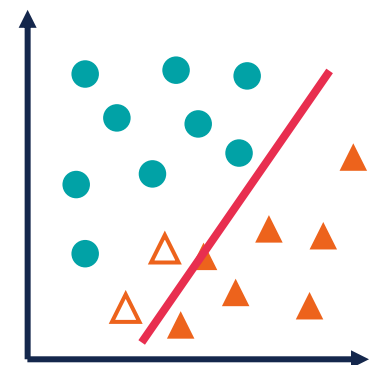


MODEL

Learns patterns
& decision rules

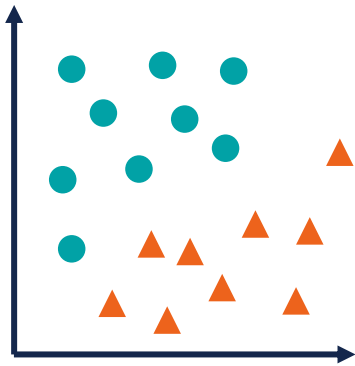


PREDICTION



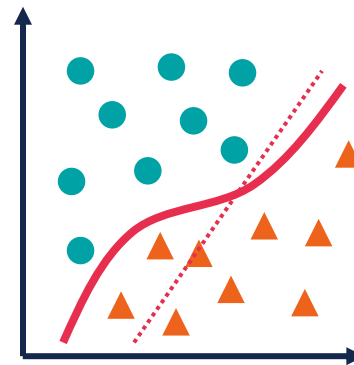
What is machine learning?

DATA

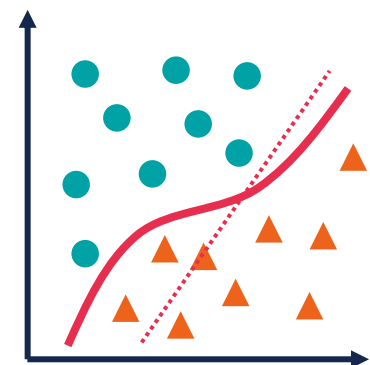


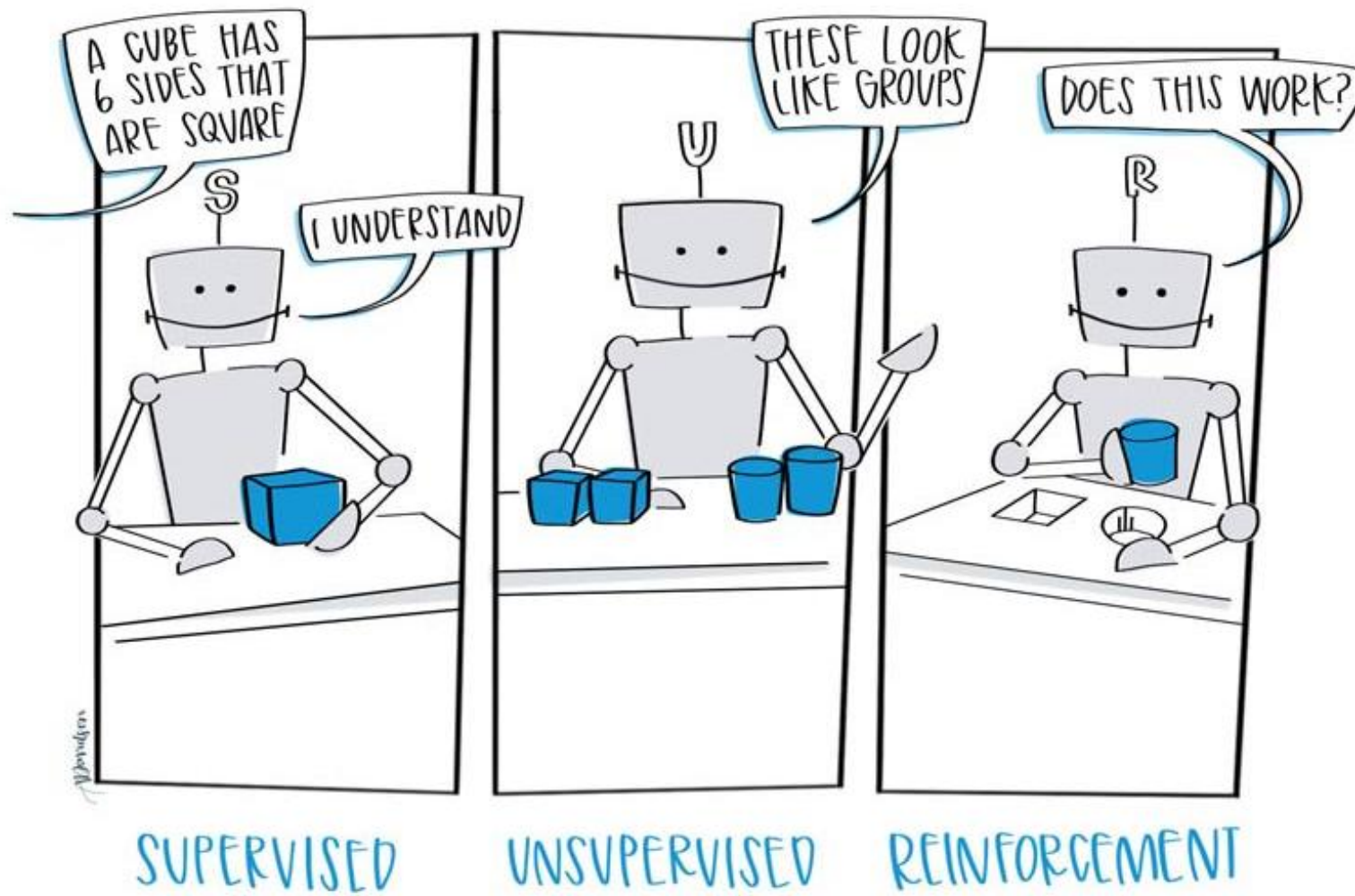
MODEL

Learns patterns
& decision rules

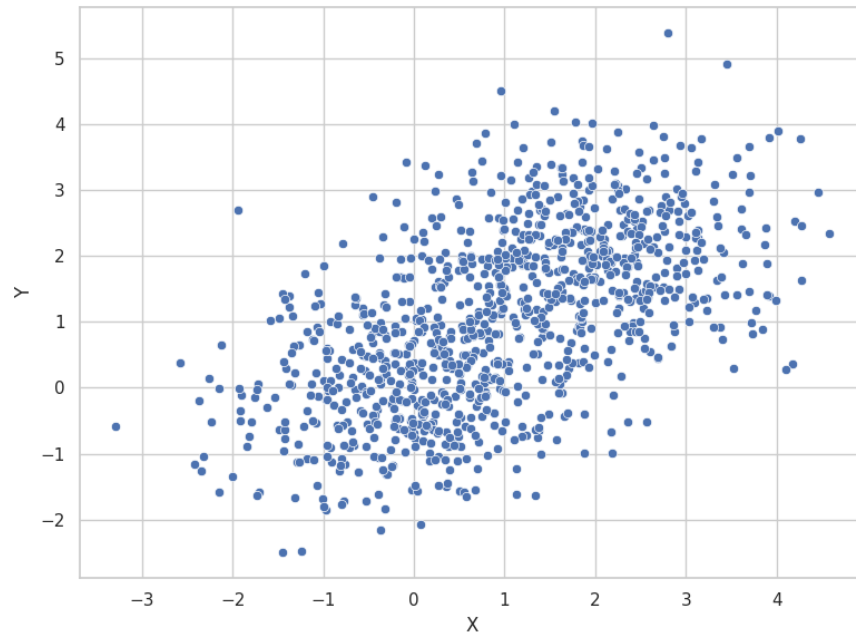


PREDICTION





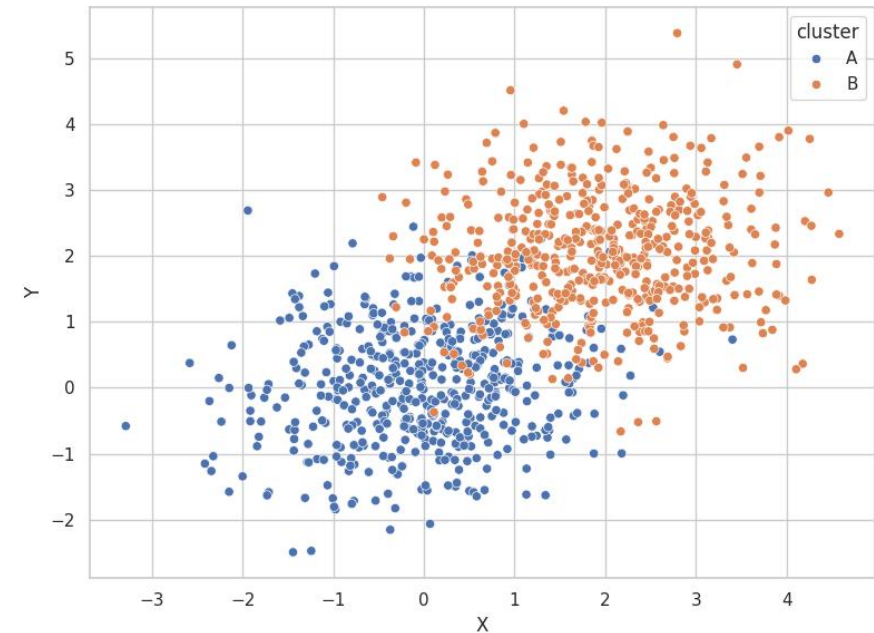
Regression vs classification



REGRESSION

Output = continuous

Based on X, predict the value of Y



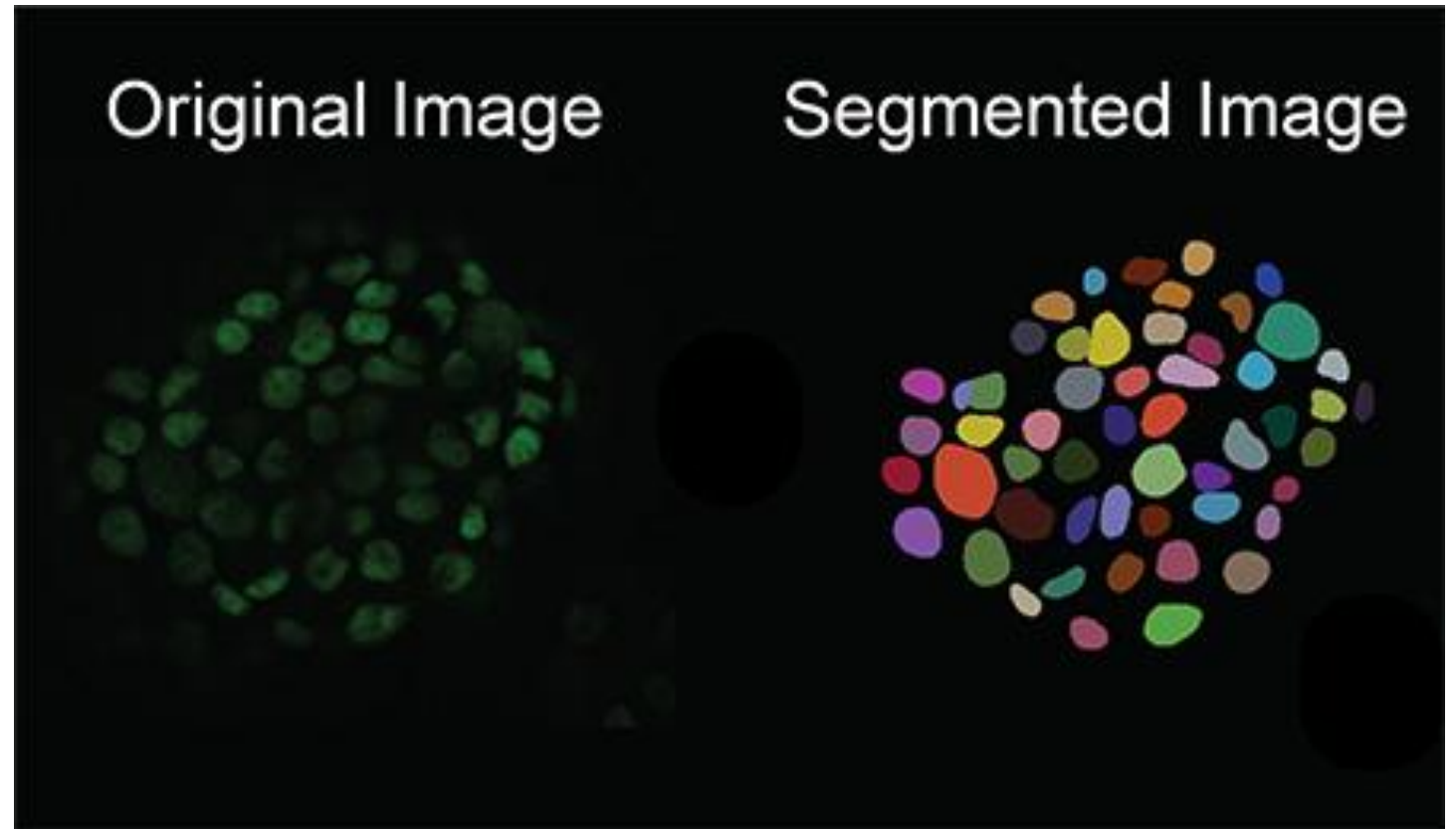
CLASSIFICATION

Output = discrete

Based on X and Y, predict the class

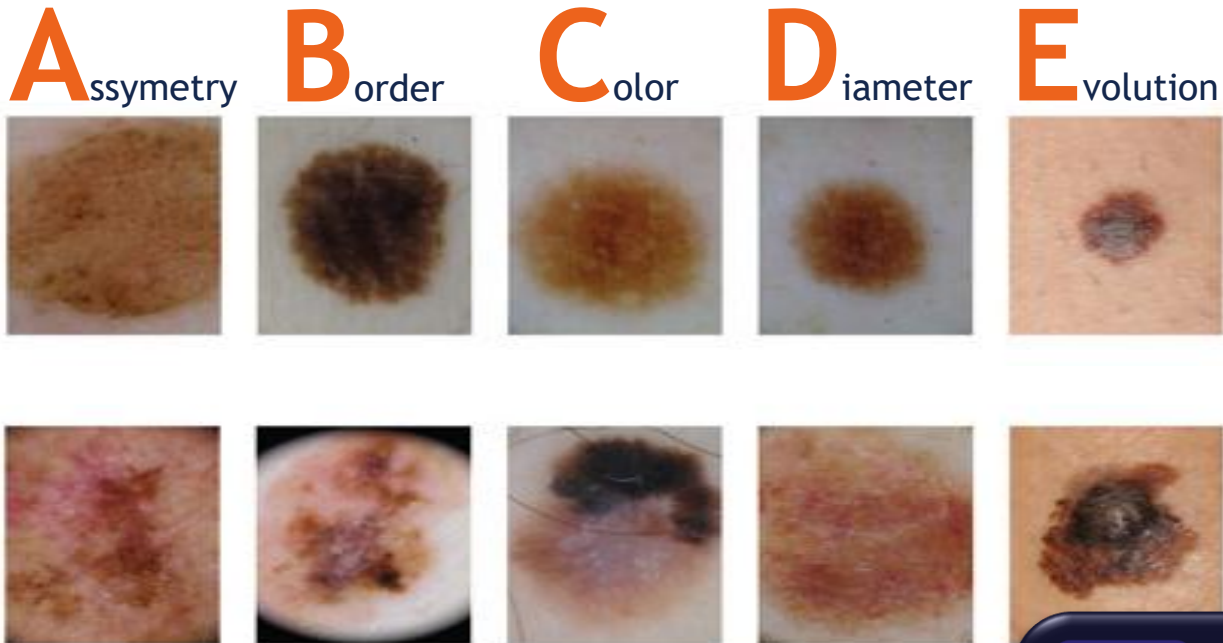
Classification applied to images

"Based on the pixel values, can I predict where the cells are?"



Maybe change this slides to something that has more to do with the subject?

Melanoma detection



“Based on the lesion characteristics, can I predict its malignancy?”

Binnenland

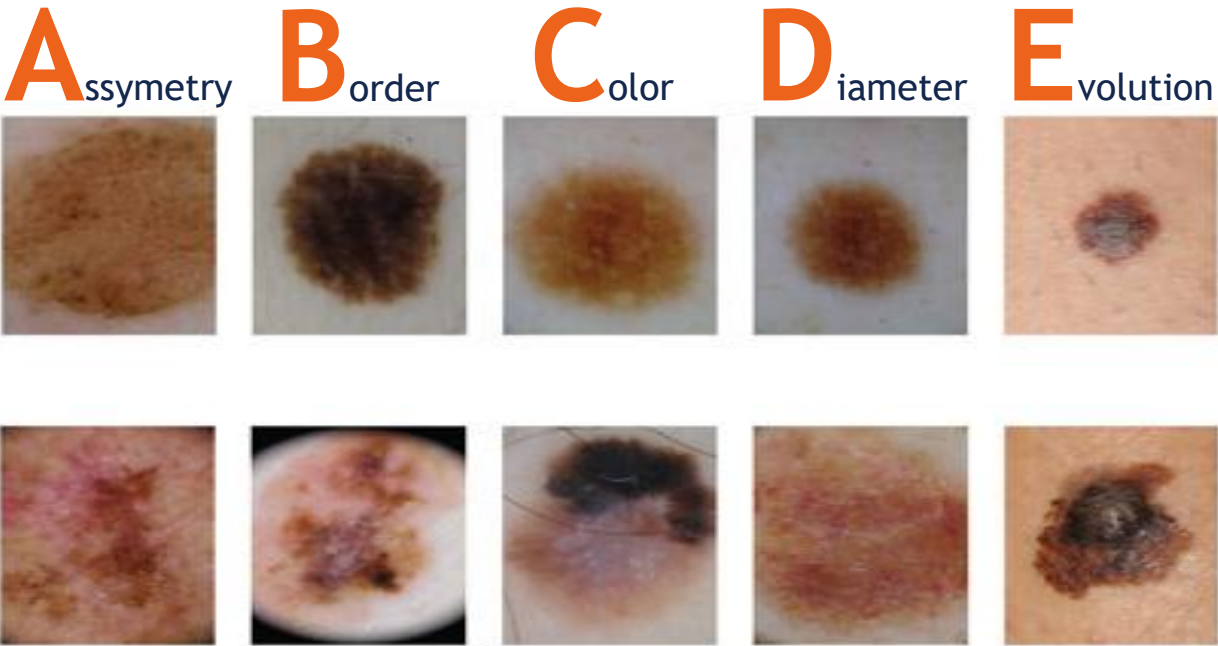
Skin-app SkinVision misses 1 in 4 cancers and gives false alarm for 1 in 8 spots, according to UGent study



Artikel luisteren | 6min

[Meer info](#)

Machine learning - key concepts

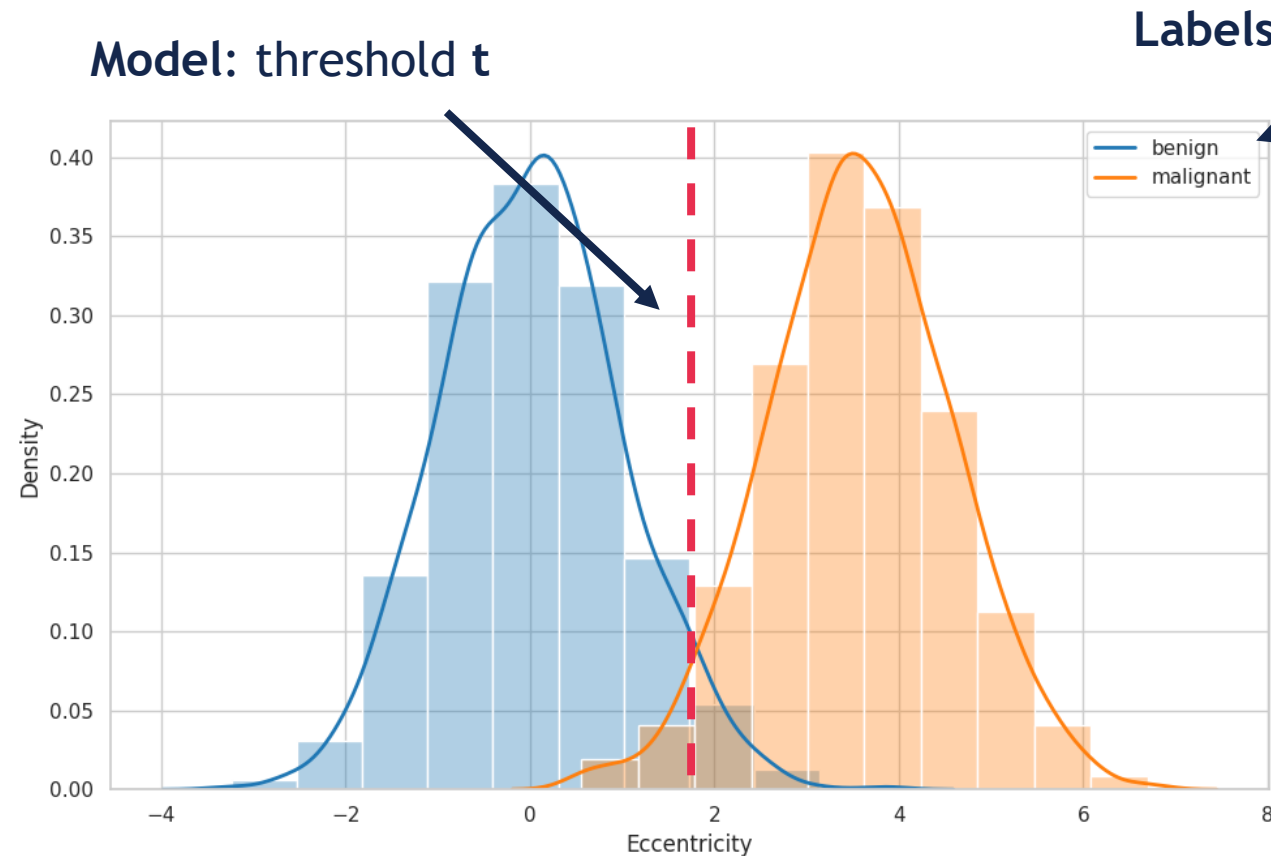


Samples



Features		Labels
A	B	C
1 Eccentricity	Solidity	label
2 -0.598720491	-2.863000525	0
3 -0.518610564	0.126108156	0
4 -1.843885879	-2.163015307	0
5 -0.289294284	-1.502927842	0
6 -1.177039495	-2.112436188	0
7 -0.934795367	-1.11164475	0
8 -1.598846606	-0.066942958	0
9 -1.455127299	0.972633286	0
10 -0.786144288	-1.096376283	0
11 -0.940576854	-1.869216938	0
12 -1.332590084	-0.206261961	0
13 -0.381720583	-0.413528409	1
14 -1.261911388	-0.946902992	0
15 -1.636395728	-1.201188514	0
16 -1.40571709	-1.605459274	0
17 -1.452295463	-1.266016012	0
18 -1.430260883	-1.378927623	0
19 -1.228654033	-2.1408459	0
20 -0.97102196	-2.366513251	0
21 -0.005350458	0.00551553	0

Machine learning - key concepts

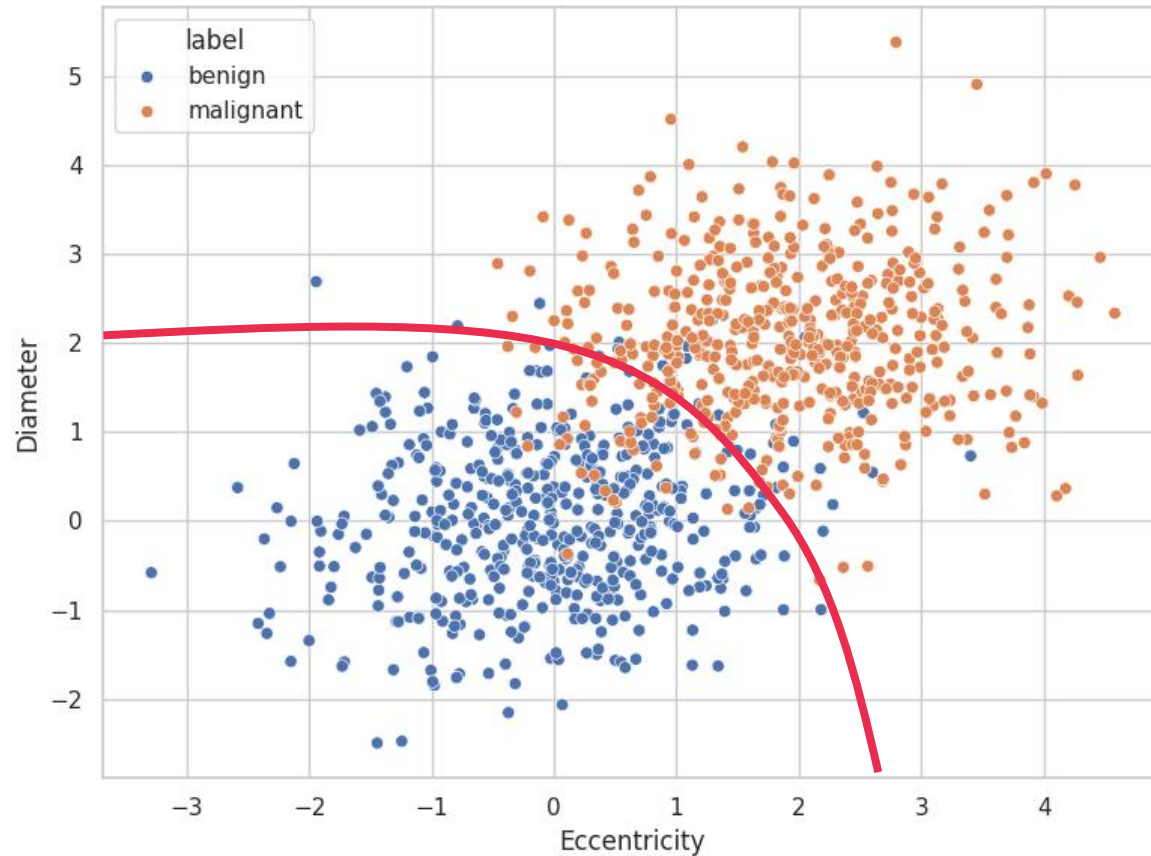


The **model** learns its prediction rules based on example data (**features** and **labels**)

For a new sample, the **features** are used to predict its **label**

Features (characteristics computed from the data)

Feature space

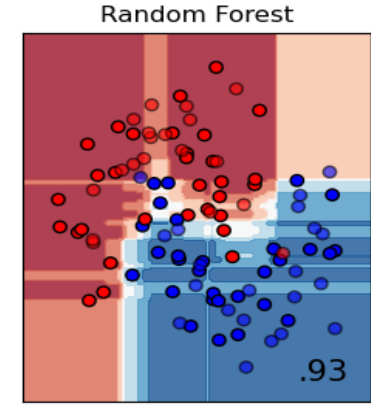
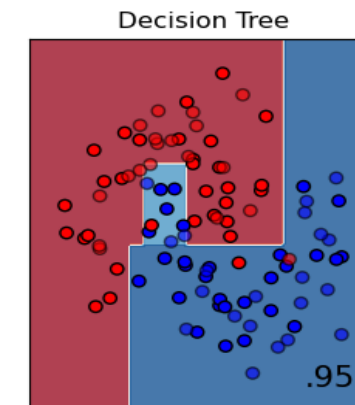
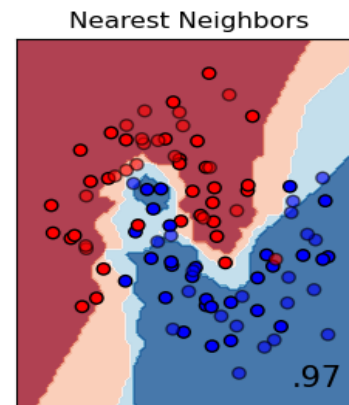
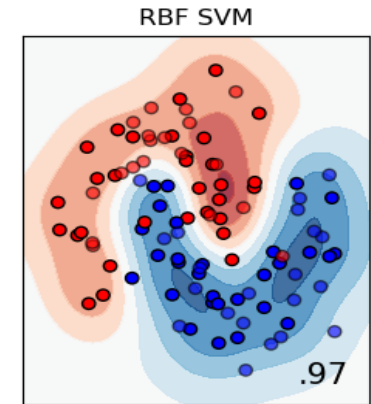
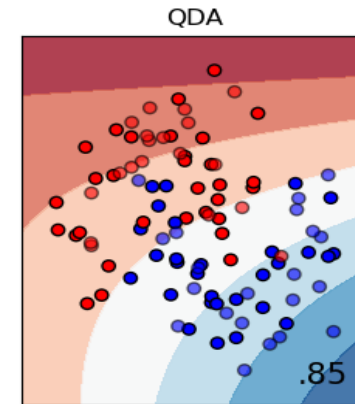
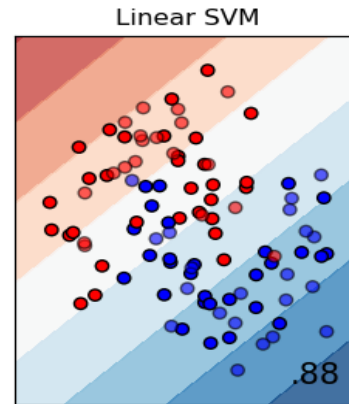
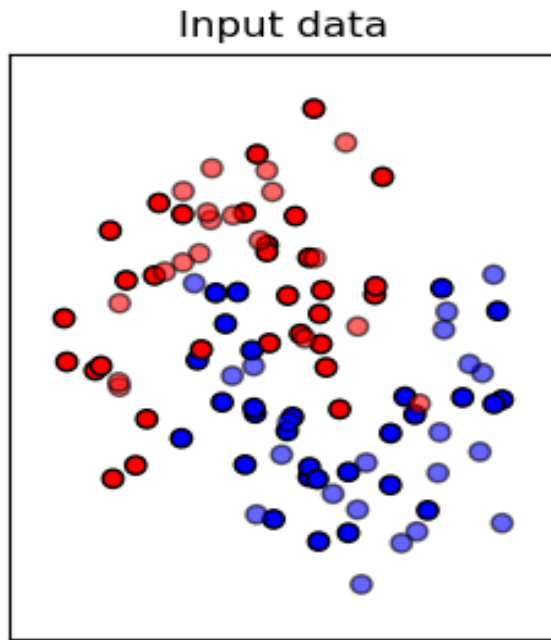


Combine multiple features to improve the predictive capabilities

→ Feature vector X

→ Euclidean vector space

Model complexity

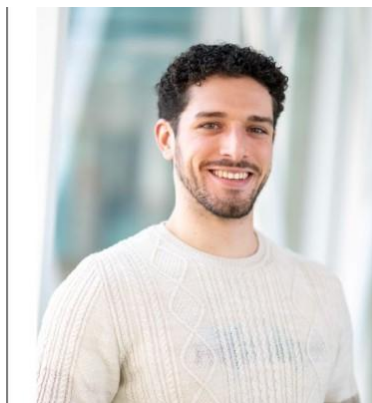
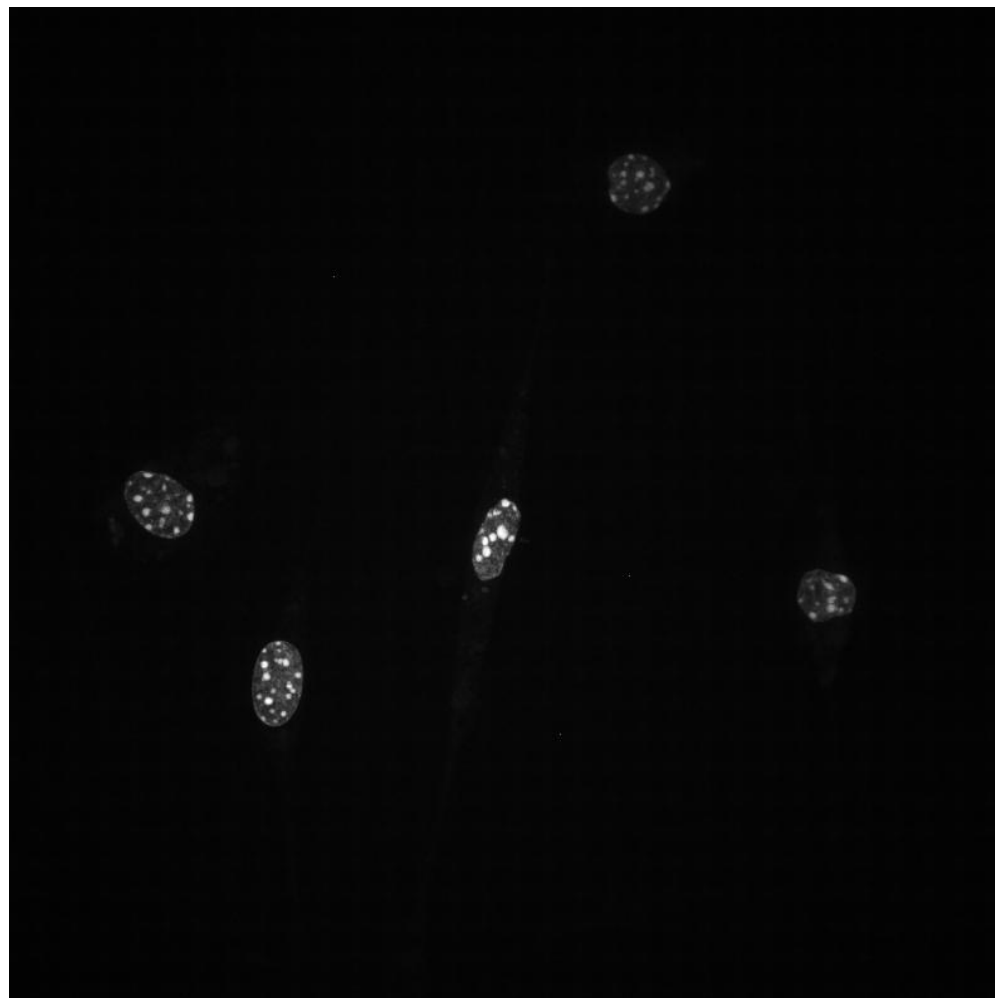


Data complexity ↑ → Model complexity ↑ → # of examples needed ↑

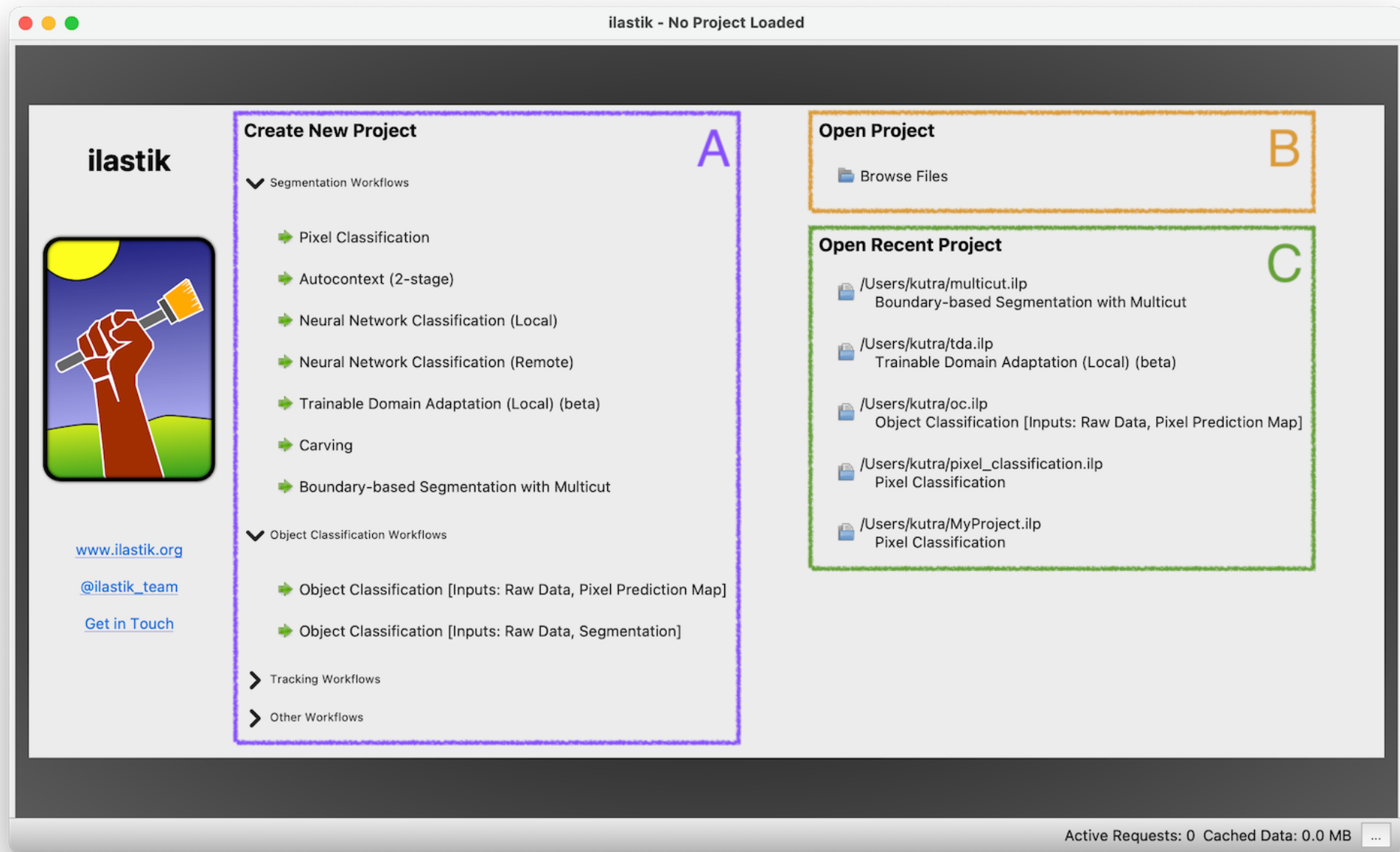
03 Pixel classification

Training a pixel classifier in Ilastik:

3D nuclei dataset - sample from Alessio Silva

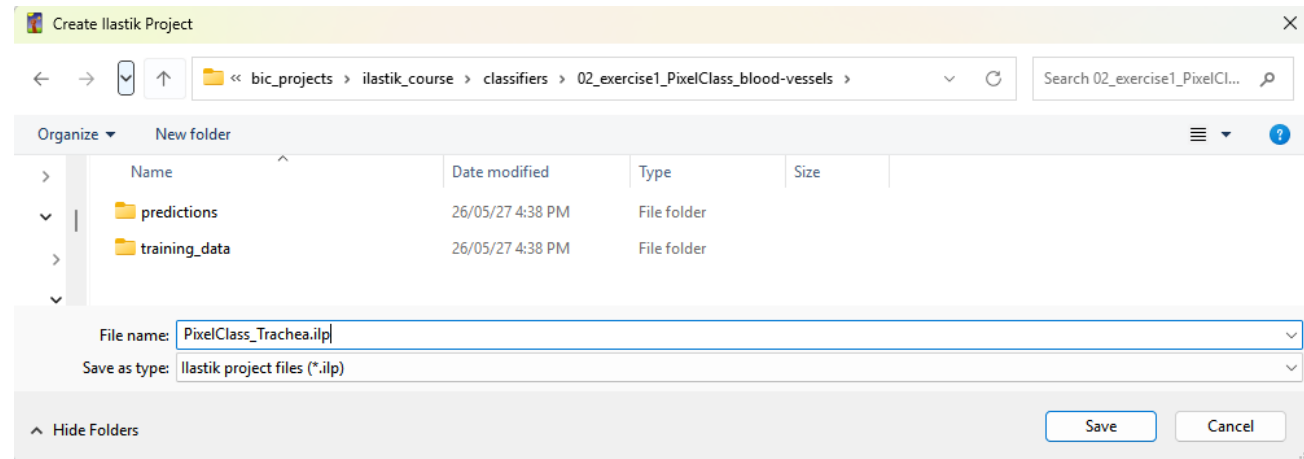
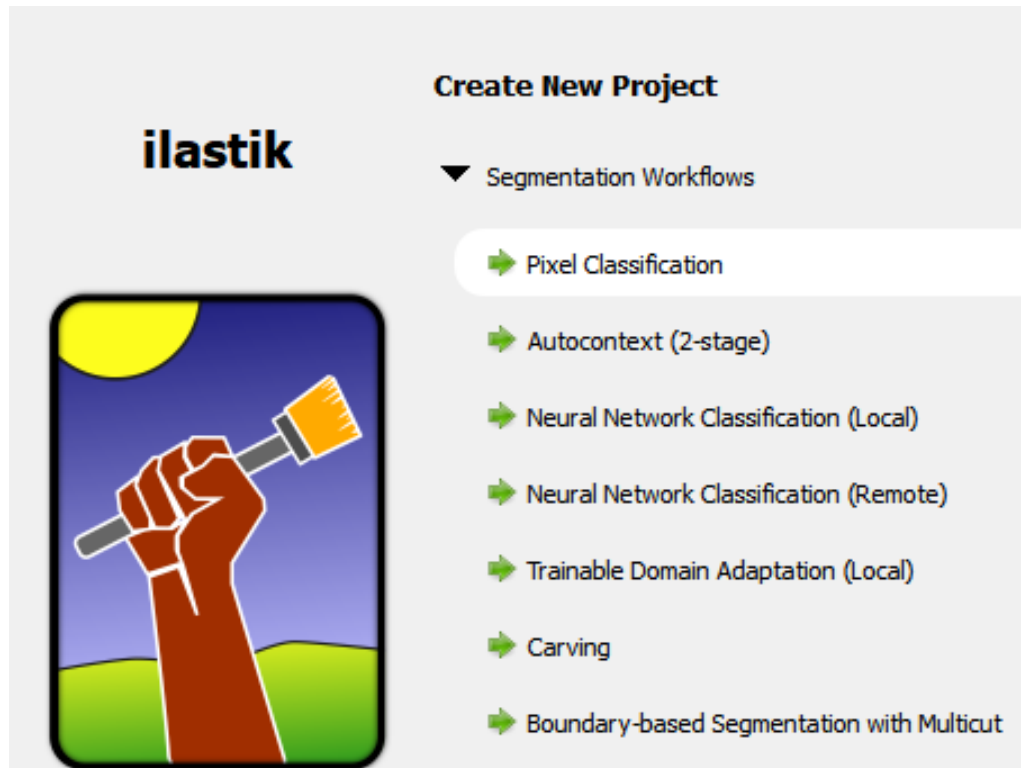


The start-up screen



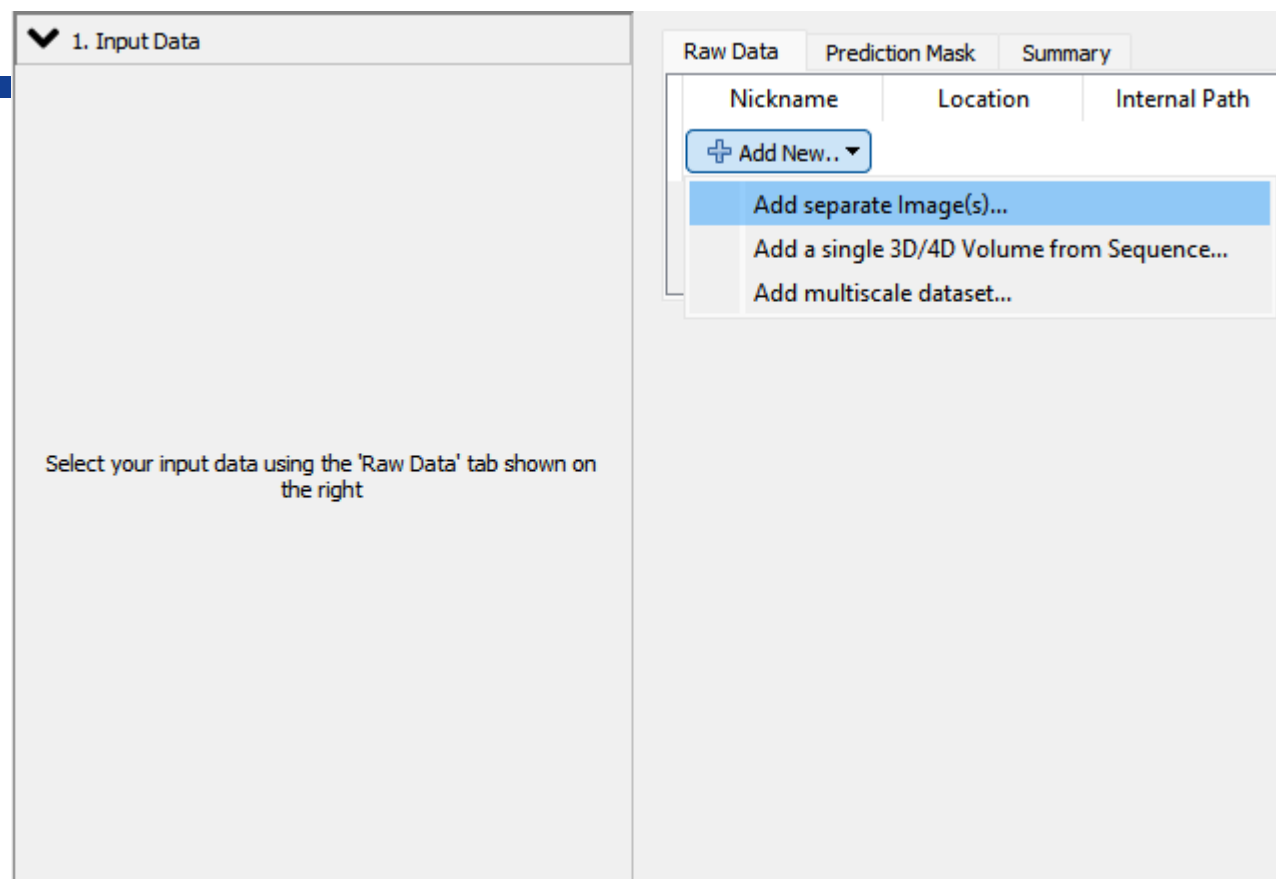
Creating an Ilastik Pixel Classifier Model

- Create new pixel classification project

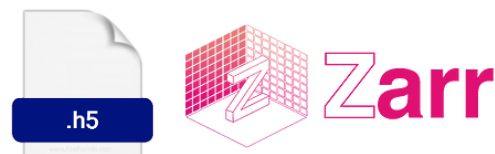


Loading images into Ilastik

- Add training image as separate image



Main formats used by ilastik are: HDF5 and N5/Zarr



Other formats that can be directly imported include:

- TIF/TIFF
- PNG
- GIF

Creating an Ilastik Pixel Classifier Model

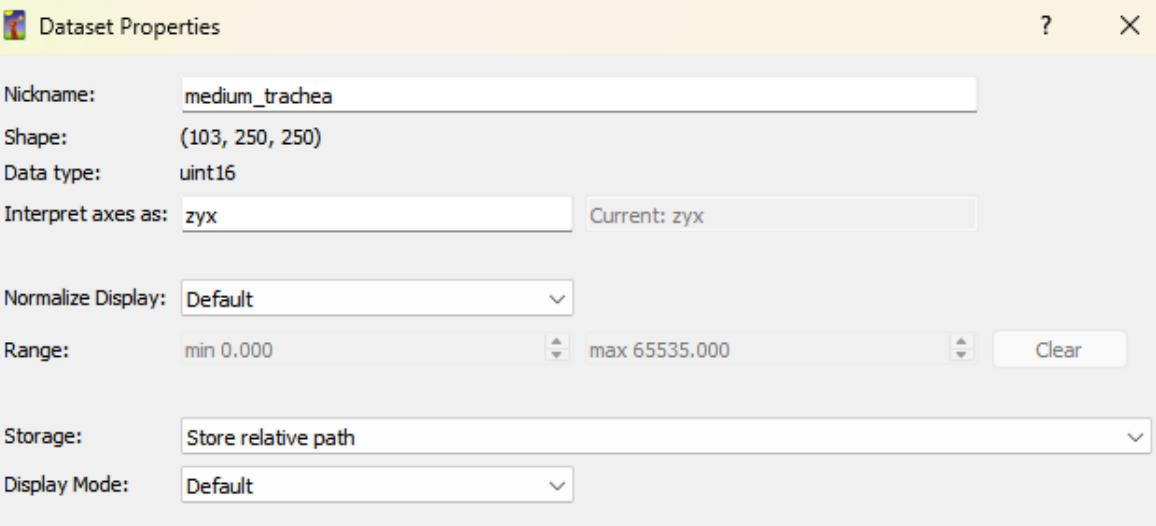
Raw Data

Prediction Mask

Summary

	Nickname	Location	Internal Path	Shape	Resolution Level	Data Range
1	medium_trachea	Relative Link: training_data\medium_trachea.tif z: 103, y: 250, x: 250				
+ Add New.. ▾						

- Edit properties...
- Replace with file...
- Replace with stack...
- Remove



Dataset Properties

Nickname: medium_trachea

Shape: (103, 250, 250)

Data type: uint16

Interpret axes as: zyx Current: zyx

Normalize Display: Default ▾

Range: min 0.000 max 65535.000 Clear

Storage: Store relative path ▾

Display Mode: Default ▾

Storage:

Store relative path

Display Mode:

Copy into project file

Store absolute path

Store relative path

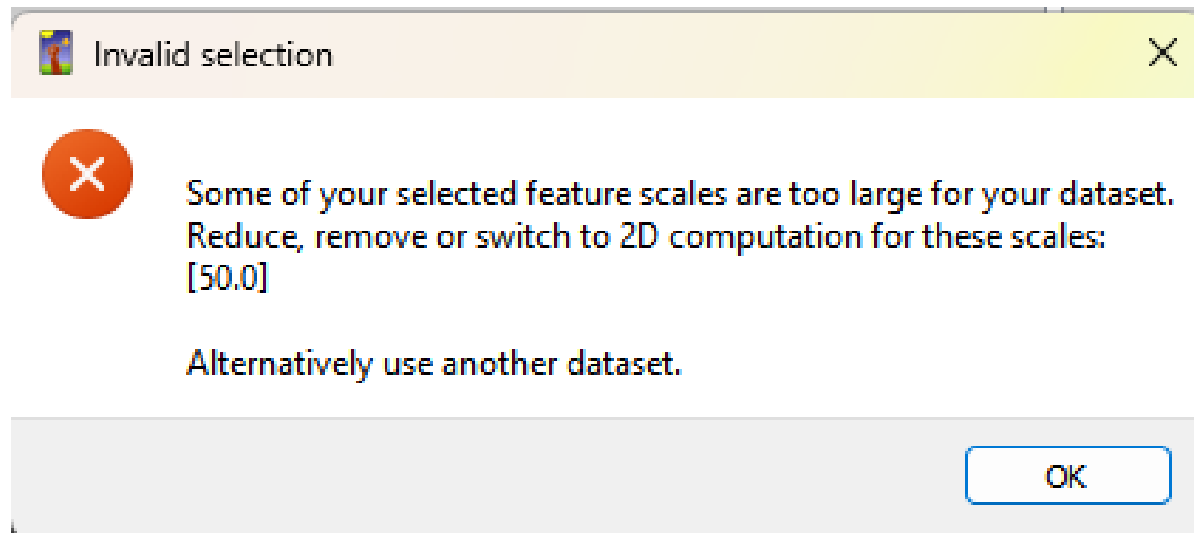
It's important to define the storage rules **before** training.

If you want to make the model shareable, the easiest is to copy the files into the project file but this makes the project file larger and slower.

You can also choose a relative path by default but be sure you always maintain the architecture of the classifier folder

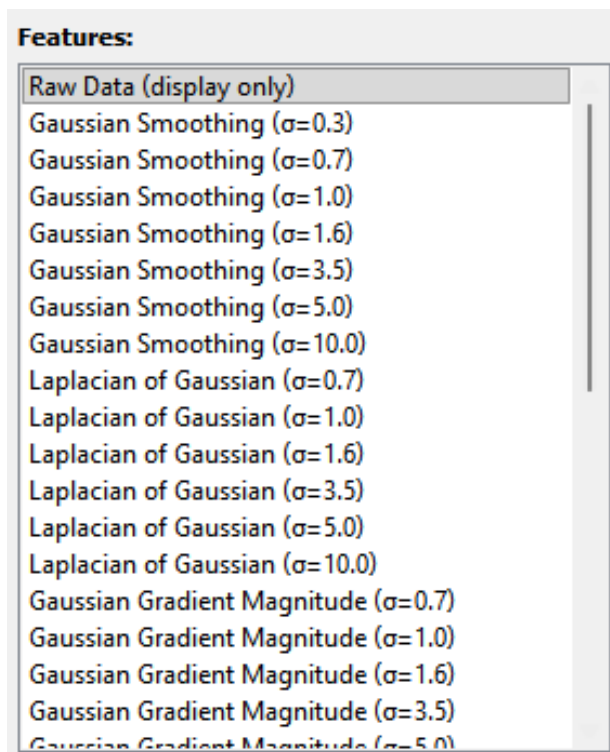
Feature Selection

- If you have less than the sigma size number of pixels or slices it will give you an error that gets solved by ticking off the sigma that is too large for your data.

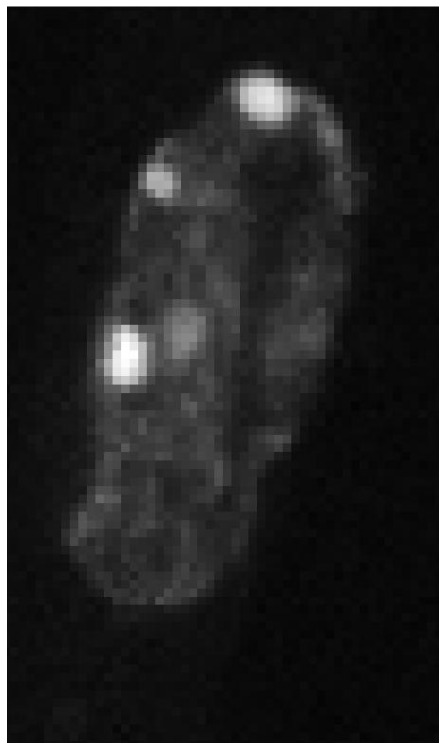


Feature Selection

- The features along with their sigma are located in the bottom left and one can see the effect of the filter on the data



Raw intensity

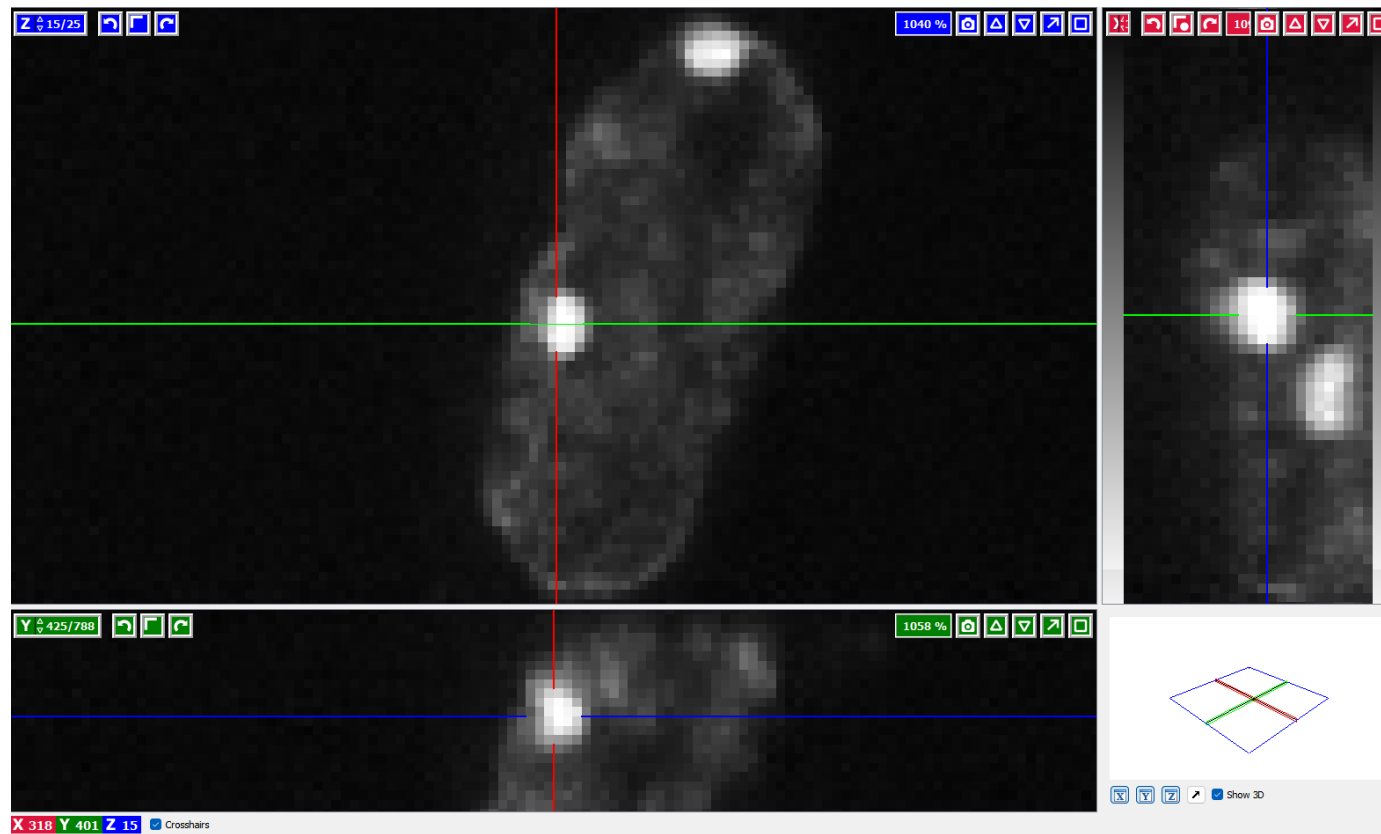


Features



Ilastik does not take calibration into account.
Be careful in 3D.

The Ilastik viewer

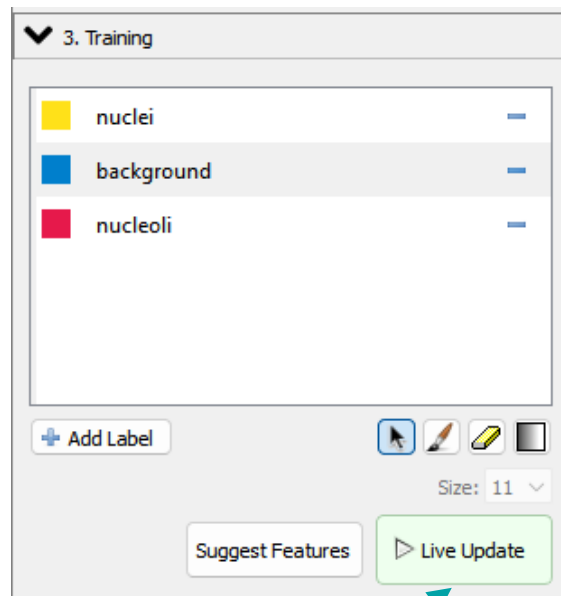


Useful shortcuts:

Shortcut	Function
Ctrl + mouse wheel	Zoom in and out
Shift + left click drag	Move in XY
Mouse wheel	Move in Z or in T

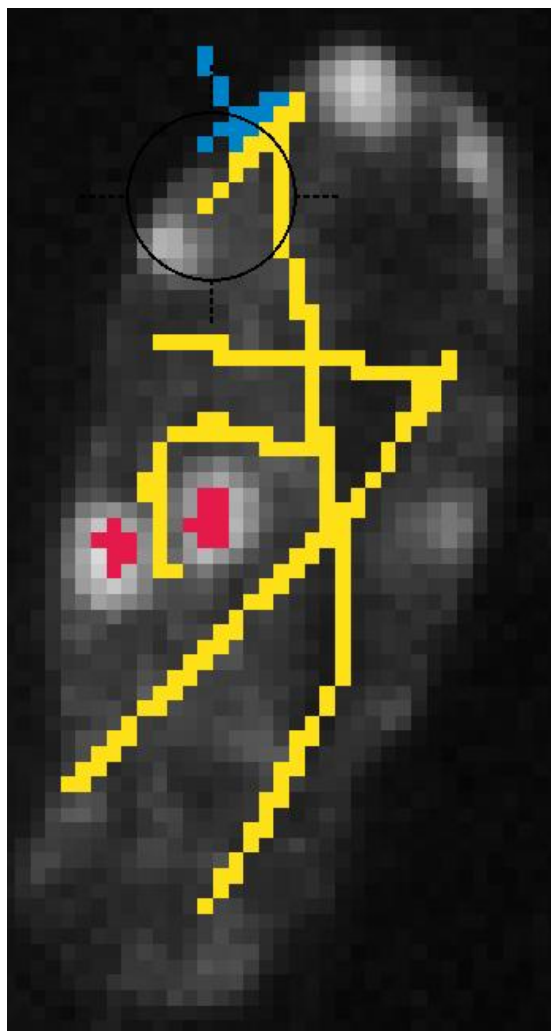
Training your model

Classes

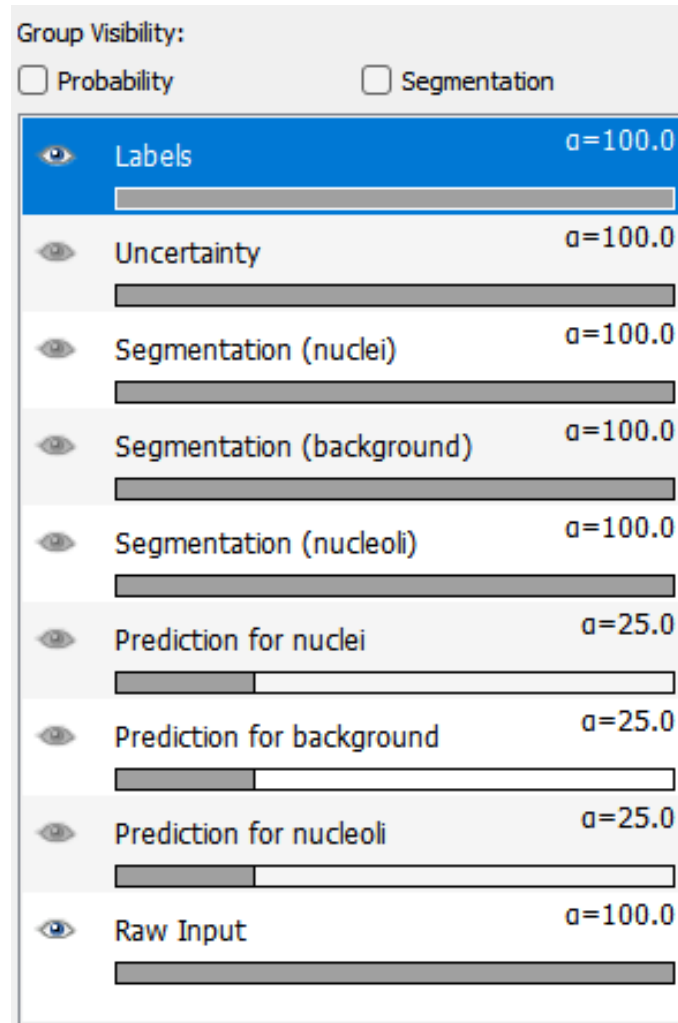


Train the model

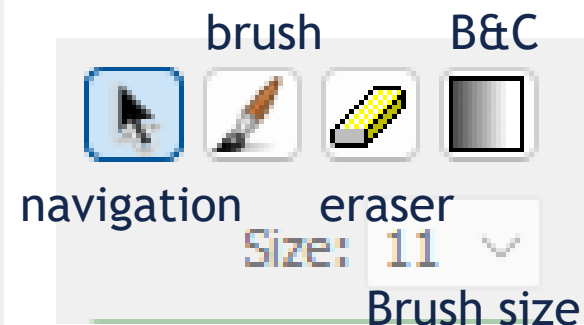
Annotations



Layers

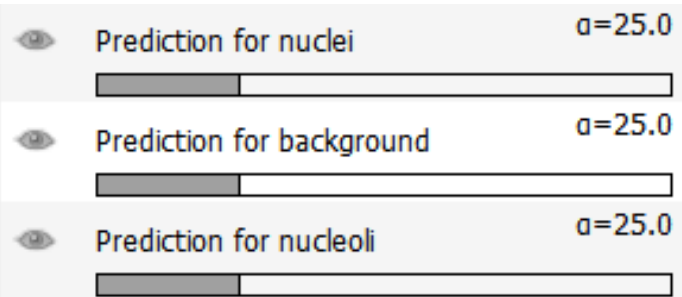
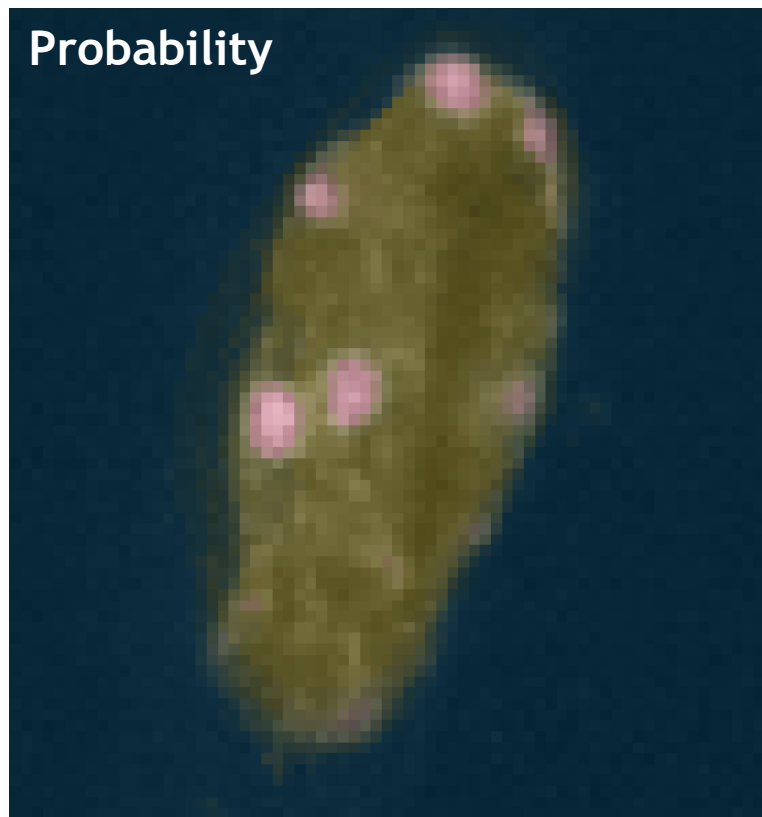


Tools

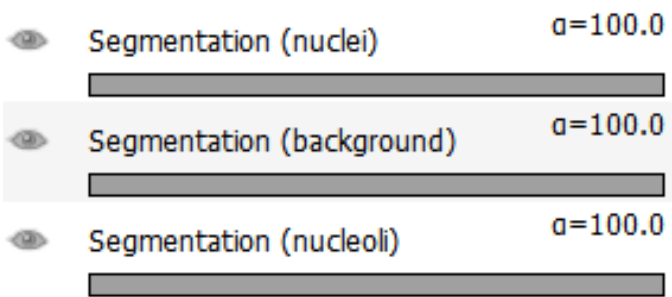
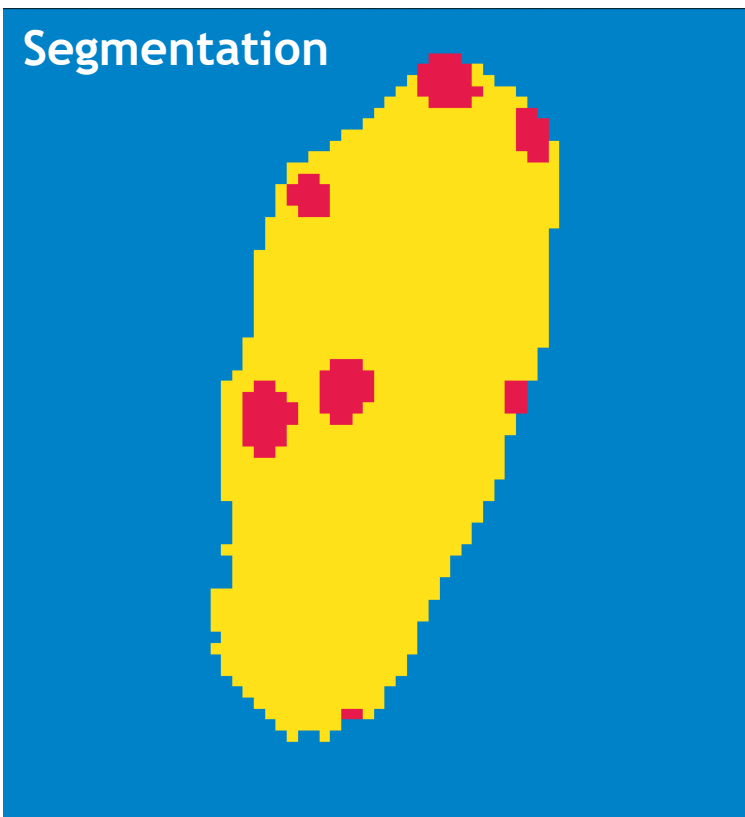


Important layers to guide the trainings

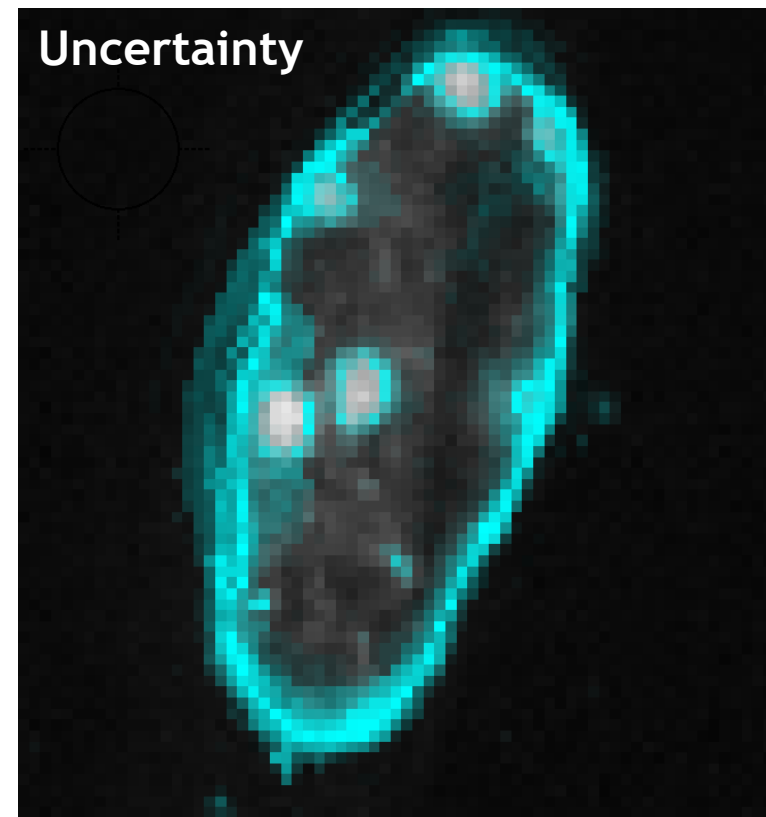
Probability



Segmentation

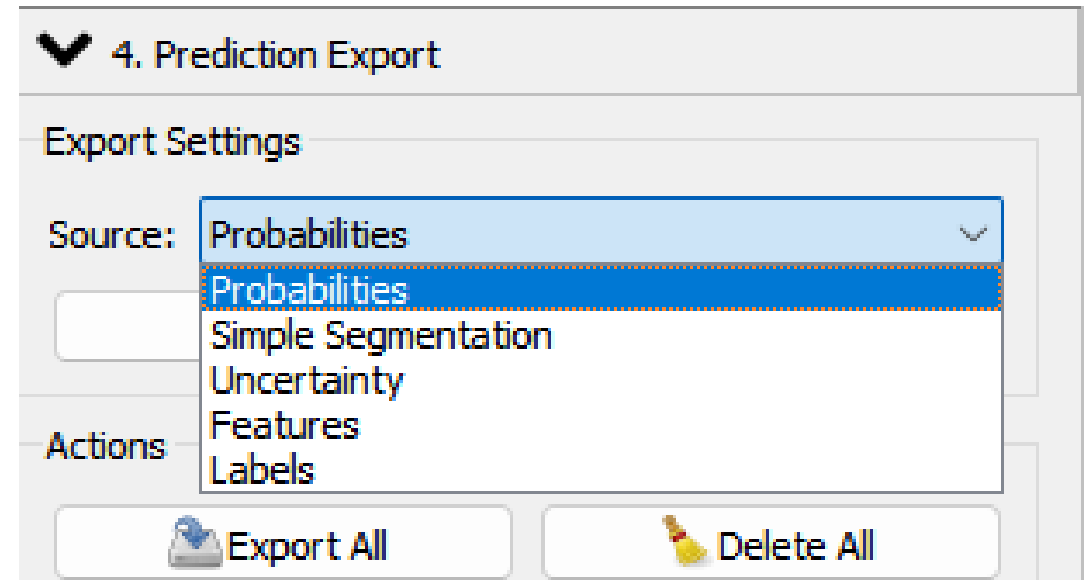


Uncertainty



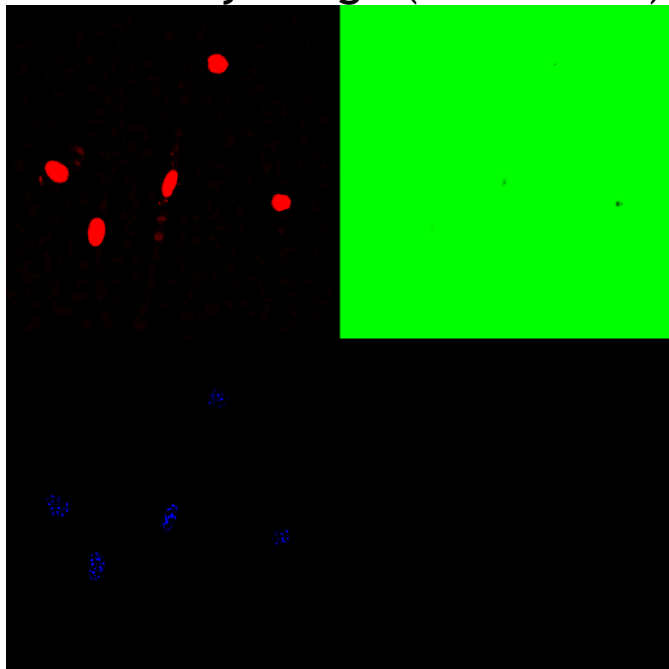
Prediction export

- Types of export: Probabilities, Simple Segmentation, Uncertainty, Features and Labels
- Probabilities versus segmentation: Ilastik outputs a **Probability Map** -> For every pixel, it outputs a float value between 0 and 1 representing how confident the Random Forest is.
 - For example: A pixel might be 0.85 Nucleus, 0.12 Nucleoli, and 0.03 Background.
- To get a segmentation, Ilastik applies a **threshold** to the probability map (eg 0.4)
- Click “Export All” to apply the classifier to all the training data

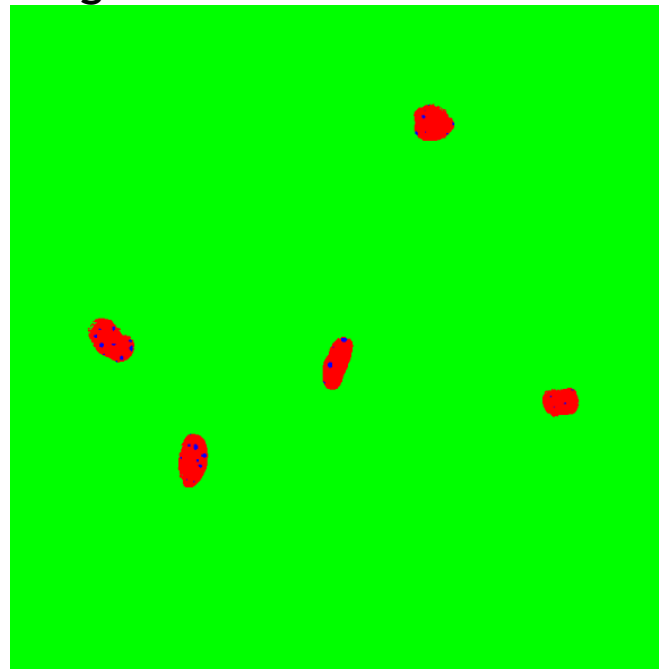


Prediction export

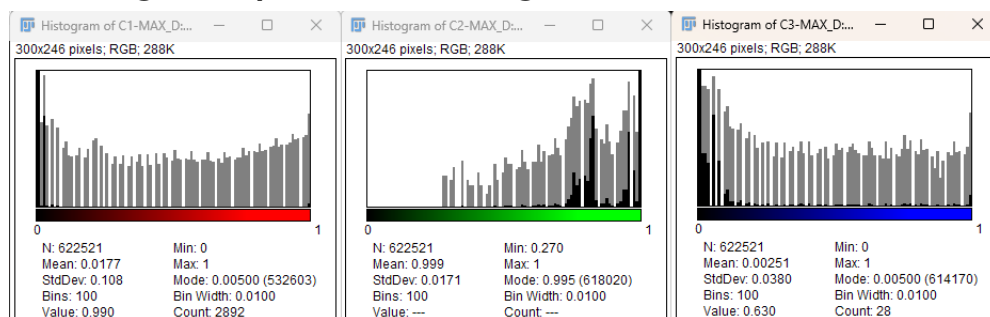
Probability image (3 channels)



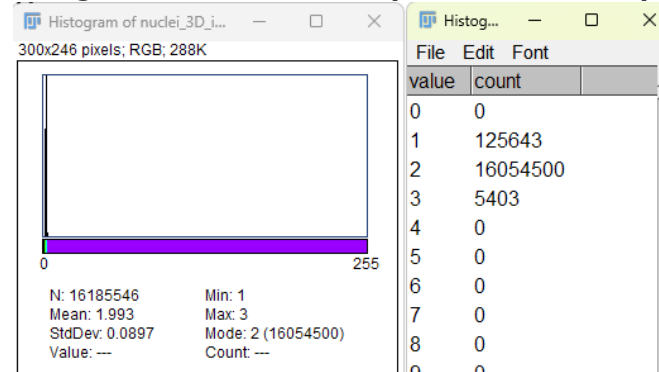
Segmentation



Histogram per class -> goes from 0 to 1



Single channel. 1 pixel value per class



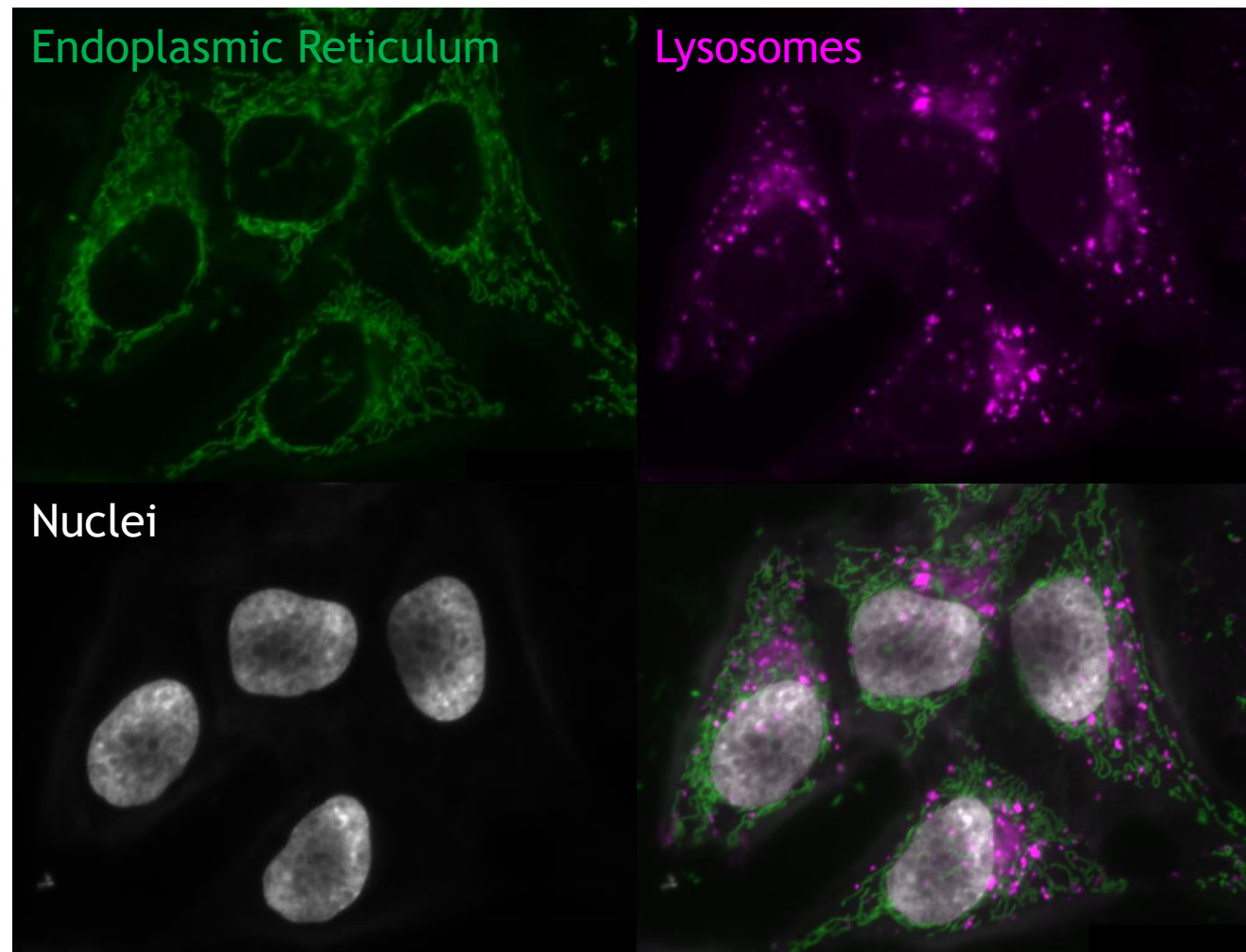
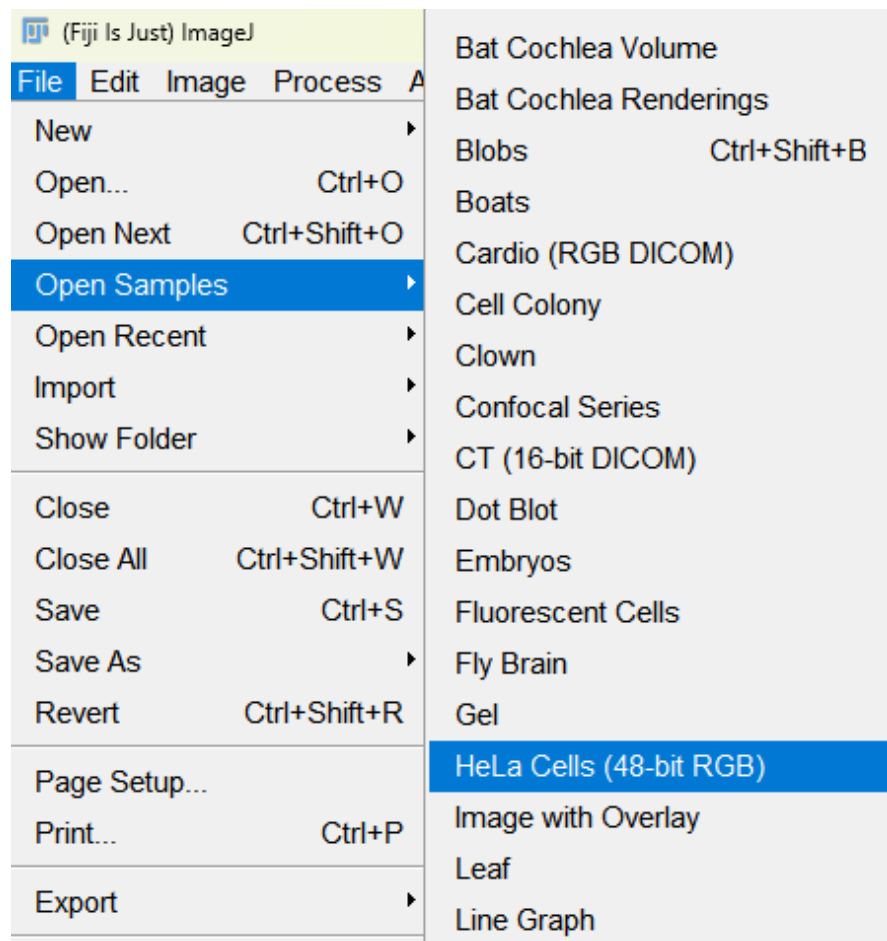


Time for a break

Exercise: Feature hacking!

Dataset origin:

FIJI>Open Samples>hela-cells.tif



Exercise: Feature hacking!

1. Open Ilastik project:

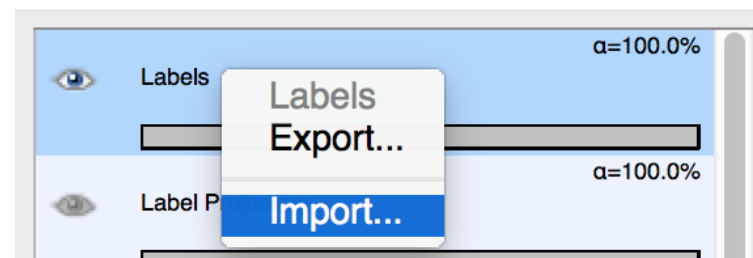
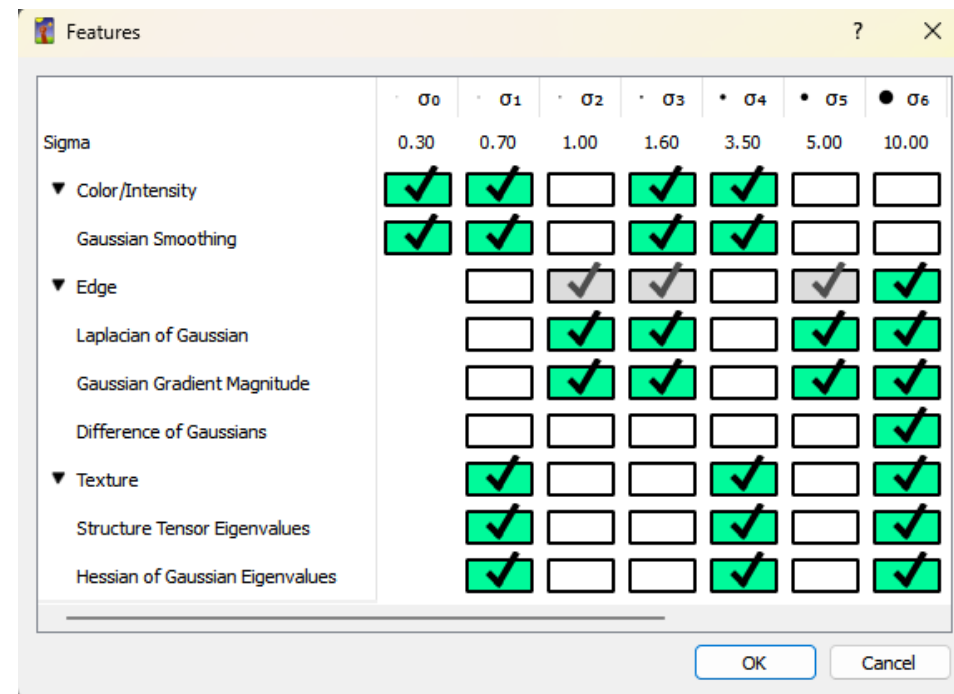
\\ilastik_course\classifiers\02_exercise_featurehacking_HeLaCell
Compartments\PixelClass_feature_hacking_2D.ilp

2. Find the minimum amount of features that give you a good result:

- a) Go to the training tab and import the annotations of the structure you are playing with. Annotations can be found:

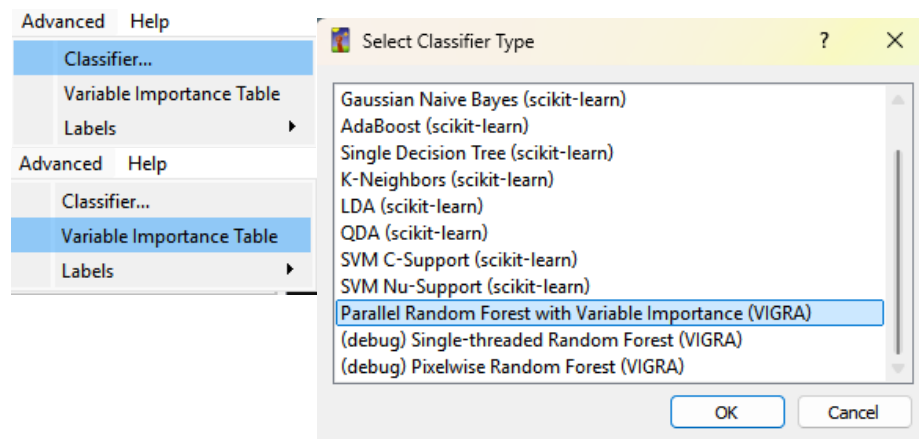
D:\Ongoing_projects\bic_projects\ilastik_course\classifiers\02_exercise_featurehacking_HeLaCellCompartments\training_data\labels

- b) Compare your selected features versus using all of them on the live prediction



Exploring feature contribution

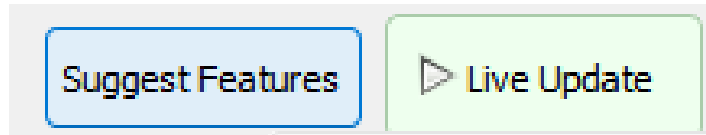
- Open the variable importance table:



Variable Importance Table				
Variable Name	Class #0	Class #1	Overall	Gini
Laplacian of Gaussian ($s=1.0$) in 2D	0.02279	0.02031	0.04309	1.36554
Laplacian of Gaussian ($s=0.7$) in 2D	0.01815	0.02044	0.03859	1.06302
Hessian of Gaussian Eigenvalues ($s=1.6$) in 2D [0]	0.01660	0.01445	0.03105	0.92346
Difference of Gaussians ($s=0.7$) in 2D	0.01371	0.01218	0.02589	0.79374
Hessian of Gaussian Eigenvalues ($s=0.7$) in 2D [0]	0.01157	0.01318	0.02475	0.68892
Difference of Gaussians ($s=1.6$) in 2D	0.01136	0.01032	0.02168	0.69106
Difference of Gaussians ($s=1.0$) in 2D	0.01017	0.00688	0.01705	0.52554

Optimizing classifier with the “Suggest Features”

- Click on the suggest features box



Feature Selection Method

Filter Method (recommended) ▼

Number of Features (0=auto) 7

Set Size Penalty 0.10

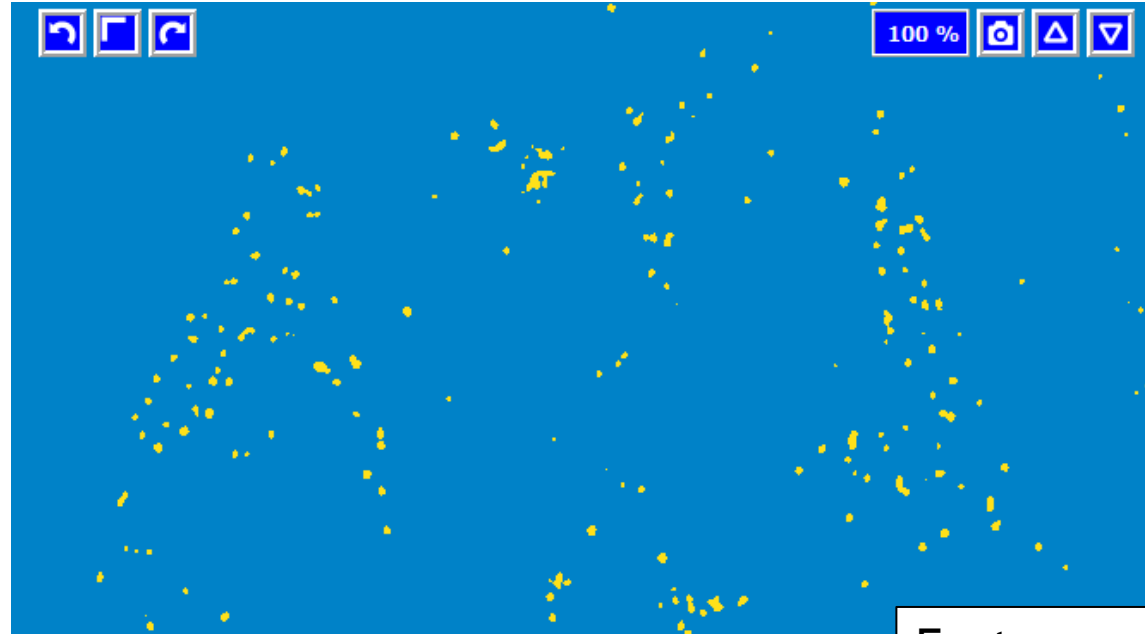
Run Feature Selection

1) Choose the feature selection method

- Gini Importance: inaccurate but fast
- Filter Method: recommended
- Wrapper Method: slow but provides the best results

2) Choose the parameters

- choose number of features: more features need more time and RAM, but provide better results. To select the number of features automatically, set this number to 0 (selection will take a while).



out-of-bag (oob) error is an internal metric generated by the Random Forest classifier to measure the accuracy of your model

all features, oob_error=2.644, computation time=0.516
user features, oob_error=1.202, computation time=0.391
5 features, filter selection, oob_error=3.365, computation time=0.183
all features, oob_error=2.644, computation time=0.516

Compare easily

user features	$\alpha=100.0$
5 features, filter selection	$\alpha=100.0$
all features	$\alpha=100.0$
raw_data	$\alpha=100.0$

Feature names

5 features, filter selection, oob_error=3.365, computation time=0.183 Show Feature Names

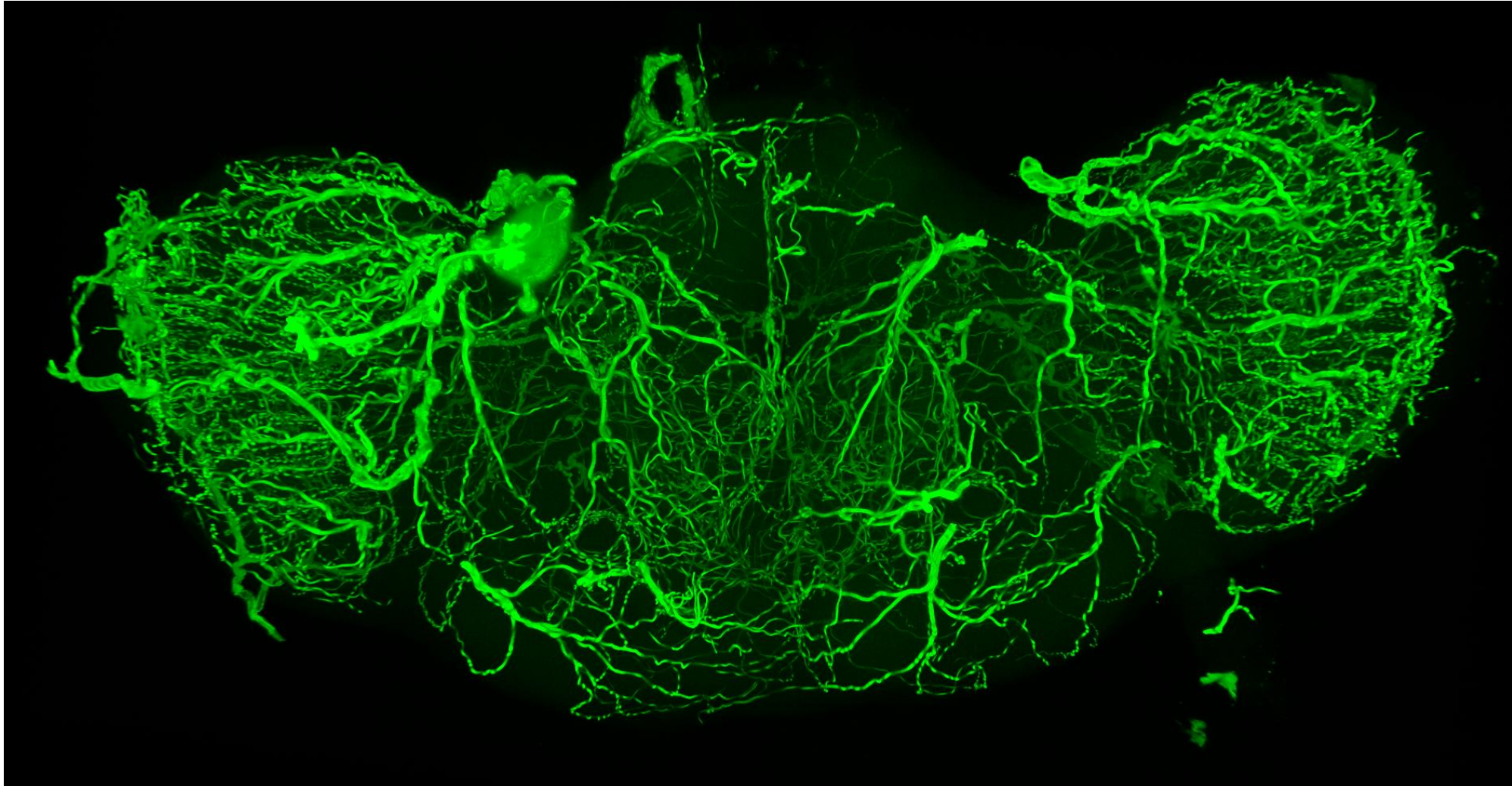
python

Gaussian Smoothing ($\sigma=10.0$) in 2D
Structure Tensor Eigenvalues ($\sigma=1.0$) in 2D [0]
Structure Tensor Eigenvalues ($\sigma=10.0$) in 2D [0]
Hessian of Gaussian Eigenvalues ($\sigma=1.6$) in 2D [1]
Hessian of Gaussian Eigenvalues ($\sigma=20.0$) in 2D [0]

ok

Exercise: Trachea segmentation

The fruit fly brain trachea dataset - Miranda Dyson

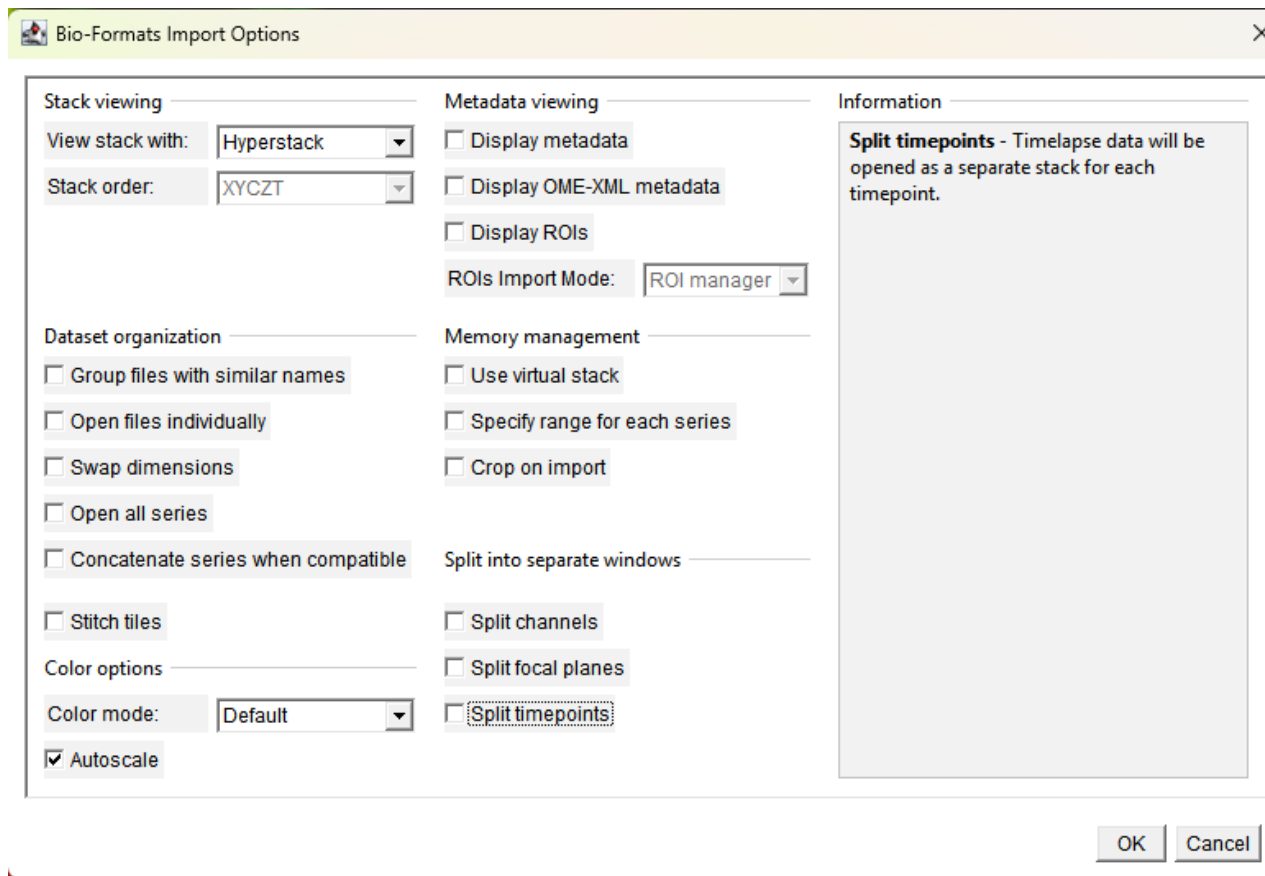


How do we a train a classifier for larger images? -> Generate a training dataset

Generating a training dataset

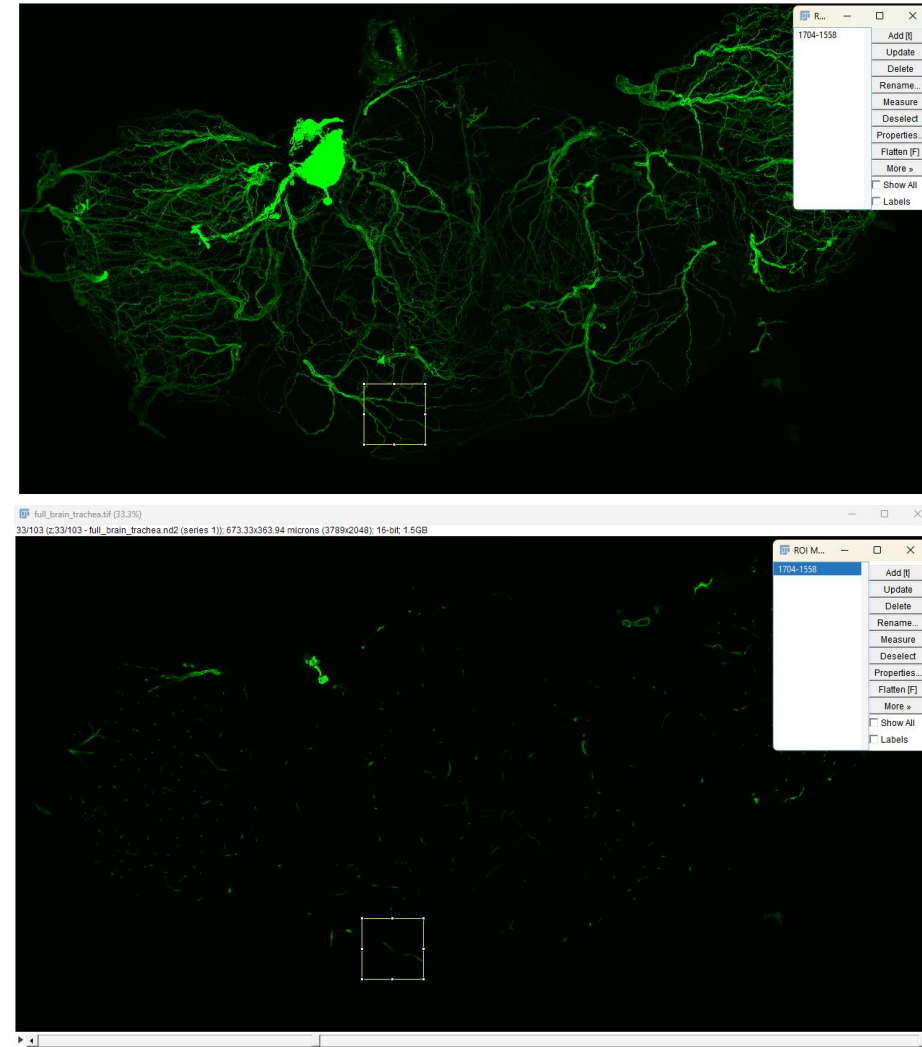
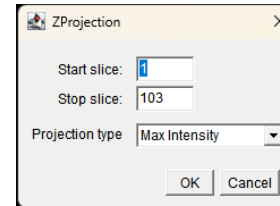
- Open FIJI
- Drag and drop image:

\\ilastik_course\data\fly_trachea_dataset_MirandaDyson\full_brain_trachea.tif



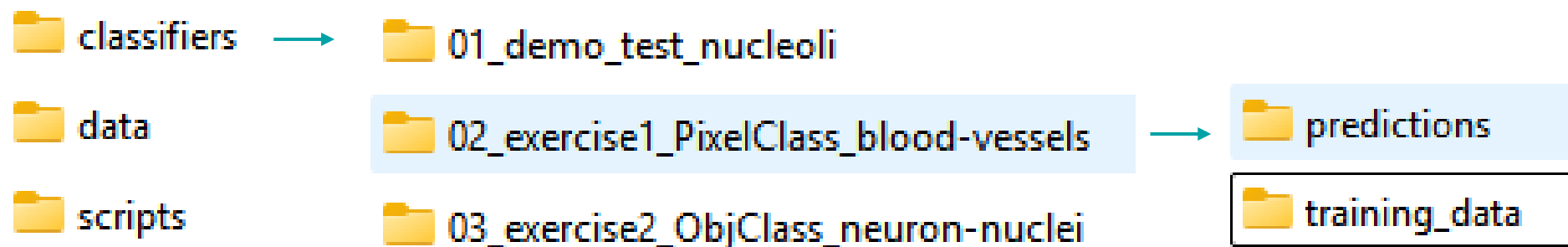
Generating a training dataset

- Create an MIP to facilitate choosing a ROI:
 - Image->Stacks->Z-Project
- Draw a small rectangle with trachea
- Add to ROI Manager with Ctrl + T
- Go back to the stack and select the ROI to add it to the image
- Duplicate with Ctrl + Shift + D



Generating a training dataset

- Save as .tif in a training_data folder:
- Always have a folder architecture:



Batch convert tifs to HDF5 chunky format

Create New Project

- ▼ Segmentation Workflows
 - ➔ Pixel Classification
 - ➔ Autocontext (2-stage)
 - ➔ Neural Network Classification (Local)
 - ➔ Neural Network Classification (Remote)
 - ➔ Trainable Domain Adaptation (Local)
 - ➔ Carving
 - ➔ Boundary-based Segmentation with Multicut
- ▼ Object Classification Workflows
 - ➔ Object Classification [Inputs: Raw Data, Pixel Prediction Map]
 - ➔ Object Classification [Inputs: Raw Data, Segmentation]
- ▶ Tracking Workflows
- ▼ Other Workflows
 - ➔ Cell Density Counting
 - ➔ Data Conversion
 - ➔ Pixel Classification + Object Classification (deprecated)

Drag and drop your tifs to the batch processing window and process all files

1. Input Data

2. Data Export

3. Batch Processing

Input Data

Select Input Data Files...

D:/Ongoing_projects/bic_projects/ilastik_course/classifiers/03_exercise1_PixelClass_trachea/training_data/large.tif
D:/Ongoing_projects/bic_projects/ilastik_course/classifiers/03_exercise1_PixelClass_trachea/training_data/low.tif
D:/Ongoing_projects/bic_projects/ilastik_course/classifiers/03_exercise1_PixelClass_trachea/training_data/medium.tif
D:/Ongoing_projects/bic_projects/ilastik_course/classifiers/03_exercise1_PixelClass_trachea/training_data/small.tif

Select the input files for batch processing using the controls on the right.
The results will be exported according to the same settings you chose in the interactive export page above.

Note: the project will be saved prior to running batch processing.

Process all files

Exercise

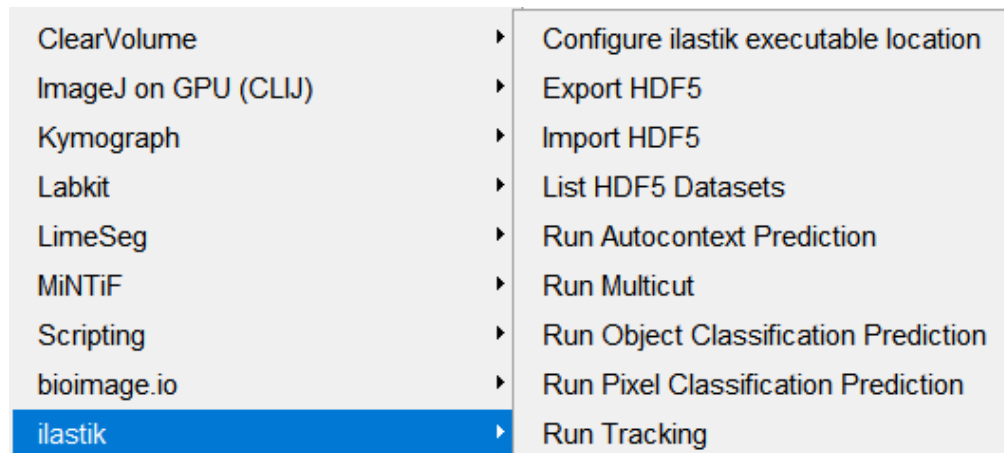
- Create a pixel classifier with your training data (h5 files)
- Find the right features and train a classifier that segments full trachea (lumen included)
- For your annotations:
 - Annotate sparsely: images/regions
 - Use the uncertainty layer to guide your strokes
 - Annotate boundaries between classes

Exercise

- Now it's time to test your classifier!
- Apply your model to another dataset of trachea using the batch analysis.

Dataset can be found in: \ilastik_course\classifiers\03_exercise1_PixelClass_trachea\training_data

- Verify the results in FIJI

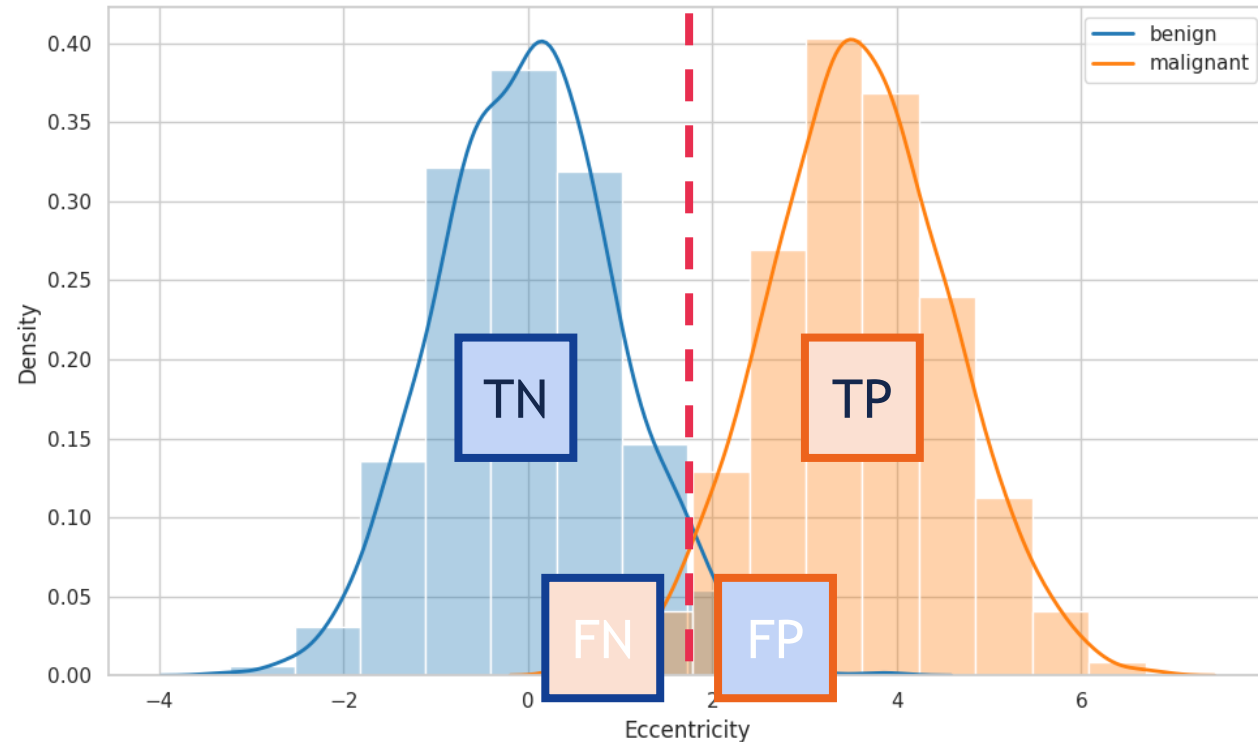




How can we evaluate our model?
(supervised learning)



Evaluation metrics



Confusion matrix

Classified as ...		positive	negative
Really is ...	positive	TP	FN
	negative	FP	TN

Jaccard similarity coefficient

$$J(A, B) = \frac{|A \cap B|}{|A \cup B|} = \frac{\text{Intersection}}{\text{Union}}$$
$$J(A, B) = \frac{\text{TP}}{\text{TP} + \text{FP} + \text{FN}}$$

Model evaluation

- Drag and drop the pixel_classifier_script.ijm to FIJI and click run. Location: \ilastik_course\scripts
- Run your train classifier but this time on the .tif data:
\\ilastik_course\classifiers\03_exercise1_PixelClass_tra
chea\training_data
- The ground truth can be found here:
\\ilastik_course\classifiers\03_exercise1_PixelClass_tra
chea\groundtruth
- This script will perform pixel classification as well as testing the pixel classifier

Script parameters

File location and format

File directory: D:\Ongoing_projects\bic_projects\ila

File suffix: .tif

Pixel Classifier Parameters

Pixel classifier path: D:\Ongoing_projects\bic_projects\ila

Channel to segment. Use 0 for all channels: 1

File suffix: Probabilities

Probability threshold for segmentation (if probabilities output was selected): 0.4000000059604645

Pre-processing parameters prior classification

Normalize intensity? ☐

Normalization parameters. Paste from recorder: saturated=0.1 normalize process_all use

XY scaling factor: 1

Z scaling factor: 1

Use interpolation ☐

Pixel classifier validation

Test pixel classifier? (Ground truth necessary) ☐

File directory: D:\Ongoing_projects\bic_projects\ila

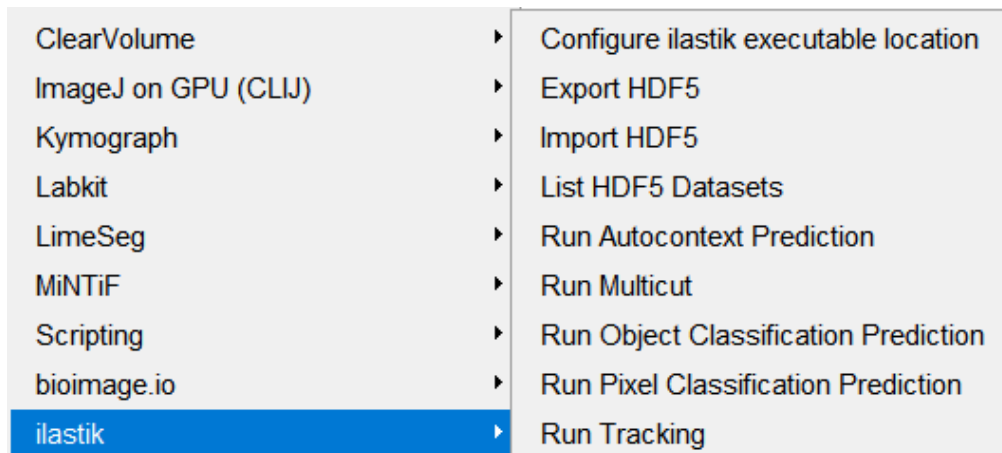
Ground truth files suffix: .tif

Exercise

- Now it's time to test your classifier!
- Apply your model to another dataset of trachea using the batch analysis.

Dataset can be found in: \ilastik_course\classifiers\03_exercise1_PixelClass_trachea\training_data

- Verify the results in FIJI



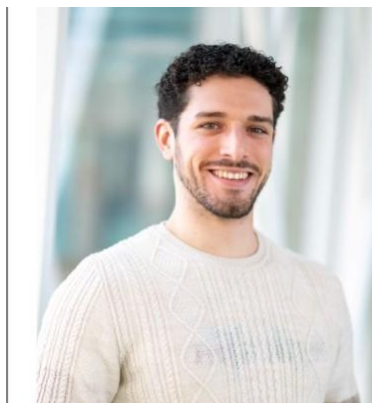
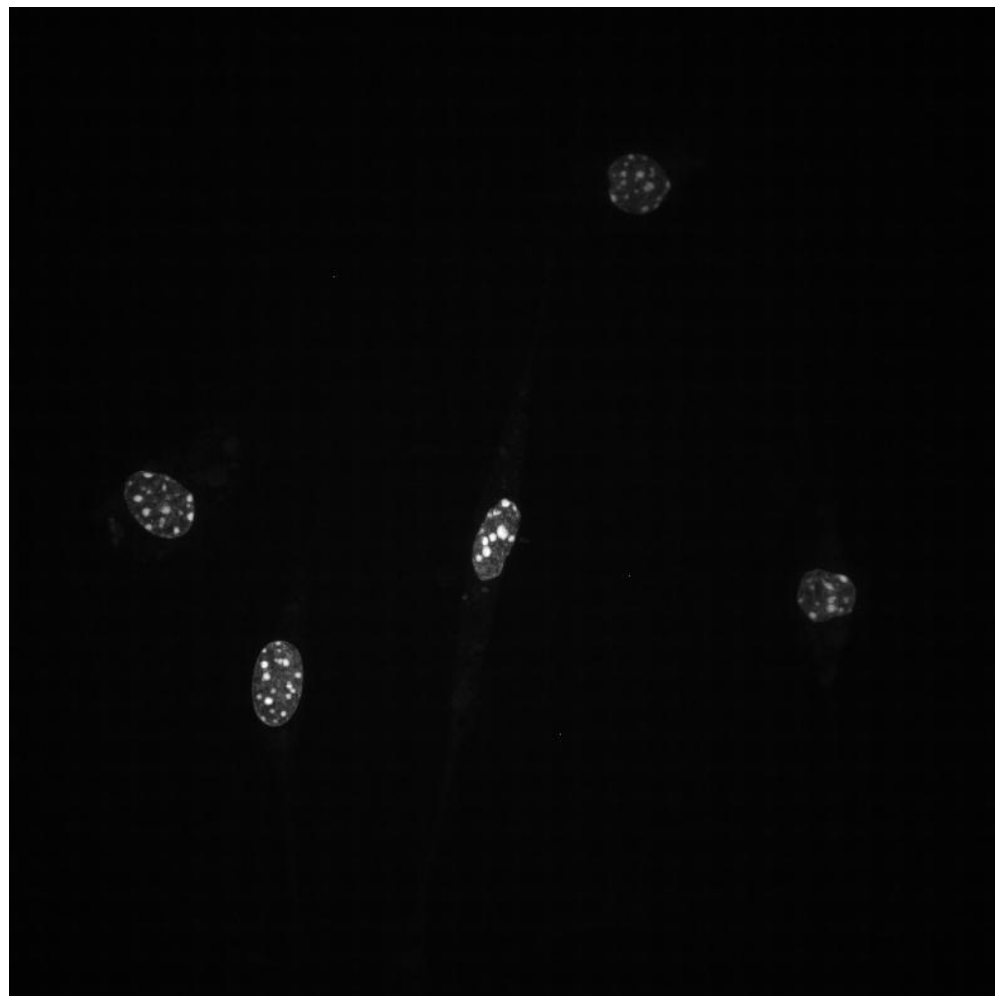
04 | Object classification

Pixel vs. Object Classification

- Most microscopy tasks require two main steps, and ilastik splits them into separate workflows:
- **Pixel Classification (Semantic Segmentation):** This answers the question: *What is this specific pixel?* You tell ilastik, "This pixel belongs to a cell nucleus, this pixel belongs to the cytoplasm, and this pixel is empty background." It does not separate touching cells; it just color-codes the pixels by category to create a **Probability Map**.
- **Object Classification (Instance Segmentation):** This answers the question: *Is this entire clump one cell or two?* It takes your Probability Map, turns it into distinct objects, and allows you to classify full objects based on size, shape, or intensity (e.g., distinguishing "healthy cells" from "apoptotic cells").

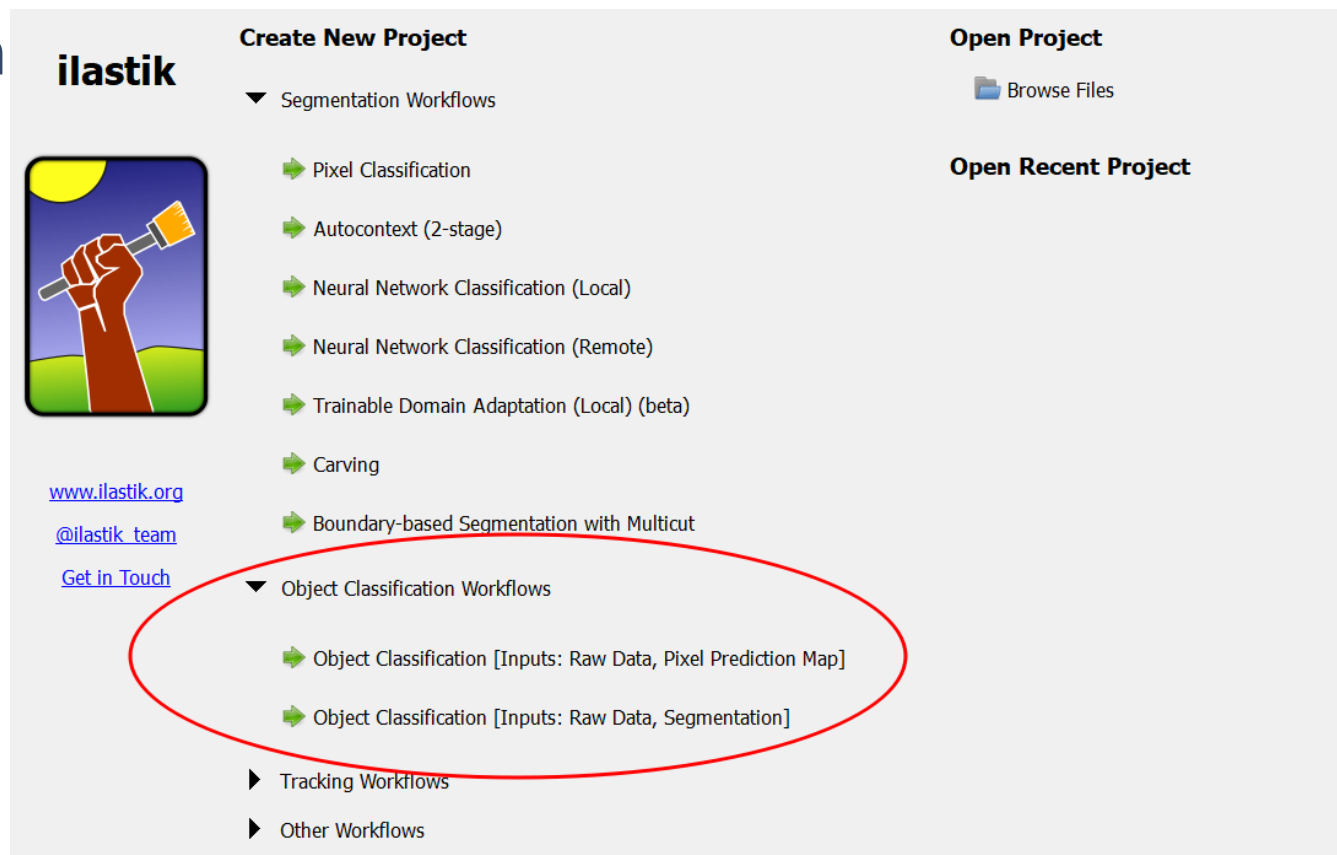
Training a pixel classifier in Ilastik:

3D nuclei dataset - sample from Alessio Silva



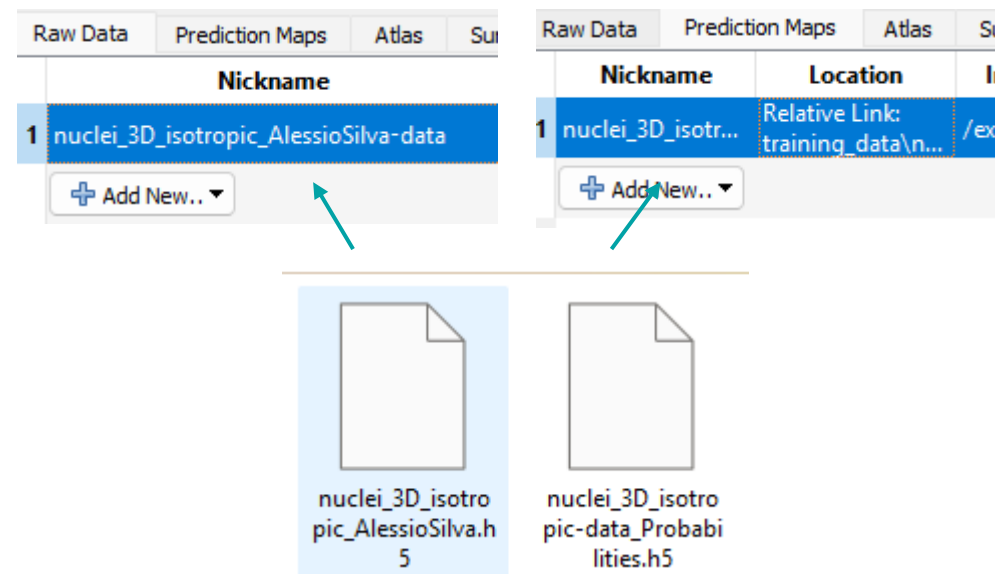
Creating an Ilastik Object Classifier Model

- Create new object classification project
- There are two pipelines:
 - Object creation from probability map + classification
 - Objects segmented by the user elsewhere + classification



Input data

- Add the intensity images in the raw data section (this data can include more channels than the data used for training the pixel classifier)
- Add the corresponding probability images in the prediction maps section



Object Creation: Threshold and Size filter

- If you have objects that are not touching. Use the Simple method.
- The input should be the class you want to create an object from.
- You can smooth the probability map
- If you want to see the effects of the size filter and the smoothing check the “Show intermediate results” box.

 2. Threshold and Size Filter

Method:

Simple 

Input:

 0 

Smooth:

1.0   1.0   1.0  

Threshold:

0.50  

Size Filter

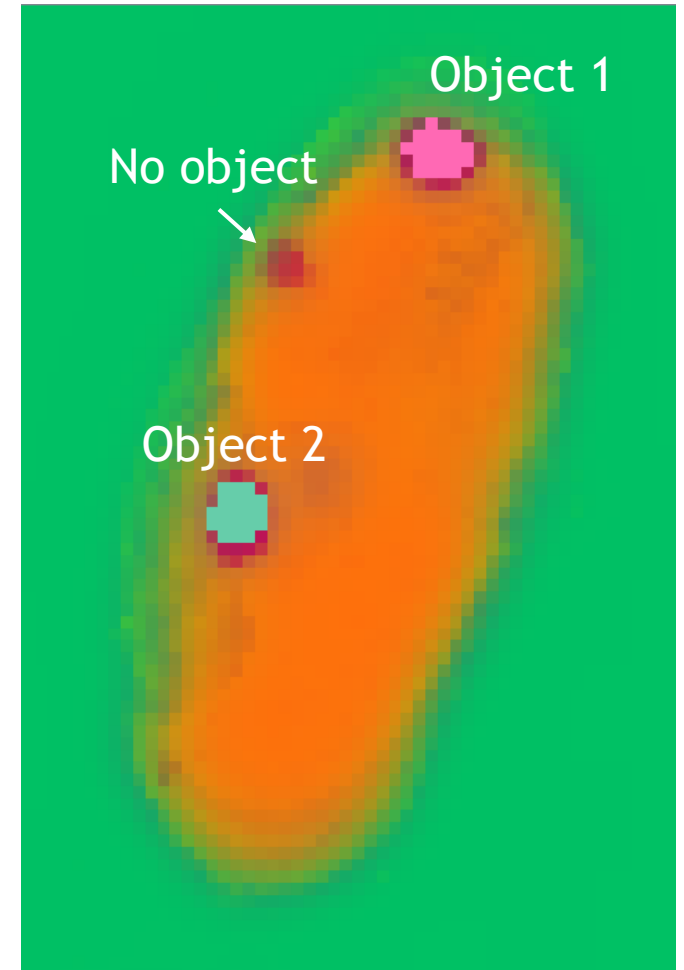
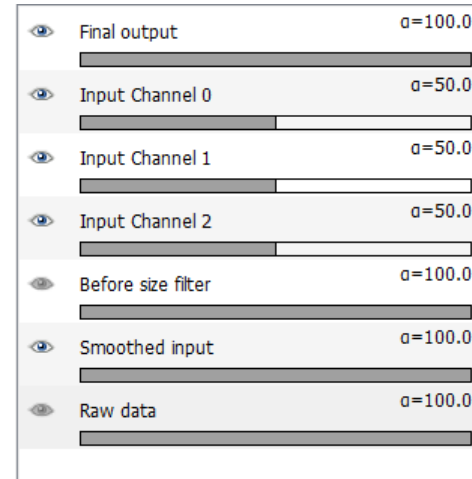
Min: 10  

Max: 1000000  

Apply

Viewer

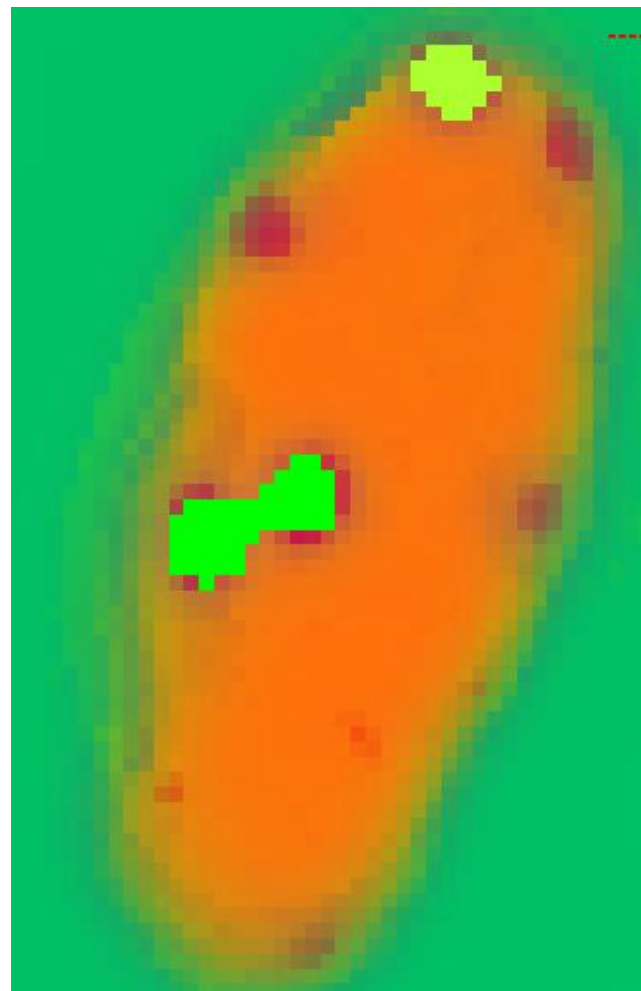
- This is how the viewer looks when “Show intermediate results is active”
- Separated objects will have individual labels and hence different colours



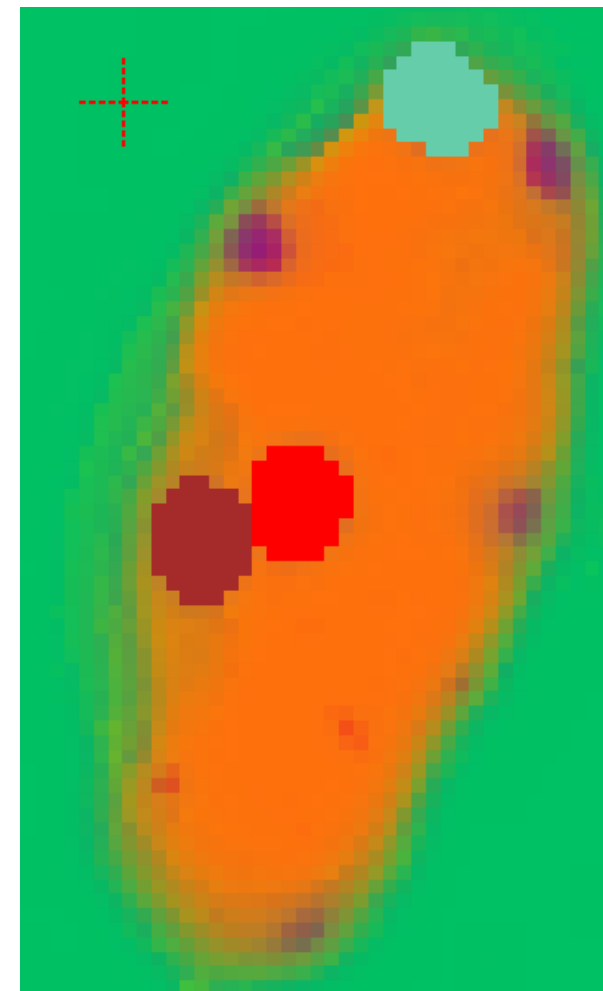
How to separate touching objects

- To separate touching objects change the Thresholding method to Hysteresis
- Hysteresis uses a higher threshold to create seeds and then grows them until reaching the second threshold. This method is known as marker seeded watershed aka seeded region growing.

Simple threshold

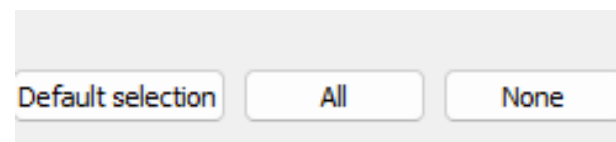
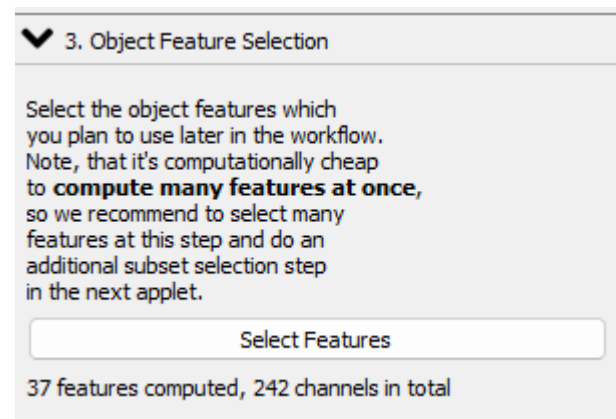


Hysteresis threshold

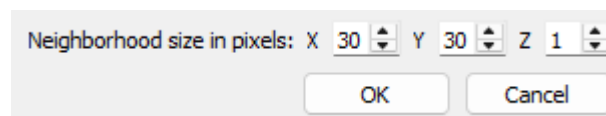
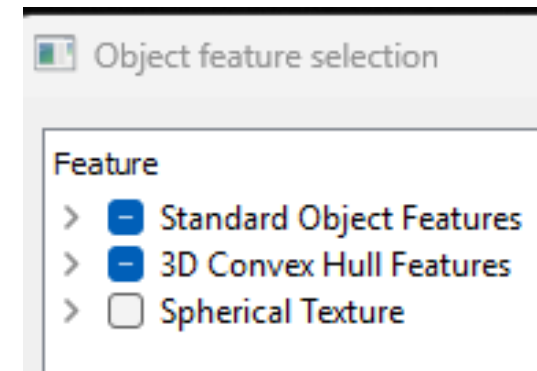


Object feature selection

- Now that we have objects, as with the pixel classifier we need to compute features to be able to train a classifier
- As before, try to think what would be the best features to differentiate your classes



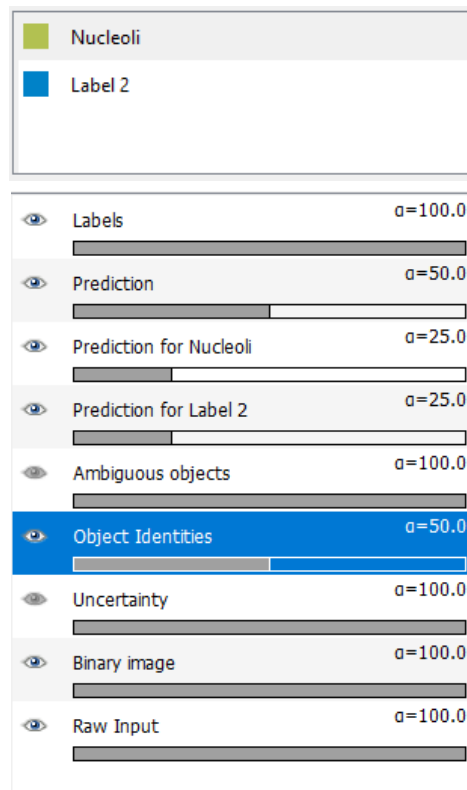
Default selection recommended



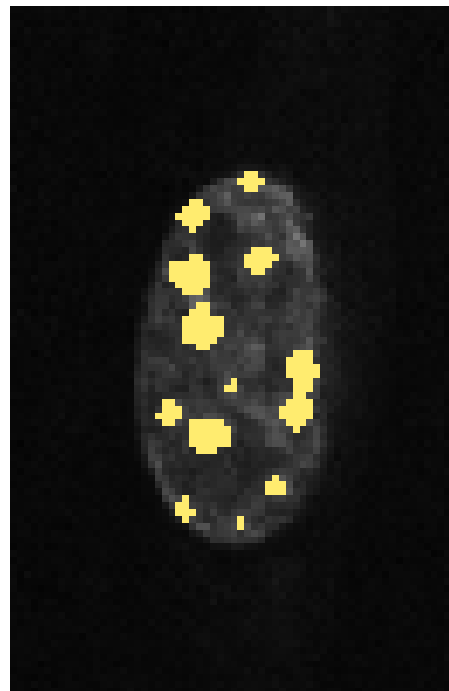
For neighbourhood features, the size can be changed

Object classification - Training the classifier

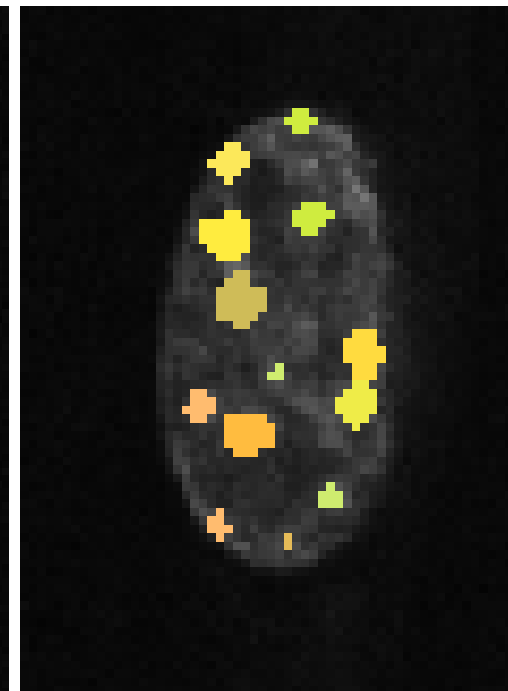
- Training is done similarly to the pixel classifier except here you click on objects instead of pixels.
- You can visualize objects as Binary image or with a random colour (by Object Identity)
- As before you can also use the uncertainty to guide your annotations.
- If you don't need the classification you can directly only annotate one time your class and export the object measurements



Prediction binary

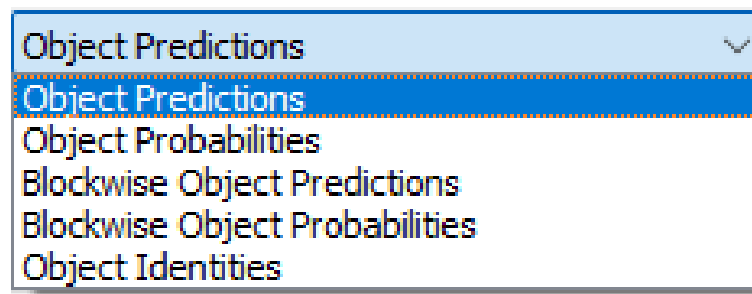
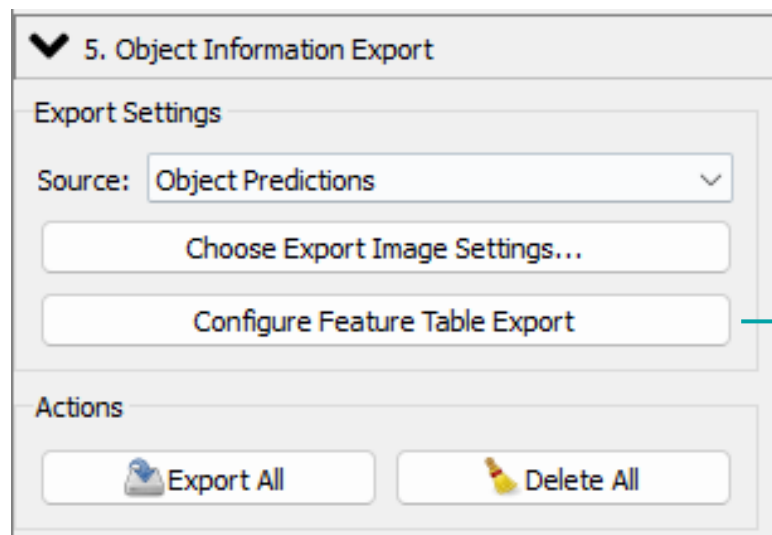


Object identity



Object information export

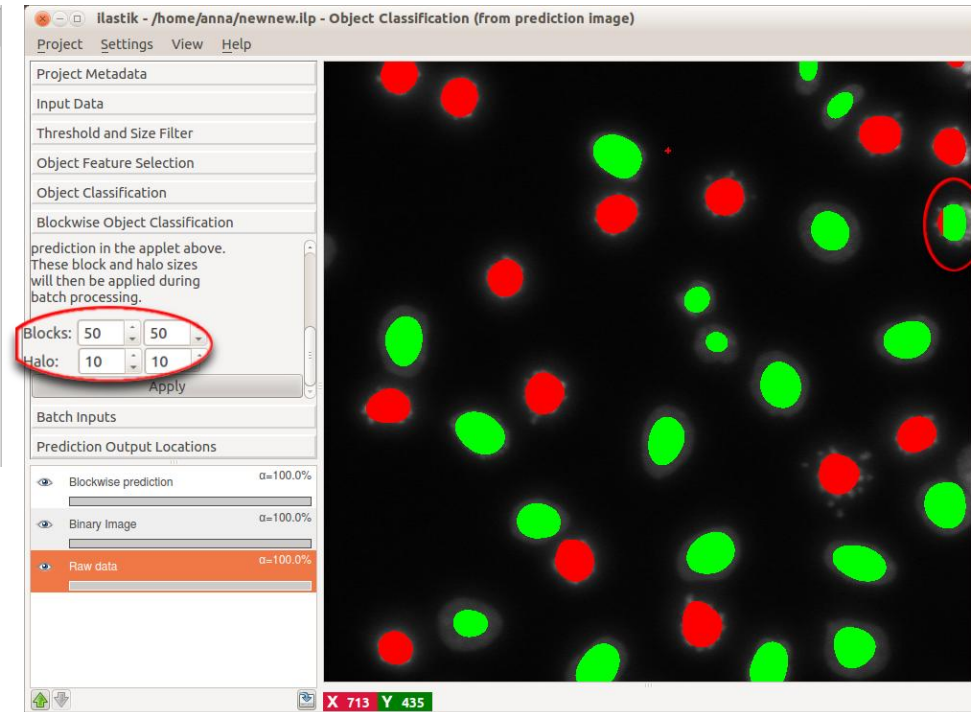
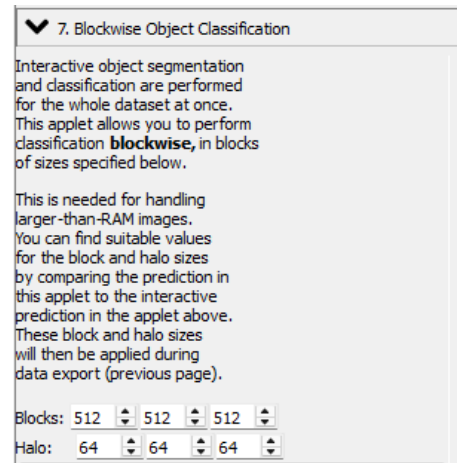
- You can export predictions or probabilities just like with the pixel classifier but you can also export the object identities to perform further measurements elsewhere.
- Ilastik can also export the feature table containing all of the selected features. Select the ones you want to export in “Configure Feature Table Export”
- This configuration will be used for the batch processing tab too



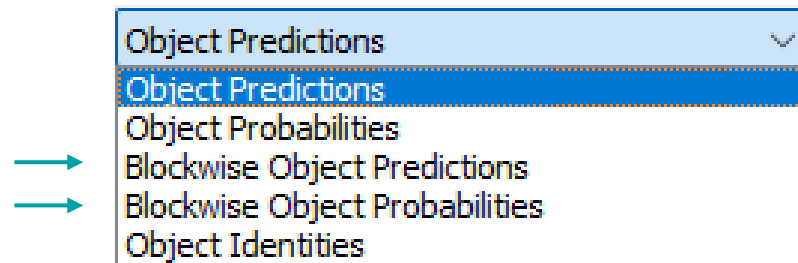
nuclei_3D_isotro
pic_AlessioSilva-
data_table.csv

Blockwise object classification

- Standard object classification in Ilastik is limited by RAM. If you are working with a dataset that is bigger than what your memory can use you need to perform the classification in blocks. You can do this with the “Blockwise Object Classification” method.
- Blocksize is important as it should be small enough to fit memory but large enough not to miss-segment objects.
- If you want to use it be sure to select blockwise in the object information export tab.

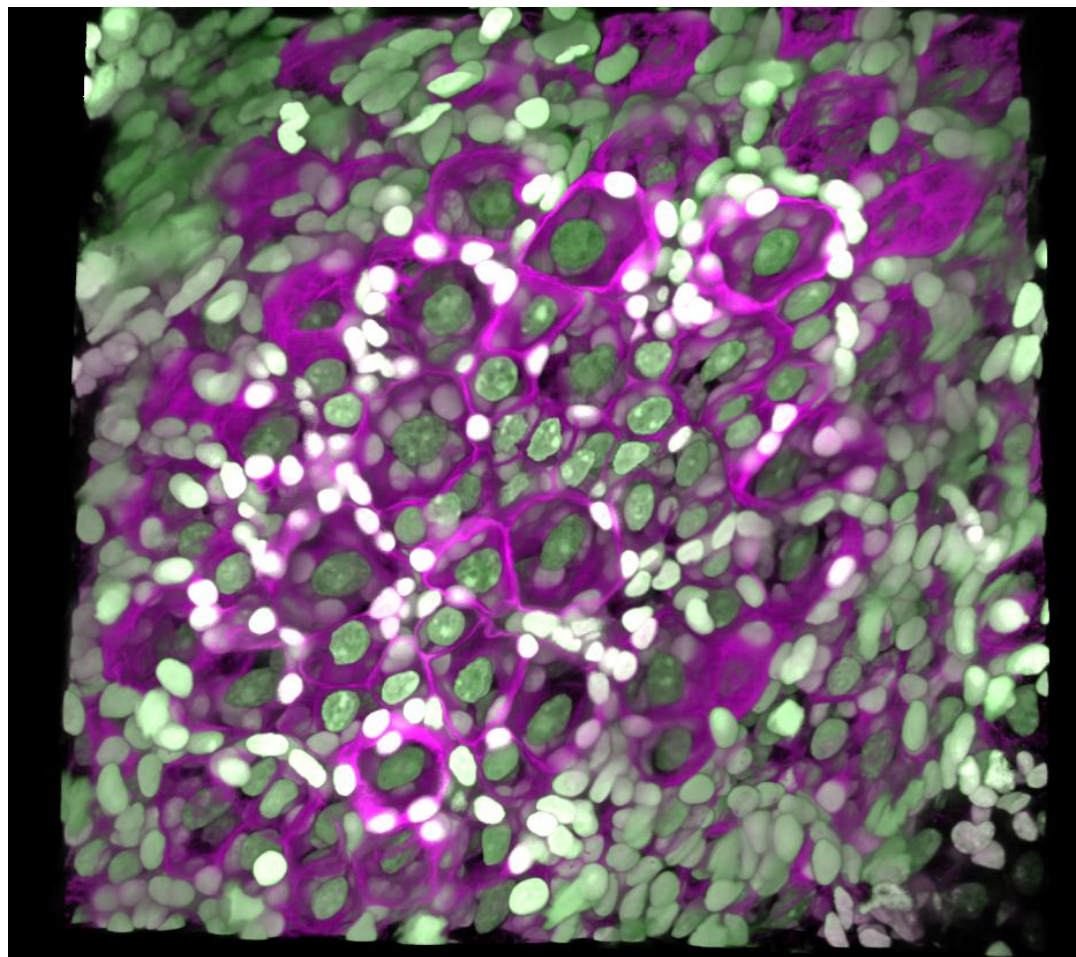


<https://www.ilastik.org/documentation/objects/objects>



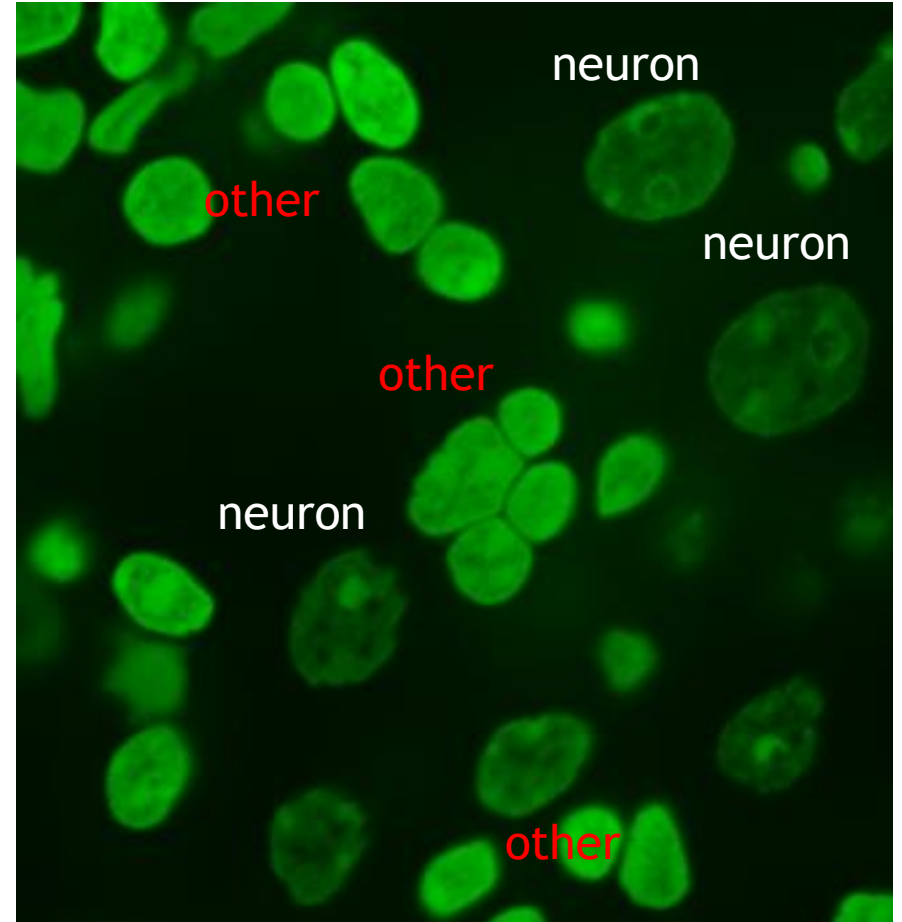
Exercise: Trachea segmentation

The dorsal root ganglia neurons dataset- Ana Nogueira Freitas Peigneur



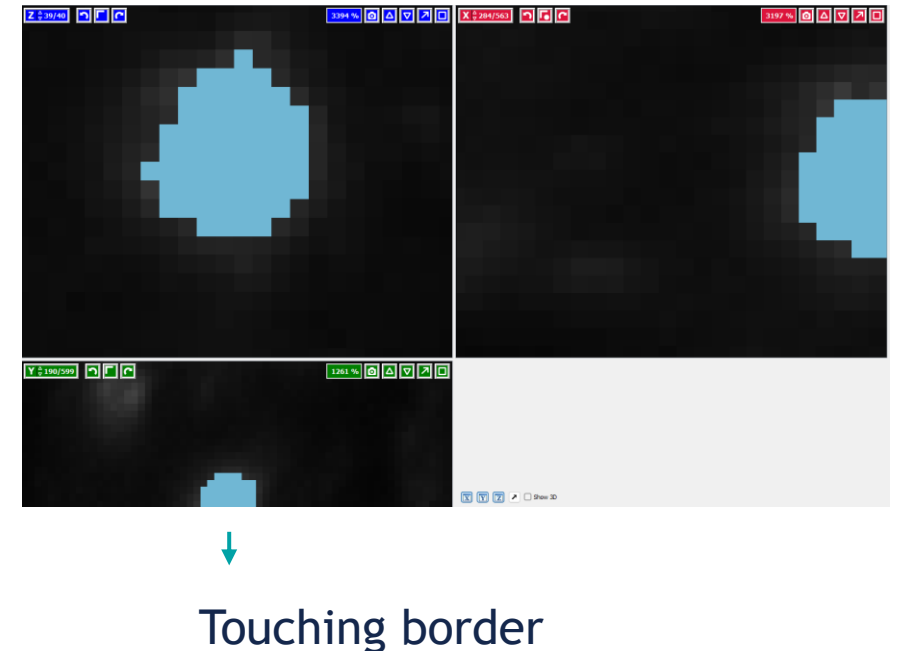
Exercise

- Now it's your turn!
- Create a pixel classifier to segment nuclei from the neuron_nuclei_3D_isotropic.tif dataset (transform it to H5 first to make Ilastik more efficient)
Dataset can be found in: ilastik_course\classifiers\06_exercise2_ObjClass_neuron-nuclei\training_data\neuron_nuclei_3D_isotropic.tif
- Use the raw image + probabilities to create nuclei instances and classify nuclei into Neuron or Other
- Find out the total number of neurons



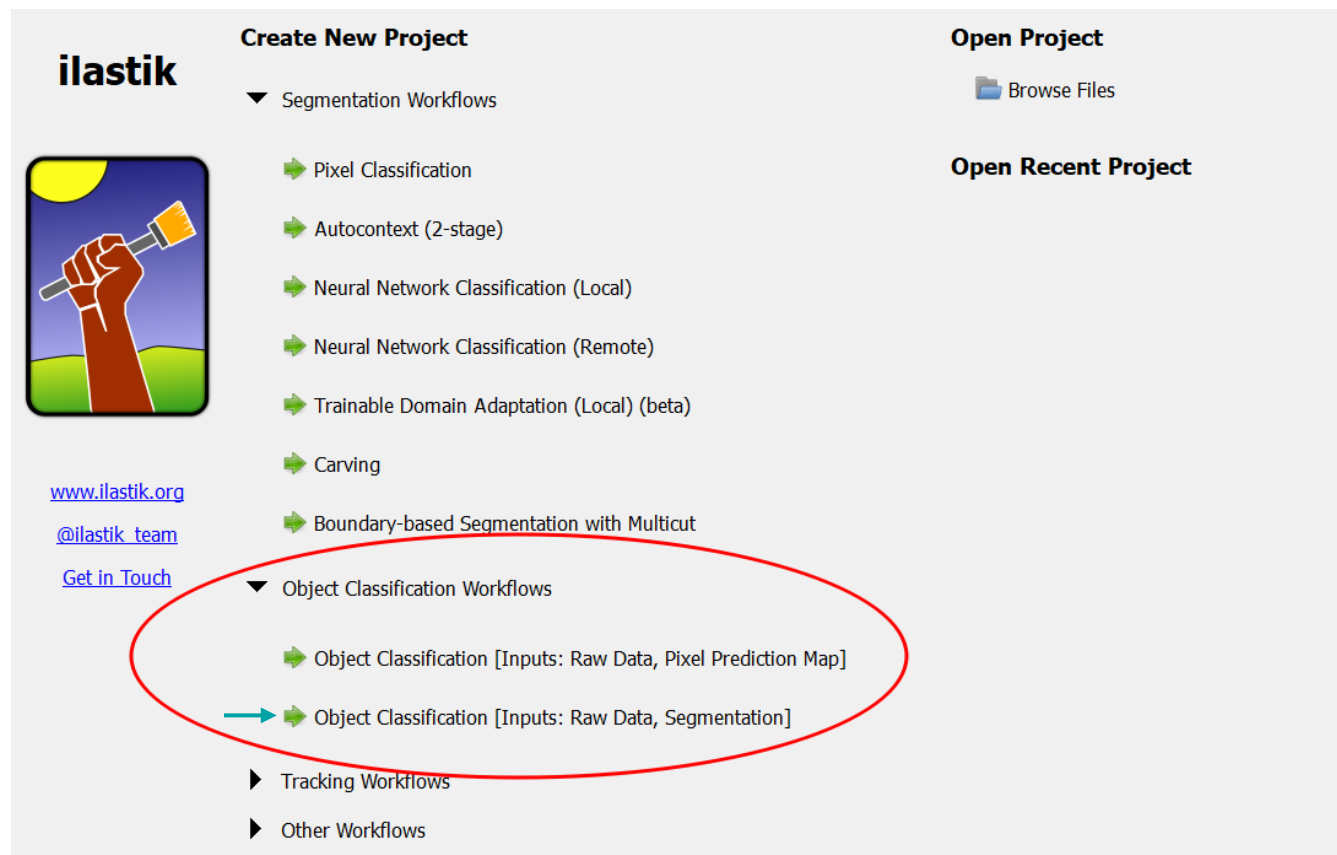
Importing objects segmented elsewhere in Ilastik

- When classifying objects it is extremely important to avoid having objects touching borders as these might confuse the classifier
- Also, we can use other methods (like deep learning models) to segment the objects and then classify them with Ilastik
- We can also use Ilastik to clean unwanted objects



Creating an Ilastik Object Classifier Model

- Create new object classification project
- This time select the Object Classification with Segmentation as an input

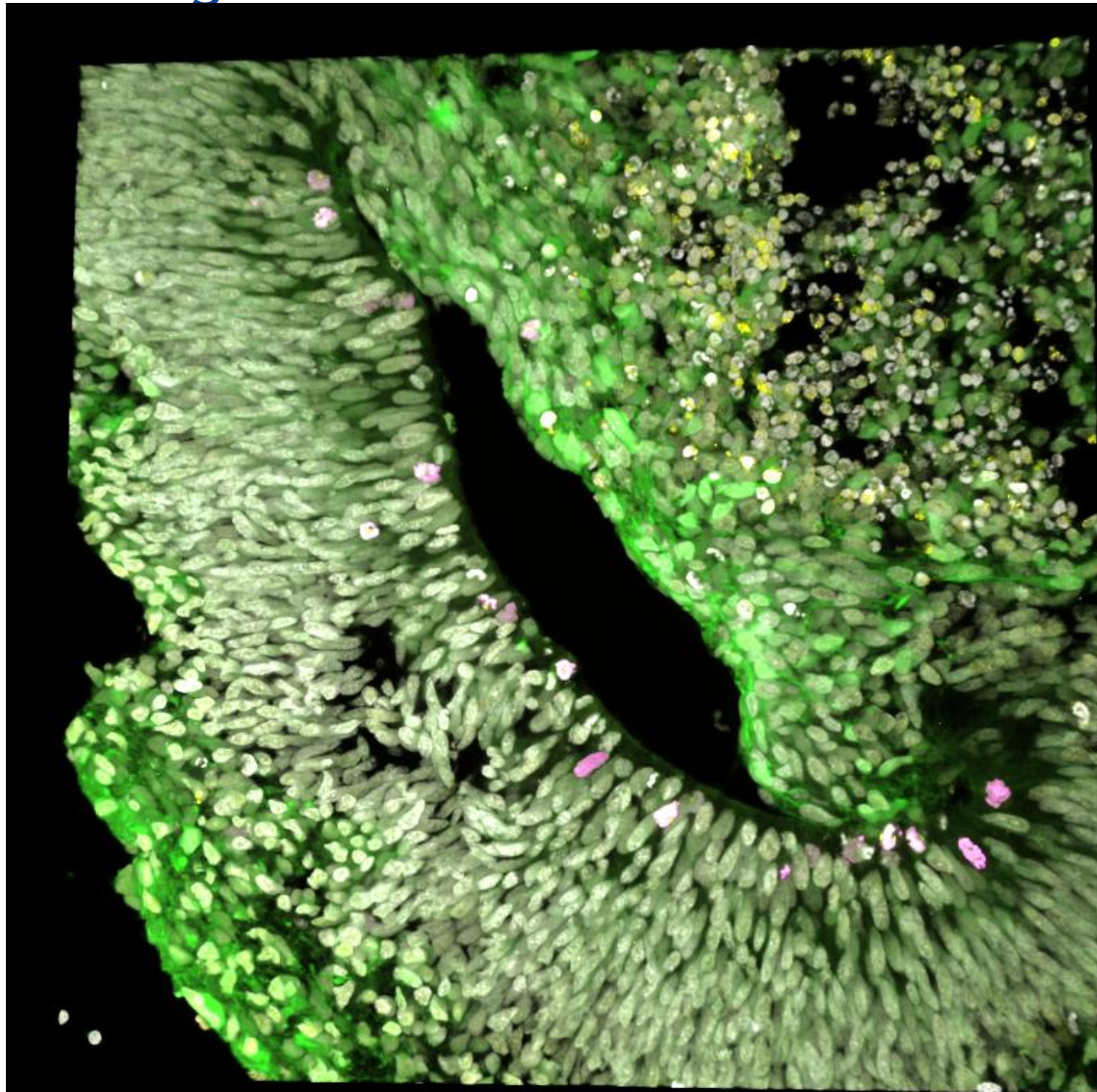


Demo: Importing cellpose segmented nuclei to Ilastik

- Use the same raw image as before but for the segmentation choose
- Find out the total number of neurons

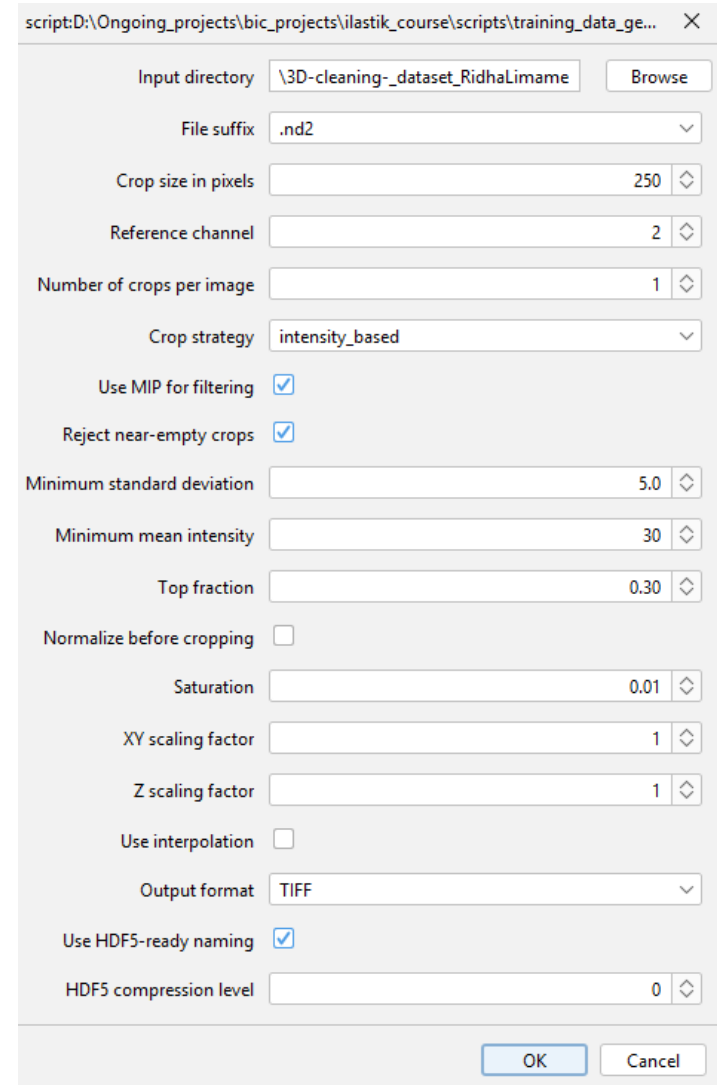
Exercise: Image segmentation cleaning

The organoid dataset- Ridha Limame



Creating training data for full datasets automatically

- Drag and drop the
\\ilastik_course\\scripts\\training_data_generation.ijm
- Adjust the parameters for your data generation



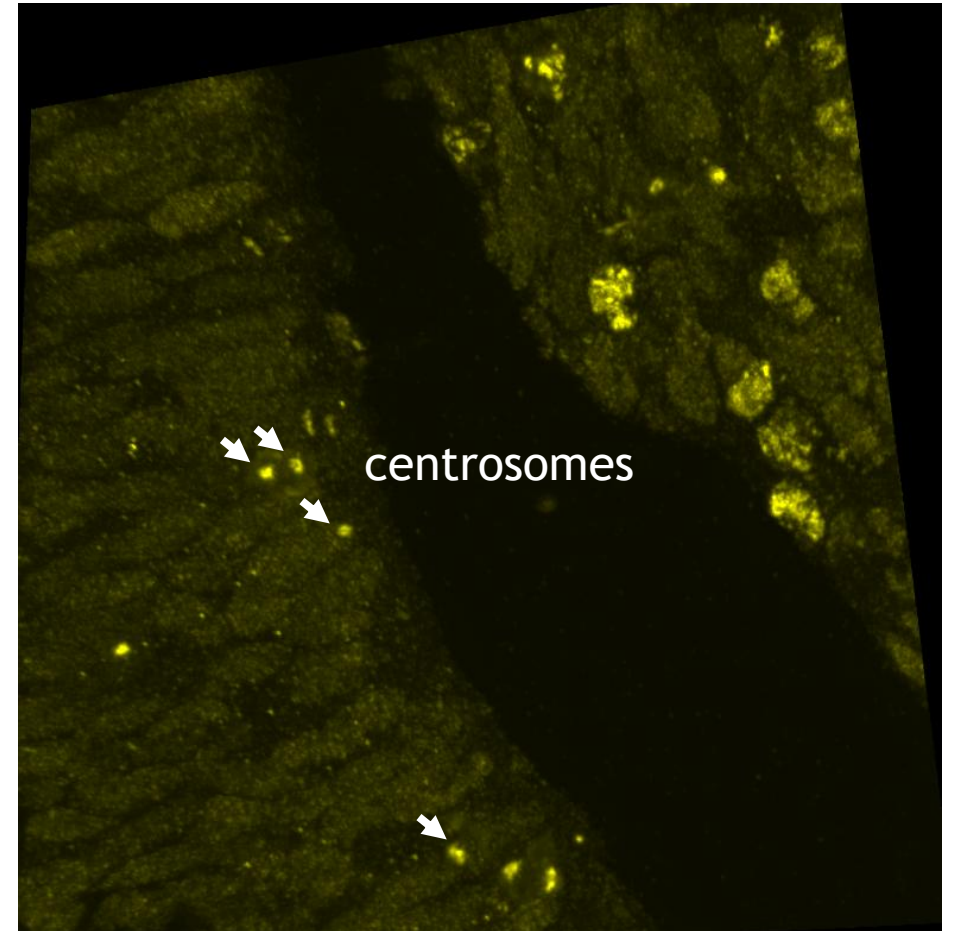
The screenshot shows a dialog box titled "script:D:\\Ongoing_projects\\bic_projects\\ilastik_course\\scripts\\training_data_ge...". It contains various settings for generating training data. The settings are as follows:

Parameter	Value
Input directory	\\3D-cleaning-_dataset_RidhaLimame
File suffix	.nd2
Crop size in pixels	250
Reference channel	2
Number of crops per image	1
Crop strategy	intensity_based
Use MIP for filtering	☒
Reject near-empty crops	☒
Minimum standard deviation	5.0
Minimum mean intensity	30
Top fraction	0.30
Normalize before cropping	☐
Saturation	0.01
XY scaling factor	1
Z scaling factor	1
Use interpolation	☐
Output format	TIFF
Use HDF5-ready naming	☒
HDF5 compression level	0

At the bottom right, there are "OK" and "Cancel" buttons.

Exercise

- Generate training dataset
- Train pixel classifier
- Train object classifier from probabilities to clean the image
- Check the results in FIJI



05 Conclusion

Segmentation strategies: strengths & trade-offs

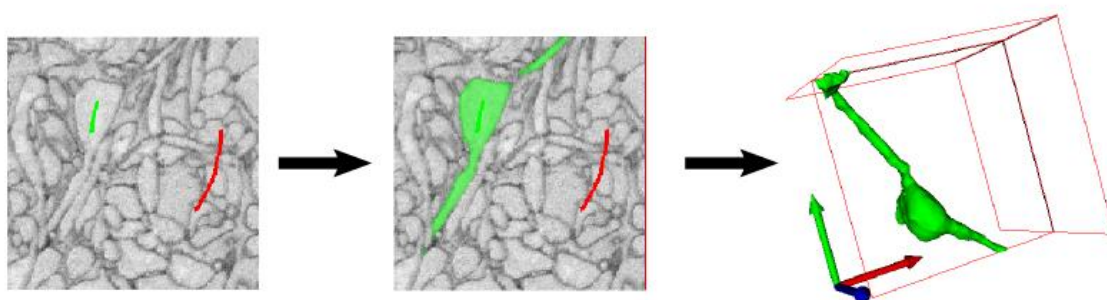
From rules → examples → learned representations

Dimension	Classic / rule-based	Supervised ML (e.g. Ilastik / Random Forest)	Deep learning (e.g. U-Net / StarDist)
Typical input	Thresholds, filters, morphology, watershed, manual parameter choices.	Small set of annotated pixels/objects + selected image features.	Many annotated images or pretrained model + fine-tuning data.
Advantages	Transparent, fast, no training data. Good when signal is clean and object definition is simple.	Interactive and data-efficient. Learns from examples; handles more subtle texture/intensity cues.	Highest flexibility and accuracy on complex images. Learns features automatically; strong generalization when trained well.
Limitations	Brittle when staining, background, density, or shape changes. Parameters often fail across datasets.	Depends on informative features and representative labels. Can overfit; struggles with unseen image styles.	Needs more data, compute, validation, and expertise. Less transparent; can fail silently outside training domain.
Best use case	Clean nuclei, bright objects, simple masks, quick measurements, preprocessing steps.	Biological classes separable by texture/intensity but hard to threshold manually.	Variable objects, crowded scenes, weak boundaries, instance segmentation at scale.
Teaching takeaway	“Can I describe the object with rules?”	“Can good labels + good features separate classes?”	“Do I have enough examples to learn the representation?”

Rule of thumb: use the simplest approach that is robust across your biological variability – then validate on images the model has not seen.

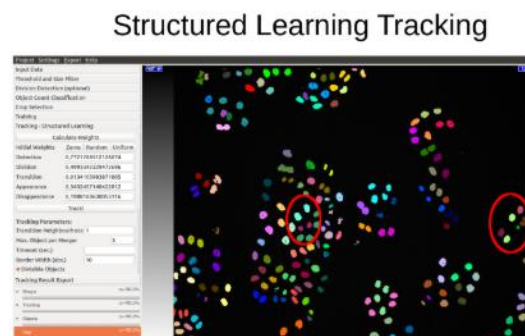
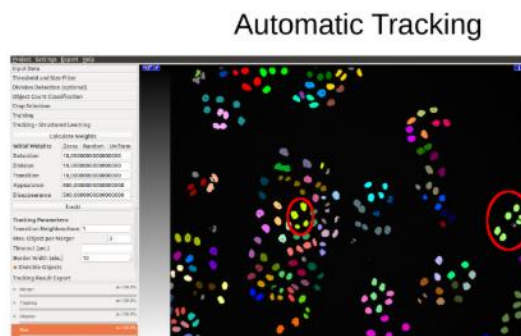
Further learning

EM boundary based segmentation in 3D



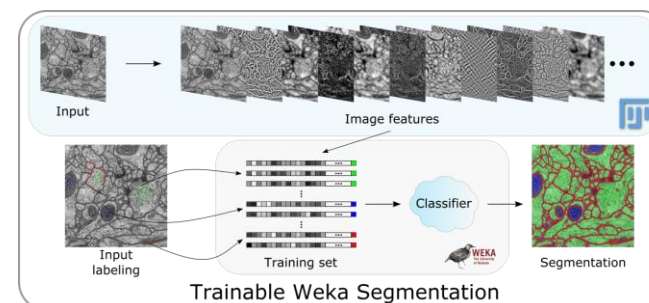
<https://www.ilastik.org/documentation/carving/carving>

Tracking

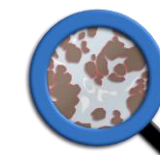


<https://www.ilastik.org/documentation/tracking/tracking>

Other classifiers



 ilastik-napari



<https://imagej.net/plugins/tws/>
<https://github.com/ilastik/ilastik-napari>
<https://qupath.github.io/>

ilastik: interactive machine learning for image analysis

Anna Kreshuk, EMBL

<https://www.youtube.com/watch?v=F6KbJ487iiU>
<https://www.ilastik.org/documentation/>

Thank you for your attention



technologies@vib.be - vib.be/technologies

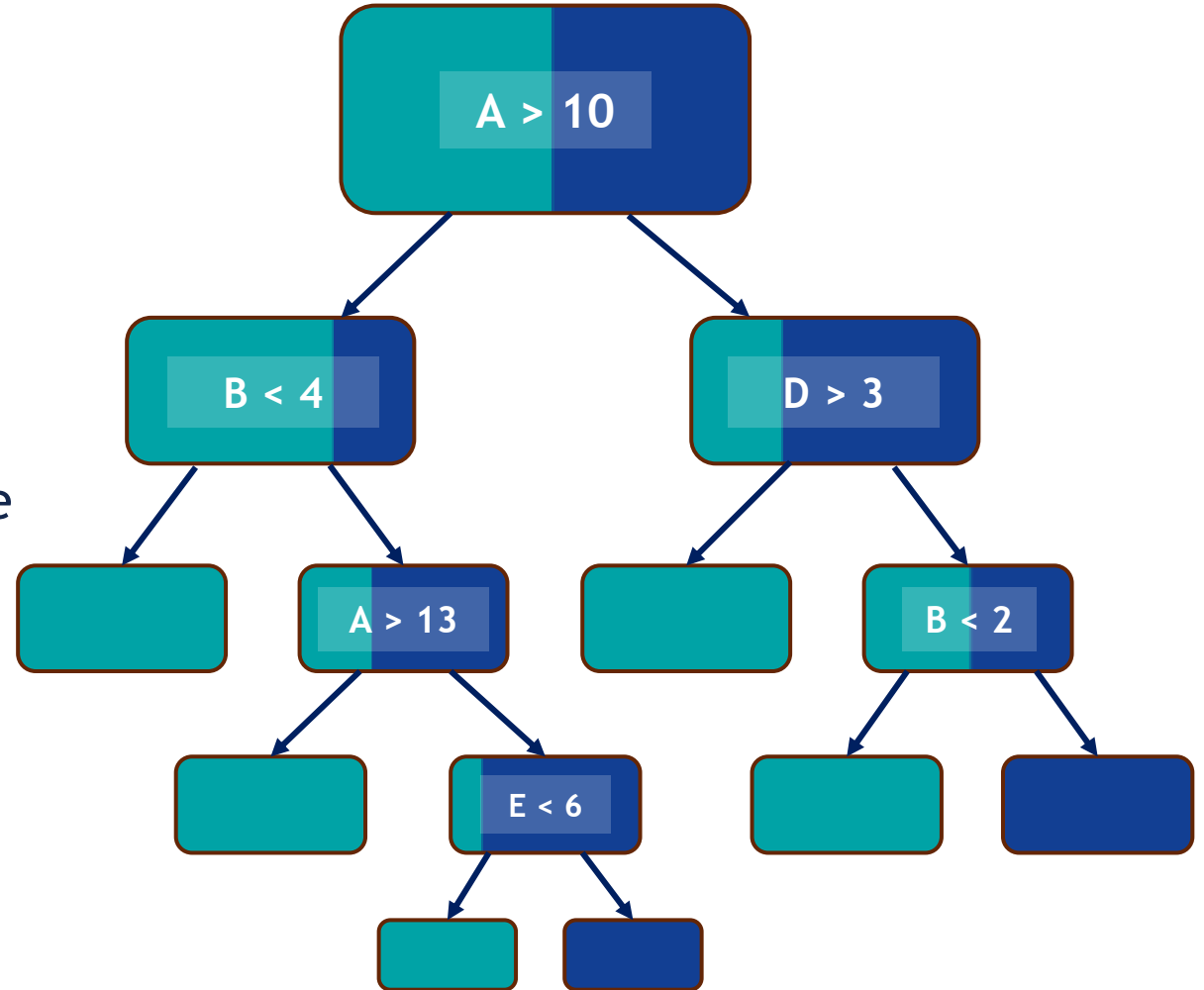


Extra slides



Decision tree

- Models decisions and their possible consequences in a tree-like structure
 - Nodes = features
 - Branches = decision rules
 - Leaves = outcomes
- **Recursively split data** based on feature value
 - Impurity (Entropy, Gini index)
 - How *mixed* the classes are
- **High interpretability**, but **prone to overfitting**
 - Max tree depth
 - Pruning





**What does the decision boundary of a
decision tree look like?**



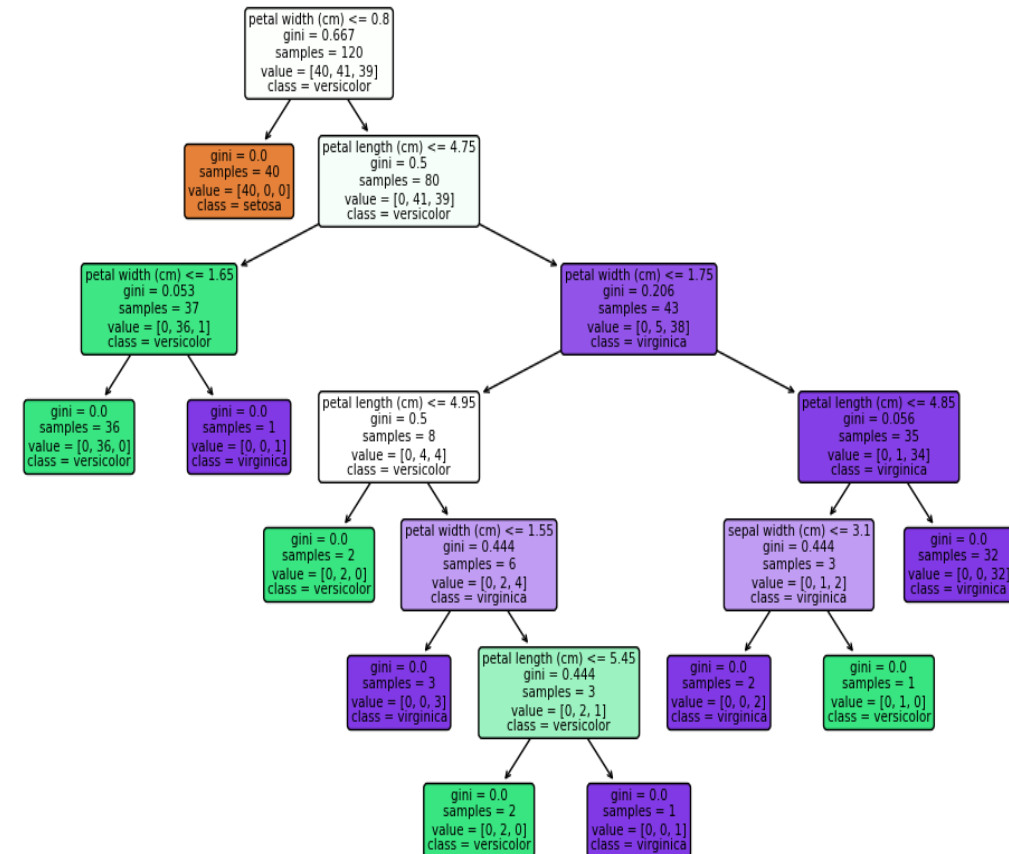
Decision tree

Pro's

- Simple to understand (**white box**)
- Require little data preprocessing
- Automatically handles non-linear relationships
- **Computationally efficient**
- Flexible output (classification, regression, multi-class)

Cons

- **High risk of overfitting (greedy)**
- Large trees become hard to interpret
- No extrapolation (outliers!)





Decision tree

- **Early stopping**

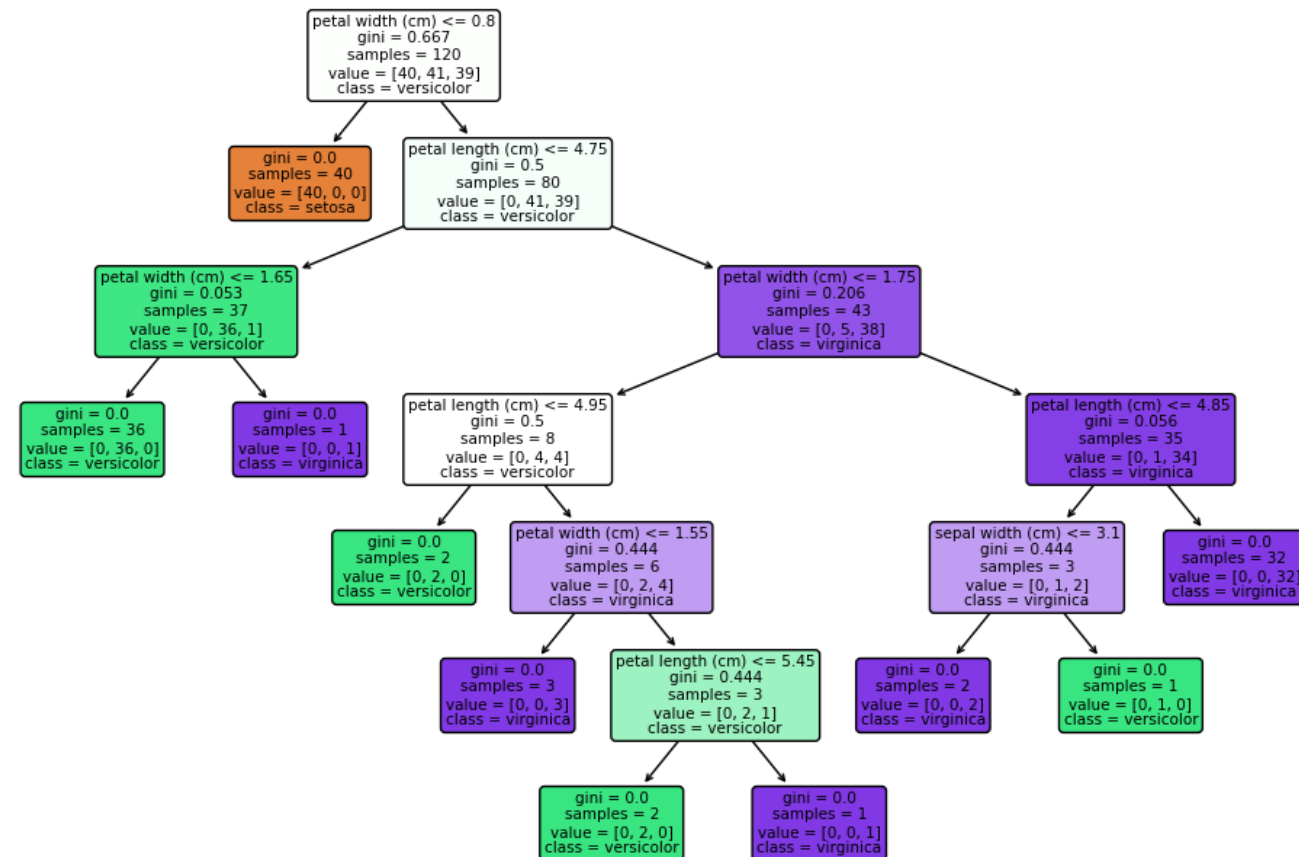
- Limit the tree depth
- Minimum samples in leaves
- Minimum impurity decrease

- **Pruning**

- cut back branches that do not improve generalization

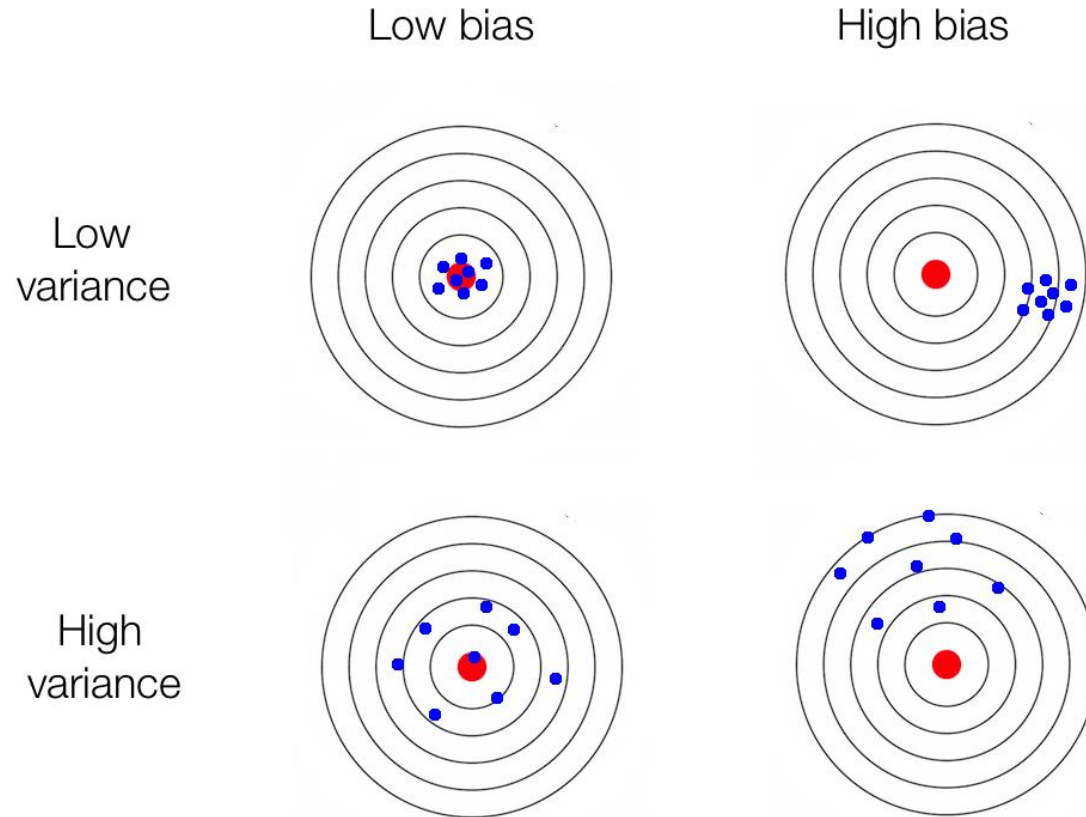
- **Ensemble methods**

- Bagging (RandomForest)
- Boosting (AdaBoost)





Ensemble methods



Bias

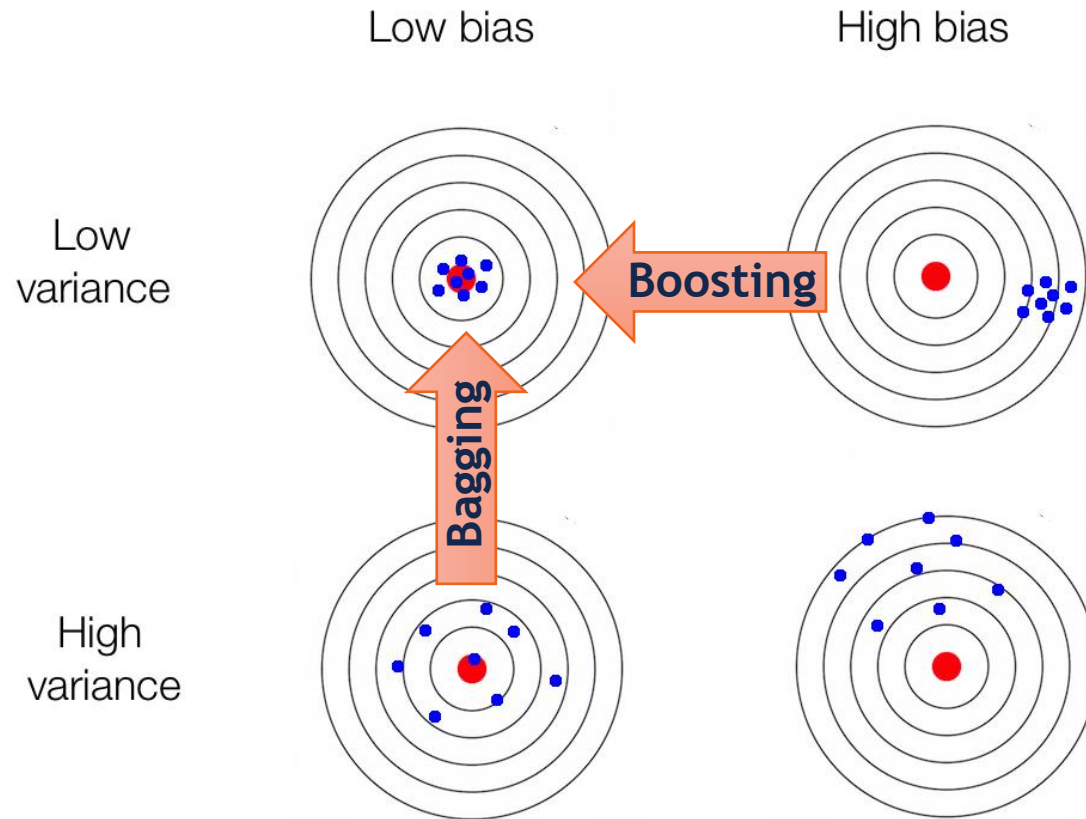
error introduced by approximating a complex problem with a simple model (underfitting)

Variance

model's sensitivity to small fluctuations in the training data (overfitting)



Ensemble methods



Boosting

- sequentially train models
- each model corrects the errors of the previous one

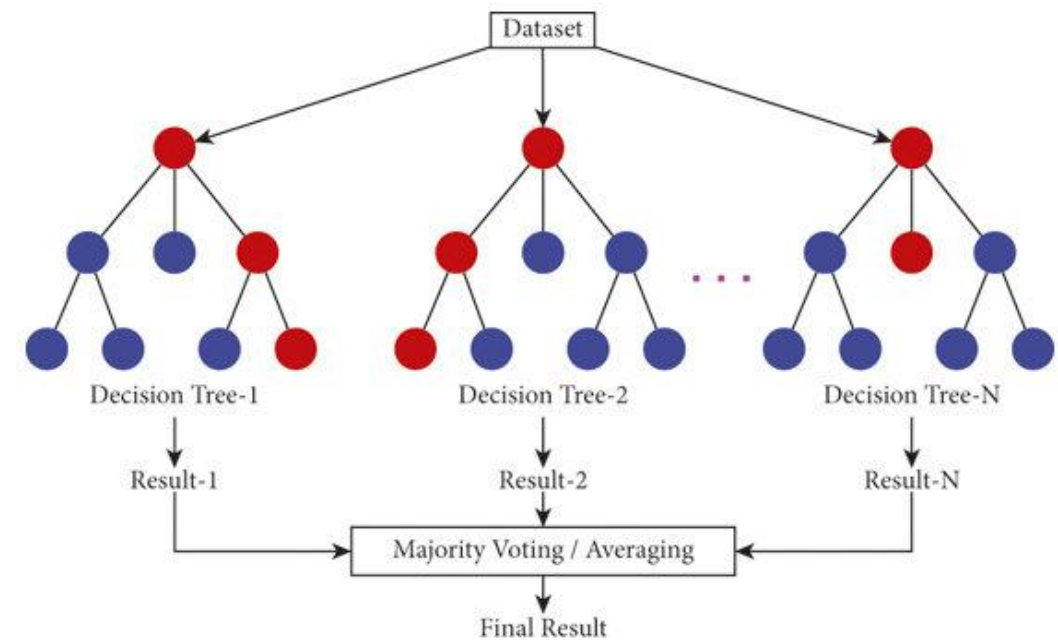
Bagging

- train multiple (small) models
- combine their predictions (averaging/majority voting)



Random forest

- Train T decision trees with low bias and high variance
 1. Select a subset of the data samples
 2. Select a subset of the features
 3. Grow a (full) tree
- **Combine the predictions**
(classification: majority voting; regression: averaging)

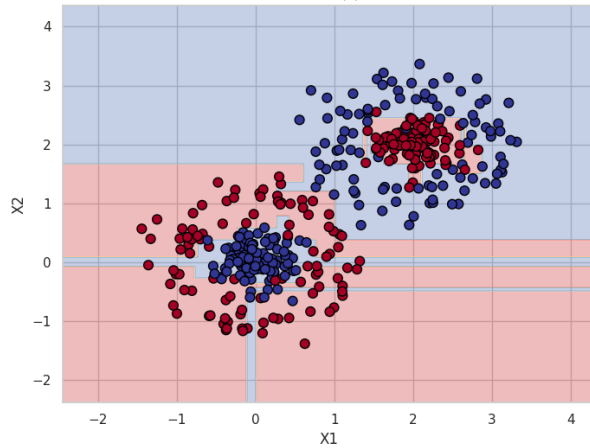




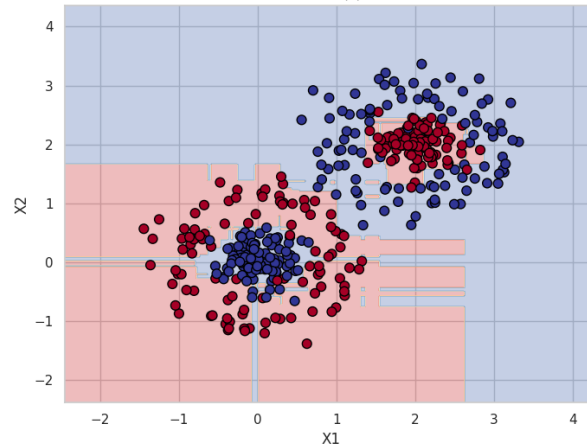
Random forest

Effect of Number of Trees on Random Forest Decision Boundary

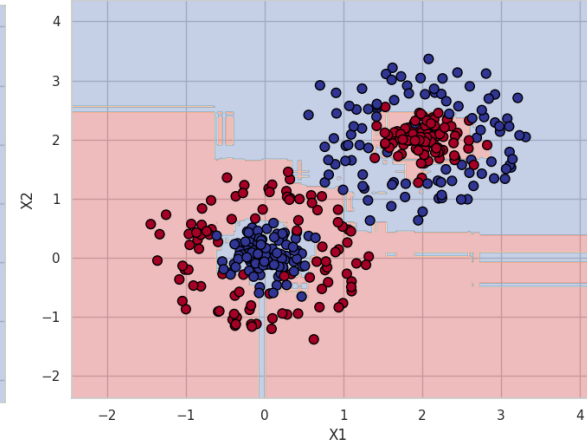
Random Forest
1 tree(s)



Random Forest
3 tree(s)



Random Forest
7 tree(s)



Random Forest
50 tree(s)

